Contents

1 字符串 2
1.1 最小表示法 2

2 数据结构 2
2.1 LCT 2
2.2 广义串并联图 3
2.3 四边形不等式 3
2.3.1 决策单调性分治 3
2.3.2 SMAWK 4
2.3.3 Wilber 4
2.3.4 二分栈 5
2.4 wqs 二分构造方案 5
2.5 O(1) RMQ 5
2.6 二区间合并 6
2.7 线性序列并查集 6
2.8 线性树上并查集 7

3 图论 8
3.1 费用流 8
3.2 支配树 9
3.3 一般图最大匹配 9
3.4 k 短路 10
3.5 拟阵交 11

4 数学 11
4.1 Barret 约简 11
4.2 拉格朗日插值 12
4.3 类欧 12
4.4 反射容斥 13
4.5 特征多项式 13
4.6 行列式多项式 13

5 数论 14
5.1 分数还原 14
5.2 O(V)-O(1) GCD 14
5.3 离散对数 14
5.4 中国剩余定理 15
5.5 Miller-Rabin 素性检验 15
5.6 Pollard-Rho 分解质因数 15
5.7 最小模线性值 16
5.8 二次剩余 16
5.9 原根与阶 16
5.10 积性函数 17
5.10.1 杜教筛 17
5.10.2 min_25 17
5.11 下取整和 18
5.12 高斯整数 18

6 多项式 19
6.1 最短线性递推 19
6.2 复数 FFT 卷积 19
6.3 NTT 20
6.4 多项式求逆 20
6.5 多项式对数 21
6.6 多项式指数 21
6.7 多项式开根 21
6.8 带余除法 21
6.9 多点求值 22
6.10 快速插值 22
6.11 线性递推第 n 项 22
6.12 多项式复合 23
6.13 多项式复合逆 23
6.14 任意模数多项式乘法 24

7 计算几何 25
7.1 选用与依赖 25
7.2 整数 25
7.2.1 整数点与精确运算 25
7.2.2 整数线段 25
7.2.3 整数面积与点包含 25
7.2.4 整数凸包与直径 26
7.2.5 整数凸包内点判定 26
7.2.6 整数最近点对 26
7.2.7 整数极角排序 27
7.2.8 整数闵可夫斯基和 27
7.3 基础 28
7.3.1 二维点与向量 28
7.3.2 直线与线段 28
7.3.3 最近点对 30
7.4 凸包 30
7.4.1 凸包、直径与裁剪 30
7.4.2 凸多边形逐点最远点 31
7.4.3 闵可夫斯基和 31
7.4.4 上凸壳 31
7.4.5 凸包内点判定 32
7.4.6 半平面交 32
7.4.7 凸包方向极值 33
7.4.8 凸包直线求交 33
7.5 多边形 33
7.5.1 面积与点包含 33
7.5.2 缠绕数包含 34
7.5.3 多边形重心 34
7.5.4 简单多边形三角剖分 34
7.6 圆 35
7.6.1 圆与内外接圆 35
7.6.2 点圆与两圆切线 35
7.6.3 圆相关求交与面积交 36
7.6.4 圆并面积与周长 37
7.6.5 最小圆覆盖 38
7.7 平面剖分 38
7.7.1 Delaunay 三角剖分 38
7.7.2 Voronoi 图 40
7.8 三维 40
7.8.1 三维向量 40
7.8.2 三维直线与线段 41
7.8.3 三维平面 42
7.8.4 三维旋转 42
7.8.5 三维凸包 43

8 通用 44
8.1 头文件 44
8.2 大质数表 45
8.3 常数速查表 45

1 字符串

1.1 最小表示法

```cpp
// 依赖无（只需序列可 size() 与下标）。
// 在 O(n) 内找最小循环表示的起点，用来判循环同构。

// a 是只读序列；返回最小循环表示的起点，空序列返回 0。
template <class T> int minrep(const T &a) {
    int n = a.size(), i = 0, j = 1, k = 0;
    while(i < n && j < n && k < n) {
        if(a[(i + k) % n] == a[(j + k) % n]) {
            ++k;
        } else {
            if(a[(i + k) % n] > a[(j + k) % n]) swap(i, j);
            j += k + 1;
            if(j == i) ++j;
            k = 0;
        }
    }
    return i;
}
// rotate(a.begin(), a.begin() + minrep(a), a.end());
```

2 数据结构

2.1 LCT

```cpp
// 依赖 通用/头文件.cpp 的 For，及全局 N 与点权 a[1..n]。
// 1..n 编号的动态森林：link/cut/路径点权异或/单点改；操作会改变辅助树形态。
namespace LCT {
#define ls(k) ch[k][0]
#define rs(k) ch[k][1]
int ch[N][2], fa[N];
bool lz[N];
int tr[N];
inline void pu(int k) {
    tr[k] = tr[ls(k)] ^ tr[rs(k)] ^ a[k];
}
inline void pl(int k) {
    if(k) lz[k] ^= 1, swap(ls(k), rs(k));
}
inline void pd(int k) {
    if(lz[k]) pl(ls(k)), pl(rs(k)), lz[k] = 0;
}
inline bool gc(int k) { return ch[fa[k]][1] == k; }
// 判断是否为辅助树根，虚父不算辅助树父亲。
inline bool ir(int k) { return ch[fa[k]][gc(k)] != k; }
void pda(int k) {
    static int st[N], t;
    st[++t] = k;
    while(!ir(k)) st[++t] = k = fa[k];
    while(t) pd(st[t--]);
}
void rot(int u) {
    int f = fa[u], g = fa[f], t = gc(u), x = ch[u][!t];
    if(!ir(f)) ch[g][gc(f)] = u;
    fa[u] = g, fa[f] = u;
    ch[u][!t] = f, ch[f][t] = x;
    if(x) fa[x] = f;
    pu(f);
}
void splay(int u) {
    for(pda(u); !ir(u); rot(u))
        if(!ir(fa[u])) {
            rot(gc(u) == gc(fa[u]) ? fa[u] : u);
        }
    pu(u);
}
// 打通根到 u0 的首选路径；结束后 u0 为辅助树根且其右儿子为空。
void access(int u0) {
    for(int u = u0, p = 0; u; p = u, u = fa[u]) splay(u), rs(u) = p, pu(u);
    splay(u0);
}
// 返回 u 所在原树的根，并将其旋到辅助树根。
int fd(int u) {
    access(u);
    while(ls(u)) u = ls(u), pd(u);
    return splay(u), u;
}
void mkr(int u) {
    access(u), pl(u);
}
void init() {
    For(i, 1, n) pu(i);
}
// 连接两棵树；u、v 已连通则不加边。
void link(int u, int v) {
    mkr(u);
    if(fd(v) != u) mkr(v), fa[v] = u;
}
// 删除已有边 u-v；要求该边存在。
void cut(int u, int v) {
    mkr(u), access(v), access(u);
    if(fa[v] == u) fa[v] = 0;
}
void upd(int u, int x) {
    a[u] = x, splay(u);
}
// 查询 u 到 v 的路径点权异或，要求两点连通。
int que(int u, int v) {
    mkr(u), access(v);
    return tr[v];
}
}  // namespace LCT
```


2

2.2 广义串并联图

```cpp
// 依赖 通用/头文件.cpp 的 For，及全局上界 N。
// 平行边合并为并联结点、反复约简度 1/2 顶点；须值初始化、m 先设为原边数，add 的边号 1..m 且无自环。
struct SPQR_tree {
    int m = 1, fa[N];
    bool typ[N];
    unordered_map<int, int> t[N];
    bitset<N> vis;
    int node(int u, int v, bool kind) {
        typ[++m] = kind;
        fa[u] = fa[v] = m;
        return m;
    }
    // 加边 u-v，编号 w；平行边合并为并联结点。
    void add(int u, int v, int w) {
        if(t[u].count(v)) {
            t[u][v] = t[v][u] = node(t[u][v], w, 0);
        } else {
            t[u][v] = t[v][u] = w;
        }
    }
    // add 完后约简 1..n 的度 1/2 顶点，修改 fa/typ/t；每张图只调用一次。
// 结果保留不可约核，不构造完整 SPQR 分解；删点的邻接表可有残留。
    void build(int n) {
        queue<int> q;
        auto ins = [&](int u) { if(!vis[u] && t[u].size() <= 2) vis[u] = 1, q.push(u); };
        For(i, 1, n) ins(i);
        while(q.size()) {
            int u = q.front();
            q.pop();
            if(t[u].empty()) continue;
            if(t[u].size() == 1) {
                int v = t[u].begin()->first;
                t[v].erase(u), ins(v);
            } else {
                int x = t[u].begin()->first, y = next(t[u].begin())->first;
                int w = node(t[u][x], t[u][y], 1);
                t[x].erase(u), t[y].erase(u);
                add(x, y, w);
                ins(x), ins(y);
            }
        }
    }
} d1;
int *fa = d1.fa, &m = d1.m;
bool *typ = d1.typ;
```

2.3 四边形不等式

区间包含单调性：区间越大权值越大，即 $a \le b \le c \le d$ 时 $w(b,c) \le w(a,d)$。

四边形不等式：两段交叉着取的代价，不会比一段包住另一段更大，即 $a \le b \le c \le d$ 时

$$w(a,d) + w(b,c) \ge w(a,c) + w(b,d)$$

令 $a = l-1$、$b = l$、$c = r$、$d = r+1$，它就等价于

$$w(l-1,r+1) + w(l,r) \ge w(l,r+1) + w(l-1,r)$$

，验性质时一般就随机代这个相邻的式子。求 max 时把 w 取负，下面都只讲 min。

1D1D DP：形如 $f_i = \min_{j<i}\{f_j + w(j,i)\}$，记 p_i 为让 f_i 取到最小的 j。w 满足四边形不等式时 p_i 单调不降。理由：对 $i<j<k<l$，四边形不等式等价于

$$w(j,l) - w(j,k) \le w(i,l) - w(i,k)$$

，也就是说 j 在 k 处比 i 优，那它在 l 处也一定还比 i 优 —— 决策点只会往右挪。

- w 能 $O(1)$ 算：从前往后用单调栈维护每段的决策点、二分分界，就是二分栈，$O(n \log n)$ 次 w；
- p 与 f 无关（离线）时：分治，同样 $O(n \log n)$；SMAWK 做到 $O(n)$，配合 Wilber 还能在线 $O(n)$。

区间 DP：形如 $f_{l,r} = \min_{l \le i < r}\{f_{l,i} + f_{i+1,r}\} + w(l,r)$。$w$ 同时满足区间包含单调性与四边形不等式时，f 也满足四边形不等式，最优决策点满足

$$p_{l,r-1} \le p_{l,r} \le p_{l+1,r}$$

，每个区间只在这对上下界之间枚举，总共 $O(n^2)$。

2D1D DP：形如 $f_{i,j} = \min_{k<j}\{f_{i-1,k} + w(k,j)\}$。$w$ 满足四边形不等式时 $f_{i,j}$ 关于 i 下凸，可以 WQS 二分掉一维，化成一串 1D1D（见 wqs 一页）。

好用的判据：只跟区间长度有关、且关于长度下凸的代价（如 $w(l,r) = (r-l+1)^2$）一定满足四边形不等式；邮局问题里“一段村庄到一个中位数的距离和”也是。拿不准就按上面那个相邻式子随机对拍。

2.3.1 决策单调性分治

```cpp
// 依赖 通用/头文件.cpp 的 For/ll，及使用者提供的窗口类型 W。
// 在线 1D1D 分治，代价用两个滑动窗口各自维护，每步 O(1) 搬运。

ll f[N];
int p[N];
struct wcur {
    W &w;
    int l, r;
    void ml(int x) { while(l > x) w.add(l--); while(l < x) w.del(++l); }
    void mr(int x) { while(r < x) w.add(++r); while(r > x) w.del(r--); }
    void rs() { w.clear(), l = r = 0; }
};
W Wa, Wb;
wcur s{Wa, 0, 0}, t{Wb, 0, 0};
void relax(ll w, int i, int k) {
```

```
    ll v = f[k] + w;
    if(v < f[i]) f[i] = v, p[i] = k;
}
void dnc(int l, int r) {
    if(r - l == 1) return;
    int m = (l + r) >> 1;
    s.mr(m), s.ml(p[l]);
    For(k, p[l], p[r]) {
        relax(Wa.w, m, k);
        if(k < p[r]) s.ml(k + 1);
    }
    s.ml(p[l]), s.mr(l);
    dnc(l, m);
    t.mr(r), t.ml(l + 1);
    For(k, l + 1, m) {
        relax(Wb.w, r, k);
        if(k < m) t.ml(k + 1);
    }
    t.ml(m), t.mr(m);
    dnc(m, r);
}
// W 需提供 ll w 与 void add(int)/del(int)/clear()，模板开 Wa、Wb 两份独立实例。
// 决策单调时算 f[0..n]、p[0..n]；先清空窗口，可重复调用；中间和在 ll 内。
void work(int n) {
    s.rs(), t.rs();
    f[0] = 0, p[0] = 0;
    if(!n) return;
    For(i, 1, n) f[i] = (ll) 4e18, p[i] = 0;
    s.mr(n), s.ml(0), relax(Wa.w, n, 0), s.ml(0), s.mr(0);
    dnc(0, n);
}
```

2.3.2 SMAWK

```
// 依赖 通用/头文件.cpp 的 For，及使用者提供的全单调矩阵 que(i,j)。
// 全单调矩阵每行最小值，O(n+m)；行列均 1-based，返回 f 与最小列 fp。
template <typename T, typename Q> pair<vector<T>, vector<int>> SMAWK(int n0, int m,
Q &&que) {
    vector<T> f(n0 + 1, T());
    vector<int> fp(n0 + 1, 0), b(m + 1, 0);
    For(i, 1, m) b[i] = i;
    auto F = [&](auto &&F, int d, int m) -> void {
        int n = (n0 >> d), k = 1, j = 1;
        if(!n) return;
        if(n < m) {
            while(j < m) {
                if(que(k << d, b[j]) > que(k << d, b[j + 1])) {
                    if(k == 1) ++j;
                    else b[j] = b[--k];
                } else {
                    if(k < n) b[k++] = b[j];
                    else b[j + 1] = b[j];
                    ++j;
                }
            }
            b[k] = b[m];
            m = k;
            j = 1;
        }
        vector<int> b2(b.begin(), b.begin() + min<int>(m + 1, (int) b.size())), b =
move(b2);
        F(F, d + 1, m);
        For(i, 1, n) if(i % 2 == 1) {
            T &s = f[i << d];
            int &p = fp[i << d];
            p = b[j];
            s = que(i << d, p);
            while(j < m && (i == n || b[j + 1] <= fp[(i + 1) << d])) {
                T s2 = que(i << d, b[++j]);
                if(s2 < s) s = s2, p = b[j];
            }
        }
    };
    F(F, 0, m);
    return {f, fp};
}
```

2.3.3 Wilber

```
// 依赖 数据结构/四边形不等式/SMAWK.cpp 与使用者提供的 que(k,i)。
// 在线 1D1D 的线性决策单调 DP，0 <= k < i <= n；返回 f 与前驱 fp。
template <typename T, int B = 16, typename Q>
pair<vector<T>, vector<int>> Wilber(int n, Q &&que, const T Z =
numeric_limits<T>::max()) {
    vector<T> f(n + 1, Z);
    f[0] = 0;
    vector<int> fp(n + 1, 0);
    int c = 0;
    while(c < n) {
        int r = fp[c], p = min(n, c + (c - r + 1));
        if(c - r + 1 <= B) {
            For(j, c + 1, p) For(i, fp[j - 1], j - 1) {
                T v = f[i] + que(i, j);
                if(v < f[j]) f[j] = v, fp[j] = i;
            }
            c = p;
            continue;
        }
        auto q1 = [&](int x, int y) { return f[y + r - 1] + que(y + r - 1, x +
c); };
        auto [g, gp] = SMAWK<T>(p - c, c - r + 1, q1);
        For(i, c + 1, p) f[i] = g[i - c], fp[i] = gp[i - c] + r - 1;
        auto q2 = [&](int x, int y) { return x > y ? f[y + c] + que(y + c, x + c) :
Z; };
        auto [h, hp] = SMAWK<T>(p - c, p - c, q2);
        int nc = p;
```

```
        For(i, c + 1, p) {
            if(f[i] > h[i - c]) {
                f[i] = h[i - c];
                fp[i] = hp[i - c] + c;
                nc = i;
                break;
            }
        }
        c = nc;
    }
    return {f, fp};
}
```

2.3.4 二分栈

```
// 依赖全局 n、数组上限 N 与使用者提供的 calc(l,r)。
// 二分栈做决策单调 DP；每段代价减惩罚值后求最优，配 WQS 用。

struct nd {
    ll x;
    int c;
    bool operator<(const nd b) const {
        return x < b.x || (x == b.x && c < b.c);
    }
    nd operator+(const nd b) const { return {x + b.x, c + b.c}; }
};
// calc(l,r) 是 [l,r] 代价，须满足决策单调；中间代价须在 ll 内。
// 每段代价减 dt，返回最优(总代价,段数)，同价取较少段；n 已设置，可重复调用。
nd work(ll dt) {
    static nd f[N];
    deque<pii> q;
    f[0] = {0, 0}, q.push_back({n, 0});
    auto get = [&](int j, int i) -> nd {
        return f[j] + nd{calc(j + 1, i) - dt, 1};
    };
    For(i, 1, n) {
        f[i] = get(q.front()[1], i);
        if(q.front()[0] <= i) q.pop_front();
        while(q.size()) {
            auto [R, p] = q.back(); q.pop_back();
            int L = q.size() ? q.back()[0] + 1 : i + 1;
            if(L > R) continue;
            if(!(get(p, L) < get(i, L))) continue;
            int l = L, r = R + 1;
            while(l + 1 < r) {
                int m = (l + r) >> 1;
                if(get(p, m) < get(i, m)) l = m;
                else r = m;
            }
            q.push_back({l, p});
            break;
        }
        q.push_back({n, i});
    }
    }
    return f[n];
}
```

2.4 wqs 二分构造方案

```
// 依赖 数据结构/四边形不等式/Wilber.cpp 与 i128。
// 外层二分惩罚值、内层 Wilber 做 DP，构造恰好 k 段的最优划分；OPT(k) 须关于 k 凸。

// 返回以 0 开始、n 结束的划分边界序列；que(l,r) 是 (l,r] 的代价。
// 要求 n >= 1、1 <= k <= n、up >= 0 且二分范围 -up..1 覆盖所需惩罚值。
template <typename Q> vector<int> wqs(int n, int k, Q &&que, i128 up) {
    int B = __lg(n) + 1;
    ll T = (1ll << B) - 1;
    auto chk = [&](i128 dt) {
        return Wilber<i128>(
            n, [&](int l, int r) { return (que(l, r) - dt) * ((i128)1 << B) + 1; },
1e36);
    };
    auto get = [&](i128 dt) {
        auto [f, fp] = chk(dt);
        vector<int> pt;
        for(int i = n; i; i = fp[i]) pt.push_back(i);
        pt.push_back(0);
        reverse(all(pt));
        return pt;
    };
    i128 l = -up - 1, r = 1;
    while(l + 1 < r) {
        i128 m = l + r >> 1;
        if((chk(m).first[n] & T) < k) l = m;
        else r = m;
    }
    auto fp = get(l), gp = get(l + 1);
    if(gp.size() - 1 == k) return gp;
    int j = 1;
    For(i, 1, (int)gp.size() - 2) {
        while(fp[j] < gp[i]) ++j;
        if(fp[j - 1] <= gp[i - 1] && fp.size() - j + i - 1 == k) {
            vector<int> r(gp.begin(), gp.begin() + i);
            r.insert(r.end(), fp.begin() + j, fp.end());
            return r;
        }
    }
    assert(0);
}
```

2.5 O(1) RMQ

```
// 依赖 通用/头文件.cpp 的 For，及模板参数 T、cmp。
// O(n log n) 预处理后 O(1) 闭区间最值，下标从 dt 开始。
template <typename T, typename cmp = less<T>, int dt = 1>
```

```cpp
struct rmq_t {
    static constexpr int B = 20;
    vector<T> a, pre, suf;
    vector<vector<T>> st;
    vector<int> f;
    // 查询块编号闭区间 [L,R]，块号从 0 开始。
    T _que(int L, int R) {
        int k = __lg(R - L + 1);
        return max(st[k][L], st[k][R - (1 << k) + 1], cmp());
    }
    // 查询第 k 块内的 0-based 闭区间 [L,R]。
    T _que_bitc(int k, int L, int R) {
        int l = k * B, x = f[R] & ~((1 << (L - l)) - 1);
        return a[l + __lg(x & -x)];
    }
    // 从 _a[dt..dt+n-1] 构建，可重复调用；n >= 1。
    void bd(int n, T *_a) {
        a.resize(n), pre.resize(n), suf.resize(n), f.resize(n);
        For(i, 0, n - 1) pre[i] = suf[i] = a[i] = _a[i + dt];
        int k = (n - 1) / B;
        st.resize(__lg(k + 1) + 1), st[0].resize(k + 1);
        For(i, 0, k) {
            int l = i * B, r = min(i * B + B - 1, n - 1);
            rFor(i, r - 1, l) pre[i] = max(pre[i], pre[i + 1], cmp());
            For(i, l + 1, r) suf[i] = max(suf[i], suf[i - 1], cmp());
            st[0][i] = suf[r];
            static int b[B + 1];
            int t = 0, s = 0;
            For(i, l, r) {
                while(t && cmp()(a[b[t]], a[i])) s ^= 1 << (b[t--] - l);
                f[i] = s ^= 1 << (i - l);
                b[++t] = i;
            }
        }
        For(i, 1, __lg(k + 1)) {
            int m = k - (1 << i) + 1;
            st[i].resize(m + 1);
            For(j, 0, m) st[i][j] = max(st[i - 1][j], st[i - 1][j + (1 << (i - 1))], cmp());
        }
    }
    // 从非空 v 构建，查询下标仍从 dt 开始。
    void bd(vector<T> &v) { bd(v.size(), v.data() - dt); }
    // 从 func(dt..dt+n-1) 构建；n >= 1。
    template <class F> void bd(int n, F func) {
        vector<T> v(n);
        For(i, 0, n - 1) v[i] = func(i + dt);
        bd(v);
    }
    // bd 后查询 [l,r]；dt <= l <= r < dt+n。
    T que(int l, int r) {
        l -= dt, r -= dt;
        int il = l / B, ir = r / B;
        if(il == ir) {
            return _que_bitc(il, l, r);
        } else {
            T s = max(pre[l], suf[r], cmp());
            if(il + 1 < ir) s = max(s, _que(il + 1, ir - 1), cmp());
            return s;
        }
    }
};
```

2.6 二区间合并

```cpp
// 依赖 通用/头文件.cpp 的 For，及模板参数 T、op。
// O(n log n) 预处理后任意 0-based 闭区间最多合并一次；op 只需结合律，T 可复制。
template <class T, class F>
struct disjoint_sparse_t {
    vector<T> a;
    vector<vector<T>> st;
    F op;
    disjoint_sparse_t(const vector<T> &v, F f) : op(f) { build(v); }
    void build(const vector<T> &v) {
        a = v;
        int n = a.size(), h = n > 1 ? __lg(n - 1) + 1 : 0;
        st.assign(h, a);
        For(k, 0, h - 1) {
            int m = 1 << k;
            for(int l = 0; l < n;) {
                int mid = l + min(m, n - l), r = mid + min(m, n - mid);
                rFor(i, mid - 2, l) st[k][i] = op(a[i], st[k][i + 1]);
                For(i, mid + 1, r - 1) st[k][i] = op(st[k][i - 1], a[i]);
                l = r;
            }
        }
    }
    // 0<=l<=r<n。
    T query(int l, int r) const {
        if(l == r) return a[l];
        int k = __lg(l ^ r);
        return op(st[k][l], st[k][r]);
    }
};
```

2.7 线性序列并查集

```cpp
// 依赖 通用/头文件.cpp 的 For 与 u64 位图。
// 长度 n 的序列只支持向右收缩：merge_right(i) 并掉原边 i->i+1，find(x) 返回分量右端；总 O(n+q)、空间 O(n/log n)。
struct sequence_dsu_t {
    int n, w;
    vector<u64> s;
    vector<int> fa, rk, r;
    void init(int m) {
```

```
        n = m, w = n ? __lg(n) + 1 : 1;
        int k = n ? (n - 1) / w + 1 : 0;
        s.resize(k), fa.resize(k), rk.assign(k, 0), r.resize(k);
        For(i, 0, k - 1) {
            fa[i] = r[i] = i;
            s[i] = (1ULL << min(w, n - i * w)) - 1;
        }
    }
    int get(int x) { return fa[x] == x ? x : fa[x] = get(fa[x]); }
    void join(int x, int y) {
        x = get(x), y = get(y);
        int t = r[y];
        if(rk[x] < rk[y]) swap(x, y);
        fa[y] = x, r[x] = t;
        if(rk[x] == rk[y]) ++rk[x];
    }
    // 0<=i<n-1；重复收缩返回 false。
    bool merge_right(int i) {
        int b = i / w;
        u64 z = 1ULL << (i % w);
        if(!(s[b] & z)) return false;
        s[b] ^= z;
        if(!s[b]) join(b, b + 1);
        return true;
    }
    // 0<=x<n；最后一点不删除。
    int find(int x) {
        int b = x / w;
        u64 z = s[b] & (~0ULL << (x % w));
        if(z) return b * w + __builtin_ctzll(z);
        b = r[get(b + 1)];
        return b * w + __builtin_ctzll(s[b]);
    }
};
```

2.8 线性树上并查集

```
// 依赖 通用/头文件.cpp 的 For 与 u64 位图。
// 静态树上向上收缩：merge_parent(v) 并掉原父边、根不动；init O(n)，q 次操作总 O(n+q)。
struct tree_dsu_t {
    int rt, w;
    vector<int> bel, id, st, a;  // 簇号、位号、起点、点号
    vector<int> up, mp, fa, rk, top;
    vector<u64> mask, live;
    void init(const vector<int> &p) {
        int n = p.size(), lg = n ? __lg(n) + 1 : 1;
        rt = -1, w = (lg + 1) / 2;
        bel.resize(n), id.resize(n), mask.resize(n), mp.assign(n, -1);
        st.clear(), a.clear(), up.clear(), live.clear();
        vector<int> son(n, -1), bro(n, -1), ord;
        For(v, 0, n - 1) {
            if(p[v] == -1) rt = v;
            else bro[v] = son[p[v]], son[p[v]] = v;
        }
        if(n) ord.push_back(rt);
        For(i, 0, n - 1) {
            for(int u = son[ord[i]]; u != -1; u = bro[u]) ord.push_back(u);
        }
        vector<int> l(n), r(n), sz(n), nxt(n, -1);
        auto add = [&](int l0, int u) {
            int c = live.size(), cnt = 0;
            st.push_back(a.size()), up.push_back(u);
            for(int v = l0; v != -1; v = nxt[v]) {
                bel[v] = c, id[v] = cnt++;
                a.push_back(v);
            }
            live.push_back((1ULL << cnt) - 1);
        };
        rFor(i, n - 1, 0) {
            int v = ord[i], l0 = -1, r0 = -1, cnt = 0;
            for(int u = son[v]; u != -1; u = bro[u]) {
                if(cnt) nxt[r0] = l[u];
                else l0 = l[u];
                r0 = r[u], cnt += sz[u];
                if(cnt >= w) {
                    add(l0, v);
                    l0 = r0 = -1, cnt = 0;
                }
            }
            nxt[v] = l0, l[v] = v;
            r[v] = cnt ? r0 : v, sz[v] = cnt + 1;
        }
        if(n) add(rt, -1);
        for(int v : ord) {
            mask[v] = 1ULL << id[v];
            if(p[v] != -1 && bel[v] == bel[p[v]]) mask[v] |= mask[p[v]];
        }
        int k = 0;
        for(int u : up)
            if(u != -1 && mp[u] == -1) mp[u] = k++;
        if(n && mp[rt] == -1) mp[rt] = k++;
        fa.resize(k), rk.assign(k, 0), top.resize(k);
        For(v, 0, n - 1) if(mp[v] != -1) {
            int x = mp[v];
            fa[x] = x, top[x] = v;
        }
    }
    int get(int x) { return fa[x] == x ? x : fa[x] = get(fa[x]); }
    // 0<=v<n；根或重复收缩返回 false。
    bool merge_parent(int v) {
        u64 s = 1ULL << id[v];
        if(v == rt || !(live[bel[v]] & s)) return false;
        live[bel[v]] ^= s;
        return true;
    }
```

```
    // 0<=v<n。
    int find(int v) {
        u64 s = mask[v] & live[bel[v]];
        if(s) return a[st[bel[v]] + 63 - __builtin_clzll(s)];
        int x = get(mp[up[bel[v]]]);
        while(true) {
            int u = top[x];
            s = mask[u] & live[bel[u]];
            if(s) return a[st[bel[u]] + 63 - __builtin_clzll(s)];
            int y = get(mp[up[bel[u]]]), t = top[y];
            if(rk[x] < rk[y]) swap(x, y);
            fa[y] = x, top[x] = t;
            if(rk[x] == rk[y]) ++rk[x];
        }
    }
};
```

3 图论

3.1 费用流

```
// 依赖 通用/头文件.cpp 的 For/cmax/min、ll 与本类的上界 N、无穷 Z。
// 1..n 编号的最小费用增广路；容量为非负 int、费用为 ll，求流修改残量网络。

struct MCMF_t {
    static constexpr int N = 1e6 + 10, FL = 1e9;
    static constexpr ll Z = 1e18;
    struct edge {
        int v, n, w;
        ll c;
    } e[N];
    int n, mx, hd[N], tot = 1, s, t, p[N];
    void _add(int u, int v, int w, ll c) {
        e[++tot] = {v, hd[u], w, c}, hd[u] = tot;
    }
    // 加有向费用容量边与反向残量边，保留平行边。
    void add(int u, int v, int w, ll c) {
        cmax(mx, u), cmax(mx, v);
        _add(u, v, w, c), _add(v, u, 0, -c);
    }
    // 清空图、边池与源汇；mx 留着当历次用点的清零上界。
    void clear() {
        For(i, 1, mx) hd[i] = 0;
        tot = 1, s = t = 0;
    }
    ll h[N], d[N];
    // 按已设的 s,n 做一次 SPFA，结果写进初始势能 h[1..n]。
    void spfa() {
        static bool vis[N];
        For(i, 1, n) h[i] = Z;
        queue<int> q;
        q.push(s), h[s] = 0, vis[s] = 1;
        while(q.size()) {
            int u = q.front();
            q.pop();
            vis[u] = 0;
            for(int i = hd[u]; i; i = e[i].n) {
                int v = e[i].v;
                if(e[i].w && h[u] + e[i].c < h[v]) {
                    h[v] = h[u] + e[i].c;
                    if(!vis[v]) q.push(v), vis[v] = 1;
                }
            }
        }
    }
    // h 有效时找最短增广路。
    // 更新 d/p/h，返回 t 是否可达。
    bool dij() {
        For(i, 1, n) d[i] = Z;
        priority_queue<pair<ll, int>> q;
        d[s] = 0;
        q.emplace(0, s);
        while(q.size()) {
            auto [tmp, u] = q.top();
            q.pop();
            if(-tmp != d[u]) continue;
            for(int i = hd[u]; i; i = e[i].n)
                if(e[i].w) {
                    int v = e[i].v;
                    auto nd = d[u] + h[u] - h[v] + e[i].c;
                    if(nd < d[v]) d[v] = nd, p[v] = i, q.emplace(-nd, v);
                }
        }
        For(i, 1, n) if(d[i] != Z) d[i] += h[i], h[i] = d[i];
        return d[t] != Z;
    }
    // 追加流：返回 (流量, 费用)，mcf=1 时只作负费用增广。
    // 须显式传 _n；要求无负环、总流量在 int 内、有限距离 < Z。
    pair<int, ll> mcmf(int _s, int _t, bool mcf = 0, int _n = 0) {
        s = _s, t = _t, n = _n, cmax(mx, _n);
        spfa();
        int a1 = 0;
        ll a2 = 0;
        while(dij() && (!mcf || d[t] < 0)) {
            int m = FL;
            for(int u = t; u != s; u = e[p[u] ^ 1].v) m = min(m, e[p[u]].w);
            a1 += m, a2 += m * d[t];
            for(int u = t; u != s; u = e[p[u] ^ 1].v) e[p[u]].w -= m, e[p[u] ^ 1].w
 += m;
        }
        return {a1, a2};
    }
    // 带下界/负费用时先饱和负费用边再求流，返回 (流量, 费用)。
    // 须传 _n 且 _n + 2 < N；实例须零初始化。
    pair<int, ll> mcmf2(int _s, int _t, bool mcf = 0, int _n = 0) {
```

```
        s = _s, t = _t, n = _n, cmax(mx, _n + 2);
        static int ex[N];
        For(i, 1, _n) ex[i] = 0;
        int a1 = 0; ll a2 = 0;
        For(i, 2, tot) if(i % 2 == 0 && e[i].c < 0) {
            int u = e[i ^ 1].v, v = e[i].v;
            swap(e[i].w, e[i ^ 1].w);
            ex[u] -= e[i ^ 1].w, ex[v] += e[i ^ 1].w;
            a2 += e[i ^ 1].w * e[i].c;
        }
        For(i, 1, _n) { ex[i] > 0 ? add(_n + 1, i, ex[i], 0) : add(i, _n + 2, -
   ex[i], 0);
        }
        add(_t, _s, FL, 0);
        int aux = tot, n0 = n;
        a2 += mcmf(_n + 1, _n + 2, 0, _n + 2).second;
        a1 += e[aux].w;
        tot = aux - 2 * _n - 2, n = n0;
        For(i, 1, n0) while(hd[i] > tot) hd[i] = e[hd[i]].n;
        auto [a3, a4] = mcmf(_s, _t, mcf, _n);
        return {a1 + a3, a2 + a4};
    }
};
```

3.2 支配树

```
// 依赖 通用/头文件.cpp 的 For/rFor/cmin、pii 与命名空间内的 N。
// 1 为根的有向图支配树，O((n+m)α)；点编号 1..n，不可达点 dm=0。

namespace DOM {

const int N = 1e6 + 10;
int n;
vector<int> t[N], t2[N], q1[N];
int fa[N], f1[N], dfn[N], pos[N], dt, sd[N], dm[N];
pii f[N];
void add(int u, int v) {
    t[u].push_back(v), t2[v].push_back(u);
}
void dfs(int u) {
    dfn[u] = ++dt, pos[dt] = u;
    for(auto v : t[u]) if(!dfn[v]) dfs(v), f1[v] = u;
}
// 返回并查集链上最小的(半支配点 DFS 序,点)，并压缩路径。
pii que(int u) {
    if(fa[u] == u) return f[u];
    cmin(f[u], que(fa[u]));
    fa[u] = fa[fa[u]];
    return f[u];
}
// 求当前图的直接支配点 dm[1..n]，可重复调用。
// 调用方维护并按需清空 t/t2；getdom 自行重置算法状态；需足够递归栈。
void getdom(int _n) {
    n = _n;
    For(i, 1, n) q1[i].clear(), fa[i] = f1[i] = dfn[i] = pos[i] = sd[i] = dm[i] = 0;
    dt = 0;
    dfs(1);
    For(i, 1, n) fa[i] = i, f[i] = {dfn[i], 0};
    rFor(i, dt, 1) {
        int u = pos[i], s = i;
        for(auto x : q1[u]) dm[x] = que(x)[1];
        if(i == 1) break;
        for(auto x : t2[u]) if(dfn[x]) cmin(s, que(x)[0]);
        sd[u] = pos[s], q1[sd[u]].push_back(u), f[u] = {dfn[sd[u]], u};
        for(auto v : t[u]) if(f1[v] == u) fa[v] = u;
    }
    For(i, 2, dt) {
        int u = pos[i];
        dm[u] = sd[dm[u]] == sd[u] ? sd[u] : dm[dm[u]];
    }
}

}  // namespace DOM
```

3.3 一般图最大匹配

```
// 依赖外部上界 N、全局 n 与邻接表 t，及 通用/头文件.cpp 的 For。
// 带花树：一次调用就某点增广一次，外层遍历未匹配点即得最大匹配。

int n, fa[N];
vector<int> t[N];
int fd(int u) { return fa[u] == u ? u : fa[u] = fd(fa[u]); }
// 从 1..n 中选一个未匹配点 rt 尝试增广。
// 匹配保存在函数内静态 mat，仅支持单图构建；外层要对每个未匹配点各调一次。
void getmat(int rt) {
    static int vis[N], mat[N], pr[N];
    For(i, 1, n) vis[i] = pr[i] = 0, fa[i] = i;
    queue<int> q;
    auto lca = [](int x, int y) {
        static int mk[N], tm; ++tm;
        while(1) {
            x = fd(x);
            if(x && exchange(mk[x], tm) == tm) return x;
            x = pr[mat[x]], swap(x, y);
        }
        return -1;
    };
    auto blossom = [&](int x, int y, int z) {
        for(; fd(x) != z; x = pr[y]) {
            pr[x] = y, y = mat[x], fa[x] = fa[y] = z;
            if(vis[y] == 1) vis[y] = 2, q.push(y);
        }
    };
    vis[rt] = 2, q.push(rt);
```

```
    while(q.size()) {
        int u = q.front(); q.pop();
        for(auto v : t[u]) {
            if(fd(u) == fd(v)) continue;
            if(!vis[v]) pr[v] = u, vis[v] = 1, vis[mat[v]] = 2, q.push(mat[v]);
            if(!mat[v]) {
                while(v) mat[v] = pr[v], swap(mat[pr[v]], v);
                return;
            }
            if(vis[v] == 2) {
                int l = lca(u, v);
                blossom(u, v, l), blossom(v, u, l);
            }
        }
    }
}
```

3.4 k 短路

```
// 依赖 通用/头文件.cpp 的 ll、i128 与 LLONG_MAX。
// 0-based 有向非负权图按边序列计重的 k 短路，允许重边、重游与零环；O((n+m)log m + k log k)。

struct k_shortest_t {
    struct edge {
        int u, v;
        ll w;
    };
    struct node {
        ll w;
        int v, l, r, d;
    };
    vector<edge> e;
    vector<vector<int>> g, rg;
    vector<node> h;
    // 清空图，按点数 n 重建两份邻接表。
    void init(int n) {
        g.assign(n, {}), rg.assign(n, {});
        e.clear(), h.clear();
    }
    void add(int u, int v, ll w) {
        int i = e.size();
        e.push_back({u, v, w});
        g[u].push_back(i), rg[v].push_back(i);
    }
    // 递归中 h 会扩容，调用方不能跨调用持有 h 元素的引用。
    int meld(int a, int b) {
        if(!a || !b) return a + b;
        if(h[a].w > h[b].w) swap(a, b);
        int x = h.size();
        h.push_back(h[a]);
        int r = meld(h[x].r, b);
        h[x].r = r;
        if(h[h[x].l].d < h[h[x].r].d) swap(h[x].l, h[x].r);
        h[x].d = h[h[x].r].d + 1;
        return x;
    }
    // 返回从 s 到 t 的前 k 个游走长度（升序）；s=t 时空游走计第一个。
    // 只枚举长度 < LLONG_MAX 的游走；k <= 0 返回空；可换起终点重复调用。
    vector<ll> solve(int s, int t, int k) {
        vector<ll> ans;
        h.assign(1, {});
        if(k <= 0) return ans;
        int n = g.size();
        const ll inf = LLONG_MAX;
        vector<ll> d(n, inf);
        vector<int> pre(n, -1), ord, rt(n);
        using pli = pair<ll, int>;
        priority_queue<pli, vector<pli>, greater<pli>> q;
        d[t] = 0, q.push({0, t});
        while(!q.empty()) {
            auto [du, u] = q.top();
            q.pop();
            if(du != d[u]) continue;
            ord.push_back(u);
            for(int i : rg[u]) {
                auto a = e[i];
                i128 nd = (i128)du + a.w;
                if(nd < d[a.u]) {
                    d[a.u] = nd, pre[a.u] = i;
                    q.push({(ll)nd, a.u});
                }
            }
        }
        if(d[s] == inf) return ans;
        for(int u : ord) {
            if(pre[u] != -1) rt[u] = rt[e[pre[u]].v];
            for(int i : g[u]) {
                auto a = e[i];
                if(i == pre[u] || d[a.v] == inf) continue;
                i128 w = (i128)a.w + d[a.v] - d[u];
                if(w >= inf) continue;
                int x = h.size();
                h.push_back({(ll)w, a.v, 0, 0, 1});
                rt[u] = meld(rt[u], x);
            }
        }
        auto push = [&](i128 w, int x) {
            if(x && w < inf) q.push({(ll)w, x});
        };
        ans.push_back(d[s]);
        push((i128)d[s] + h[rt[s]].w, rt[s]);
        while((int)ans.size() < k && !q.empty()) {
            auto [w, x] = q.top();
            q.pop();
            ans.push_back(w);
```


10

```
            for(int y : {h[x].l, h[x].r}) push((i128)w + h[y].w - h[x].w, y);
            int y = rt[h[x].v];
            push((i128)w + h[y].w, y);
        }
        return ans;
    }
};
```

3.5 拟阵交

```
// 依赖 通用/头文件.cpp 的 For。
// 求两个拟阵的最大公共独立集，地集 [0,n)；O(n(r+1)^2) 次 oracle 调用，空间 O(n(r+1))。

// a(s)、b(s) 判断 s 是否独立，须为同一地集上的拟阵且不得改 s。
// 返回最大公共独立集的元素编号；每次从空集开始，可重复调用。
template <class A, class B>
vector<int> matroid_intersection(int n, A a, B b) {
    vector<int> in(n);
    while(true) {
        vector<int> s, pre(n, -1), ok(n);
        vector<vector<int>> g(n);
        queue<int> q;
        For(x, 0, n - 1) if(in[x]) s.push_back(x);
        For(x, 0, n - 1) if(!in[x]) {
            s.push_back(x);
            if(a(s)) pre[x] = n, q.push(x);
            ok[x] = b(s);
            s.pop_back();
            for(int &y : s) {
                int v = y;
                y = x;
                if(a(s)) g[v].push_back(x);
                if(b(s)) g[x].push_back(v);
                y = v;
            }
        }
        int t = -1;
        while(!q.empty()) {
            int x = q.front();
            q.pop();
            if(ok[x]) {
                t = x;
                break;
            }
            for(int y : g[x])
                if(pre[y] == -1) pre[y] = x, q.push(y);
        }
        if(t == -1) return s;
        for(int x = t; x != n; x = pre[x]) in[x] ^= 1;
    }
}

```

```
// 划分拟阵：c[x] 为元素类别，lim[c]>=0 为该类容量。
// 单次 oracle 时间 O(类数 + |s|)，空间 O(类数)。
struct partition_matroid_t {
    vector<int> c, lim;
    bool operator()(const vector<int> &s) const {
        vector<int> cnt(lim.size());
        for(int x : s)
            if(++cnt[c[x]] > lim[c[x]]) return false;
        return true;
    }
};

// 图拟阵：顶点 [0,n)，e[x] 为无向边；允许重边、自环。
// 单次 oracle 时间 O(n+|s| log(n+1))，空间 O(n)。
struct graphic_matroid_t {
    int n;
    vector<pii> e;
    bool operator()(const vector<int> &s) const {
        vector<int> fa(n, -1);
        auto find = [&](int x) {
            while(fa[x] >= 0) x = fa[x];
            return x;
        };
        for(int x : s) {
            int u = find(e[x][0]), v = find(e[x][1]);
            if(u == v) return false;
            if(fa[u] > fa[v]) swap(u, v);
            fa[u] += fa[v], fa[v] = u;
        }
        return true;
    }
};
```

4 数学

4.1 Barret 约简

```
// 依赖 通用/头文件.cpp 的 u64、u128。
// 用 128 位乘取高 64 位做 const 模约简，比取模快；构造时算一次倒置。

struct barret {
    u128 w; u64 M;
    // 构造：要求 2 <= M <= 2^64-1；O(1)。
    barret(const u64 M) : M(M), w(u64((u128(1) << 64) / M)) {}
    // 返回未归一余数（x % M 或 x % M + M），省一次条件减法。
    inline u64 mod_without_chk(u64 x) const { return x - ((w * x) >> 64) * M; }
    // 返回 [0,M) 内的余数。
    inline u64 mod(u64 x) const {
        u64 r = mod_without_chk(x);
        return r >= M ? r - M : r;
    }
};
```

4.2 拉格朗日插值

已知 $n+1$ 个点 $(x_0,y_0),(x_1,y_1),...,(x_n,y_n)$，其中 x_i 互不相同，则存在唯一的次数不超过 n 的多项式 $L(x)$ 满足 $L(x_i)=y_i$。

Lagrange 插值多项式为：

$$L(x)=\sum_{i=0}^{n}y_i\prod_{j=0,j\neq i}^{n}\frac{x-x_j}{x_i-x_j}$$

当 x_i 是连续整数（如 $0,1,...,n$）时，可 $O(n)$ 计算 $L(x)$（需预处理阶乘逆元）。此时：

$$L(x)=\sum_{i=0}^{n}y_i*\frac{\prod_{j=0,j\neq i}^{n}(x-j)}{(-1)^{n-i}i!(n-i)!}$$

其中分子可通过前后缀积 $O(1)$ 得到。

```cpp
// 依赖 For/rFor、ll、qpow 与 M = 998244353。
// 由点值还原多项式或求单点值；点值与系数都在 [0,M)；lag_i/lag_p 需返回 int 的 ifac(n)。

// 由互异横坐标点值还原多项式，要求点数 >= 1 且横坐标互异。
// 返回低到高 n+1 项系数（末项为 0），O(n^2)。
vector<int> lag(vector<pair<int, int>> a) {
    int n = a.size();
    vector<int> f(n + 1), ans(n + 1, 0);
    f[0] = 1;
    For(i, 1, n) {
        auto [x, y] = a[i - 1];
        rFor(j, i - 1, 0) f[j + 1] = (f[j + 1] + f[j]) % M, f[j] = 1ll * f[j] * (M - x) % M;
    }
    For(i, 1, n) {
        auto [x, y] = a[i - 1];
        int w = 1, z = 0;
        For(j, 1, n) if(i != j) w = 1ll * w * (x - a[j - 1].first + M) % M;
        w = qpow(w, M - 2) * y % M;
        rFor(j, n - 1, 0) z = (f[j + 1] + 1ll * x * z) % M, ans[j] = (ans[j] + 1ll * z * w) % M;
    }
    return ans;
}

// a[1..n] 是 1..n 处的值，原地写回 a[0..n-1] 的系数。
// 要求 1 <= n < M、ifac 表已就位；末尾把 a[n] 清零。
void lag_i(int n, int *a) {
    vector<int> f(n + 1, 0), g(n, 0);
    f[0] = 1;
    For(i, 1, n) rFor(j, i - 1, 0) f[j + 1] = (f[j + 1] + f[j]) % M, f[j] = 1ll * f[j] * (M - i) % M;
    For(i, 1, n) {
        int w = 1ll * a[i] * ifac(n - i) % M * ifac(i - 1) % M, s = 0;
        if((i ^ n) & 1) w = (M - w) % M;
        rFor(j, n - 1, 0) s = (f[j + 1] + 1ll * i * s) % M, g[j] = (g[j] + 1ll * s * w) % M;
    }
    For(i, 0, n - 1) a[i] = g[i];
    a[n] = 0;
}
// a[0..n-1] 是 0..n-1 处的值，求 x 处的值。
// 要求 x 在 [0, M)；x < n 时直接返回 a[x]，否则 O(n)。
int lag_p(int n, int *a, int x) {
    if(x < n) return a[x];
    vector<int> f(n + 1), g(n + 1);
    int ans = 0;
    f[0] = g[n] = 1;
    For(i, 0, n - 1) f[i + 1] = 1ll * f[i] * (x - i + M) % M;
    rFor(i, n - 1, 0) g[i] = 1ll * g[i + 1] * (x - i + M) % M;
    For(i, 0, n - 1) {
        int w = 1ll * f[i] * g[i + 1] % M * a[i] % M * ifac(n - 1 - i) % M * ifac(i) % M;
        if(!((i ^ n) & 1)) w = (M - w) % M;
        ans = (ans + w) % M;
    }
    return ans;
}
```

4.3 类欧

```cpp
// 依赖 通用/头文件.cpp 的 ll、qpow；结果按 M = 998244353 取模。
// 一次递归同时求下取整和、平方和与带权和，是 O(log) 级的辗转相除。

const int i2 = (M + 1) / 2, i6 = qpow(6, M - 2);
struct nd {
    int f, g, h;
};
// 求 i=0..n 的 f=Σ⌊(ai+b)/c⌋、g=Σ⌊·⌋²、h=Σi⌊·⌋。
// 要求 n,a,b >= 0、c > 0；返回 {f,g,h}，三者均已 mod M。
nd f(int n_, int a, int b, int c) {
    if(n_ < 0) return {0, 0, 0};
    ll n = n_ % M, x = (a / c) % M, y = (b / c) % M;
    ll s0 = (n + 1) % M, s1 = n * s0 % M * i2 % M;
    ll s2 = n * s0 % M * ((n + n + 1) % M) % M * i6 % M;
    if(!a) return {int(s0 * y % M), int(s0 * y % M * y % M), int(s1 * y % M)};
    if(a >= c || b >= c) {
        nd rs = f(n_, a % c, b % c, c);
        ll F = rs.f, G = rs.g, H = rs.h;
        ll cross = (x * H + y * F + s1 * x % M * y % M) % M;
        ll g = (G + s2 * x % M * x % M + s0 * y % M * y % M + 2 * cross) % M;
        return {int((F + s1 * x + s0 * y) % M), int(g), int((H + s2 * x + s1 * y) % M)};
    }
    int m = (1ll * a * n_ + b) / c;
    nd rs = f(m - 1, c, c - b - 1, a);
    ll F = rs.f, G = rs.g, H = rs.h, z = m % M, nz = n * z % M;
```

```
    ll g = (nz * z - H * 2 - F + 3ll * M) % M;
    ll h = i2 * ((n * nz % M + nz - G - F + 2ll * M) % M) % M;
    return {int((nz - F + M) % M), int(g), int(h)};
}
```

4.4 反射容斥

从 $(0,0)$ 走到 (x,y)，只能往右往上，不触碰 $y-x=k$（$k>0$），往右的连续段数量为 b 的方案数

1. 开头 R，结尾 R

$$N_{RR}=\binom{x-1}{b-1}\binom{y-1}{b-2}-\binom{x+k-2}{b-2}\binom{y-k}{b-1}$$

2. 开头 R，结尾 U

$$N_{RU}=\binom{x-1}{b-1}\binom{y-1}{b-1}-\binom{x+k-2}{b-2}\binom{y-k}{b}$$

3. 开头 U，结尾 R

$$N_{UR}=\binom{x-1}{b-1}\binom{y-1}{b-1}-\binom{x+k-2}{b-1}\binom{y-k}{b-1}$$

4. 开头 U，结尾 U

$$N_{UU}=\binom{x-1}{b-1}\binom{y-1}{b}-\binom{x+k-2}{b-1}\binom{y-k}{b}$$

推导方法

记不限制开头结尾的总方案数为

$$F(x,y,k,b)=\binom{x-1}{b-1}\binom{y+1}{b}-\binom{x+k-1}{b-1}\binom{y-k+1}{b}$$

利用删除首尾的 U 步，可以得到：

$$N_{UU}=F(x,y-2,k-1,b)$$

$$N_{UR}=F(x,y-1,k-1,b)-N_{UU}$$

$$N_{RU}=F(x,y-1,k,b)-N_{UU}$$

$$N_{RR}=F(x,y,k,b)-N_{UU}-N_{UR}-N_{RU}$$

4.5 特征多项式

```
// 依赖 qpow 与 M = 998244353。
// 先相似变换到上 Hessenberg 再递推特征多项式；mat 为 n 阶方阵、元素在 [0,M)。

using poly = vector<int>;
using mat = vector<vector<int>>;
// 返回 det(xI-a) 的升幂 n+1 项；空矩阵返回 {1}，不改实参。
// O(n^3) 时间、O(n^2) 空间。
poly charpoly(mat a) {
    int n = a.size();
    For(i, 1, n - 1) {
        For(j, i, n - 1) if(a[j][i - 1]) {
            swap(a[i], a[j]);
            For(k, 0, n - 1) swap(a[k][i], a[k][j]);
            break;
        }
        if(!a[i][i - 1]) continue;
        int x = a[i][i - 1], inv = qpow(x, M - 2);
        For(j, 0, n - 1) a[j][i] = (ll)a[j][i] * x % M;
        For(j, i - 1, n - 1) a[i][j] = (ll)a[i][j] * inv % M;
        For(j, i + 1, n - 1) {
            int z = a[j][i - 1];
            For(k, 0, n - 1) a[k][i] = (a[k][i] + (ll)a[k][j] * z) % M;
            For(k, i - 1, n - 1) a[j][k] = (a[j][k] + (ll)a[i][k] * (M - z)) % M;
        }
    }
    mat dp(n + 1, poly(n + 1));
    dp[0][0] = 1;
    For(i, 0, n - 1) {
        int x = 1;
        rFor(j, i, 0) {
            int y = (ll)(M - x) * a[j][i] % M;
            For(k, 0, j) dp[i + 1][k] = (dp[i + 1][k] + (ll)y * dp[j][k]) % M;
            if(j) x = (ll)x * a[j][j - 1] % M;
        }
        For(k, 0, i) dp[i + 1][k + 1] = (dp[i + 1][k + 1] + dp[i][k]) % M;
    }
    return dp[n];
}
```

4.6 行列式多项式

```
// 依赖 数学/特征多项式.cpp 的 charpoly、mat、poly，及 qpow、M = 998244353。
// 求 det(a*x+b)：a,b 同阶、元素在 [0,M)，可奇异，不需要随机找可逆点。

// 返回升幂 n+1 项；空矩阵为 {1}，不改实参；O(n^3) 时间、O(n^2) 空间。
poly det_poly(mat a, mat b) {
    int n = a.size(), t = 1, d = 0;
    poly ans(n + 1);
    For(i, 0, n - 1) {
        while(!a[i][i]) {
            For(j, i + 1, n - 1) if(a[i][j]) {
                t = M - t;
                For(k, 0, n - 1) swap(a[k][i], a[k][j]), swap(b[k][i], b[k][j]);
                break;
            }
            if(a[i][i]) break;
            if(++d > n) return ans;
            For(j, 0, n - 1) a[i][j] = b[i][j], b[i][j] = 0;
            For(j, 0, i - 1) {
                int c = M - a[i][j];
```

```
                For(k, 0, n - 1) a[i][k] = (a[i][k] + (ll)c * a[j][k]) % M;
                For(k, 0, n - 1) b[i][k] = (b[i][k] + (ll)c * b[j][k]) % M;
            }
        }
        t = (ll)t * a[i][i] % M;
        int inv = qpow(a[i][i], M - 2);
        For(j, 0, n - 1) a[i][j] = (ll)a[i][j] * inv % M;
        For(j, 0, n - 1) b[i][j] = (ll)b[i][j] * inv % M;
        For(j, 0, n - 1) if(j != i) {
            int c = M - a[j][i];
            For(k, 0, n - 1) a[j][k] = (a[j][k] + (ll)c * a[i][k]) % M;
            For(k, 0, n - 1) b[j][k] = (b[j][k] + (ll)c * b[i][k]) % M;
        }
    }
    For(i, 0, n - 1) For(j, 0, n - 1) b[i][j] = (M - b[i][j]) % M;
    poly r = charpoly(b);
    For(i, d, n) ans[i - d] = (ll)t * r[i] % M;
    return ans;
}
```

5 数论

5.1 分数还原

```
// 依赖 通用/头文件.cpp 的 ll。
// 由模 p 下的值还小分母分数，辗转相除 O(log p)。

// 返回 (x, a)，满足 x ≡ q*a (mod p)、|x| <= A、a != 0 且 |a| 最小。
// 解释成模意义的 x/a 时要求 gcd(a, p) = 1；要求 p > 0、A >= 0。
pii approx(int p, int q, int A) {
    q %= p;
    if(q < 0) q += p;
    int x = q, y = p, a = 1, b = 0;
    while(x > A) {
        swap(x, y), swap(a, b);
        a -= x / y * b, x %= y;
    }
    return {x, a};
}
```

5.2 O(V)-O(1) GCD

```
// 依赖 通用/头文件.cpp 的 For；对象内含大数组。
// 预处理 O(V)，之后每次 gcd 为 O(1)。

struct fgcd_t {
    static constexpr int n = 1e7, B = 3163;
    array<int, 3> a[n + 1];
    int s[B + 1][B + 1];
    bitset<n + 1> b;
    // 对象含大数组不宜放栈上；单个对象可重复查询。
    fgcd_t() {
        vector<int> p;
        a[1] = {1, 1, 1};
        For(i, 2, n) {
            if(!b[i]) {
                if(i <= B) p.push_back(i);
                a[i] = {i, 1, 1};
            }
            for(auto z : p) {
                int x = i * z; if(x > n) break;
                b[x] = 1, a[x] = a[i];
                *min_element(all(a[x])) *= z;
                if(i % z == 0) break;
            }
        }
        For(i, 0, B) For(j, 0, i) s[i][j] = !j ? i : s[j][i % j];
    }
    // 要求 0 <= x,y <= n；返回 gcd，支持 0。
    int operator()(int x, int y) const {
        if(!x || !y) return x | y;
        if(x == 1 || y == 1) return 1;
        auto g1 = [&](int u, int v) {
            if(v == 1) return 1;
            int t = u % v;
            return t == 0 ? v : (v > B ? 1 : s[v][t]);
        };
        int a0 = g1(x, a[y][0]);
        int a1 = g1(x /= a0, a[y][1]);
        int a2 = g1(x /= a1, a[y][2]);
        return a0 * a1 * a2;
    }
};
```

5.3 离散对数

```
// 依赖 通用/头文件.cpp 的 ll 与 unordered_map。
// 求模素数 p 的离散对数，O(sqrt(p)) 时间与空间。

// 返回最小的 x >= 0 使 a^x ≡ b (mod p)，无解返回 -1。
// 要求 gcd(a, p) = 1 且 p 为素数。
ll BSGS(ll a, ll b, ll p) {
    a %= p, b %= p;
    if(p == 1) return 0;
    if(b == 1) return 0;
    if(!a) return -1;
    ll l = p - 1;
    for(ll q = 2; q * q <= l; ++q)
        for(; l % q == 0 && qpow(a, l / q, p) == 1; l /= q);
    if(l > 1 && qpow(a, l - 1, p) == 1) l = 1;
    ll m = (ll) sqrt((db) l);
    while(m * m < l) ++m;
    for(ll y = 0, w = 1 % p; y < std::min(m, l); ++y) {
        if(w == b) return y;
```

```
        w = (i128) w * a % p;
    }
    unordered_map<ll, ll> pos;
    pos.reserve(m + 2);
    for(ll j = 0, w = 1 % p; j < m; ++j) {
        pos.emplace(w, j);
        w = (i128) w * a % p;
    }
    ll step = qpow(a, m, p), iv = qpow(step, p - 2, p), w = (i128) b * iv % p, ans =
-1;
    for(ll i = 1; i <= m; ++i, w = (i128) w * iv % p)
        if(auto it = pos.find(w); it != pos.end()) {
            ll y = (ll) i * m + it->second;
            if(qpow(a, y, p) == b && (ans < 0 || y < ans)) ans = y;
        }
    return ans < 0 ? -1 : ans;
}
```

5.4 中国剩余定理

```
// 依赖 数论/exgcd.cpp 与 通用/头文件.cpp 的 ll、i128。
// 合并同余方程，返回 [0, lcm(p1,p2)) 内的解，无解返回 -1。

// 要求 0 <= a1 < p1、0 <= a2 < p2、1 <= p1,p2 <= 2^31；更大模数需另行核验溢出。
ll CRT(ll a1, ll p1, ll a2, ll p2) {
    ll x, y, g = exgcd(p1, p2, x, y);
    if((a2 - a1) % g) return -1;
    ll m1 = p1 / g, m2 = p2 / g, L = m1 * p2;
    ll k = (a2 - a1) / g % m2 * (x % m2 + m2) % m2;
    ll ans = ((i128) k * p1 + a1) % L;
    return (ans + L) % L;
}
// 多方程:ans = 0, mod = 1;
//   For(i, 1, n) {
//       ans = CRT(ans, mod, a[i], p[i]);
//       if(ans < 0) 无解;
//       mod = mod / gcd(mod, p[i]) * p[i];
//   }
```

5.5 Miller-Rabin 素性检验

```
// 依赖 通用/头文件.cpp 的 u64、u128。
// 确定性判定 u64 素数，7 个底覆盖全 u64；O(log p)。

constexpr bool chkp(u64 p) {
    if(p < 2) return 0;
    auto mul = [](u64 a, u64 b, u64 p) { return (u128) a * b % p; };
    auto pw = [&](u64 a, u64 x, u64 p, u64 r = 1) {
        for(; x; x >>= 1, a = mul(a, a, p))
            if(x & 1) r = mul(r, a, p);
        return r;
    };
    constexpr u64 B[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022};
    u64 s = __builtin_ctzll(p - 1), d = (p - 1) >> s;
    for(u64 a : B) {
        if(a % p == 0) continue;
        u64 x = pw(a % p, d, p);
        if(x == 1 || x == p - 1) continue;
        bool ok = 0;
        For(i, 1, s - 1) {
            x = mul(x, x, p);
            if(x == p - 1) { ok = 1; break; }
        }
        if(!ok) return 0;
    }
    return 1;
}
```

5.6 Pollard-Rho 分解质因数

```
// 依赖 数论/Miller-Rabin.cpp 的 chkp，以及 通用/头文件.cpp 的 u64、u128、gcd。
// 分解 u64 的质因数，返回升序含重因子的向量；期望 O(n^1/4)。

u64 isqrt(u64 x) {
    if(x < 2) return x;
    u64 r = sqrtl((long double) x);
    for(; (u128) r * r > x; --r);
    for(; (u128) (r + 1) * (r + 1) <= x; ++r);
    return r;
}
u64 rho(u64 x) {
    if(x % 2 == 0) return 2;
    if(x % 3 == 0) return 3;
    if(x % 5 == 0) return 5;
    if(u64 s = isqrt(x); s * s == x) return s;
    if(chkp(x)) return x;
    static mt19937_64 gen(114514);
    for(;;) {
        u64 c = 1 + gen() % (x - 1), t = 0, r = 0, w = 1;
        auto f = [&](u64 v) { return ((u128) v * v + c) % x; };
        for(int i = 1;; ++i) {
            t = f(t), r = f(f(r));
            if(t == r) break;
            u64 d = t > r ? t - r : r - t;
            w = (u128) w * d % x;
            if(!w) {
                if(u64 g = gcd(d, x); g > 1 && g < x) return g;
                break;
            }
            if(i % 127 == 0) {
                u64 g = gcd(w, x);
                if(g > 1 && g < x) return g;
                if(g == x) break;
            }
```

```
        }
    }
}
void fact_rec(u64 x, vector<u64> &res) {
    if(x == 1) return;
    if(chkp(x)) return res.push_back(x), void();
    u64 d = rho(x);
    fact_rec(d, res), fact_rec(x / d, res);
}
```

5.7 最小模线性值

```
// 依赖 通用/头文件.cpp 的 ll。
// 类欧式递归求等差数列取模的最小值，O(log m)。

// 返回 min_{0<=i<n} (a*i+b) mod m。
// 要求 n >= 0、m > 0、a >= 0，b 可负；n=0 时返回 m。
int minmod(int n, int m, int a, ll b) {
    int ans = m;
    while(n) {
        a %= m;
        b %= m;
        if(b < 0) b += m;
        cmin(ans, (int) b);
        n = ((ll) (n - 1) * a + b) / m;
        b -= (ll) m * n;
        swap(a, m);
    }
    return ans;
}
```

5.8 二次剩余

```
// 依赖 通用/头文件.cpp 的 ll 与 qpow。
// 判二次剩余 + Cipolla 开根，均要求 p 为奇素数。

// 判断 a 是否为奇素数 p 的二次剩余；要求 0 <= a < p。
bool quadres(u64 a, u64 p) {
    if(!a) return 1;
    auto mul = [p](u64 x, u64 y) { return (u64) ((u128) x * y % p); };
    auto pw = [&](u64 x, u64 e) {
        u64 r = 1;
        for(x %= p; e; e >>= 1, x = mul(x, x))
            if(e & 1) r = mul(r, x);
        return r;
    };
    return pw(a, (p - 1) >> 1) == 1;
}
// 求 x 模奇素数 p 的平方根；要求 0 <= x < p；x 非二次剩余时返回 -1。
ll sqrtp(ll x, ll p) {
    if(x <= 1) return x;
    auto mul = [p](ll a, ll b) { return (ll) ((u64) ((u128) a * b % p)); };
    auto add = [p](ll a, ll b) { return (ll) (((u64) a + (u64) b) % (u64) p); };
    auto pw = [&](ll a, ll e) {
        ll r = 1;
        for(a %= p; e; e >>= 1, a = mul(a, a))
            if(e & 1) r = mul(r, a);
        return r;
    };
    if(pw(x, (p - 1) >> 1) != 1) return -1;
    if(p % 4 == 3) return pw(x, (p + 1) >> 2);
    static mt19937_64 gen;
    ll w, a;
    do a = gen() % p; while(!a || pw(w = add(mul(a, a), p - x), (p - 1) >> 1) != p - 1);
    auto prod = [&](pll u, pll v) {
        ll t0 = add(mul(u[0], v[0]), mul(mul(u[1], v[1]), w));
        ll t1 = add(mul(u[0], v[1]), mul(u[1], v[0]));
        return pll{t0, t1};
    };
    pll r{1, 0}, b{a, 1};
    for(ll k = (p + 1) >> 1; k; k >>= 1, b = prod(b, b))
        if(k & 1) r = prod(r, b);
    return r[0];
}
```

5.9 原根与阶

```
// 依赖 通用/头文件.cpp 的 For、qpow 与 vector。
// 求素数的原根、元素的阶，并共用一份素因子分解；均 O(sqrt p + log^2 p)。

// 返回 x 的所有互异素因子（升序）；O(sqrt x)。
vector<int> fac_of(int x) {
    vector<int> fac;
    for(int i = 2; 1ll * i * i <= x; ++i)
        if(x % i == 0) {
            fac.push_back(i);
            while(x % i == 0) x /= i;
        }
    if(x > 1) fac.push_back(x);
    return fac;
}
// 求模素数 p 的最小原根；要求 p 为素数；p=2 返回 1，无原根返回 -1。
// g 是原根 <=> 对 p-1 的每个素因子 q 都有 g^((p-1)/q) != 1。
int root(int p) {
    if(p == 2) return 1;
    auto fac = fac_of(p - 1);
    For(g, 2, p - 1) {
        bool ok = 1;
        for(int q : fac) ok &= qpow(g, (p - 1) / q, p) != 1;
        if(ok) return g;
    }
    return -1;
}
```

```cpp
// a 模素数 p 的阶：最小 d > 0 使 a^d ≡ 1 (mod p)；要求 gcd(a,p)=1。
int ord(int a, int p) {
    if(p == 2) return 1;
    int x = p - 1;
    auto fac = fac_of(x);
    for(int q : fac)
        while(x % q == 0 && qpow(a, x / q, p) == 1) x /= q;
    return x;
}
```

5.10 积性函数

狄利克雷卷积:

$$(f*g)(n)=\sum_{d|n}f(d)g\left(\frac{n}{d}\right)$$

$$g(1)S_{f(n)}=S_{f*g}(n)-\sum_{i=2}^{n}g(i)S_{f(\lfloor\frac{n}{i}\rfloor)}$$

其中 $S_{h(n)}=\sum_{i=1}^{n}h(i)$ 是前缀和。

5.10.1 杜教筛

```cpp
// 依赖 通用/头文件.cpp 的 ll 与 unordered_map。
// 求数论函数前缀和 S(n)，O(n^{2/3})；需调用方提供阈值 N 与小范围前缀和。

// 求 S(n) = sum_{i=1}^n f(i)；O(n^{2/3})。
// 前置：调用方提供阈值 N 及预处理的前缀和 smu[0..N]、sphi[0..N]（下标 0 为 0）。
unordered_map<ll, ll> smu_big, sphi_big;
// 莫比乌斯函数前缀和。
ll DJS_mu(ll n) {
    if(n <= N) return smu[n];
    if(smu_big.count(n)) return smu_big[n];
    ll ans = 1;
    for(ll l = 2, r; l <= n; l = r + 1)
        r = n / (n / l), ans -= (r - l + 1) * DJS_mu(n / l);
    return smu_big[n] = ans;
}
// 欧拉函数前缀和；n*(n+1)/2 不溢出 ll。
ll DJS_phi(ll n) {
    if(n <= N) return sphi[n];
    if(sphi_big.count(n)) return sphi_big[n];
    ll ans = n * (n + 1) / 2;
    for(ll l = 2, r; l <= n; l = r + 1)
        r = n / (n / l), ans -= (r - l + 1) * DJS_phi(n / l);
    return sphi_big[n] = ans;
}
// 通用形式:只需实现 f 与 g,其余照抄上面两个函数的结构
//   ll S(ll n) {
//       if(n <= N) return pre[n];
//       if(mp.count(n)) return mp[n];
//       ll ans = H(n); // f * g 的前缀和
//       for(ll l = 2, r; l <= n; l = r + 1)
//           r = n / (n / l), ans -= (g_pre(r) - g_pre(l - 1)) * S(n / l);
//       return mp[n] = ans;
//   }
```

5.10.2 min_25

$O\left(\frac{n^{\frac{3}{4}}}{\log}n\right)$ 求积性函数 f 的前缀和,要求 f 在素数点 $f(p)$ 是关于 p 的低次多项式(最后拆成 p 的幂的线性组合)。

两个递推

设 p_j 是第 j 个素数($p_1=2$),$g_{k(n,j)}$ 是把 $i\le n$ 里所有“i 是素数,或 i 的最小素因子 $>p_j$”的 i^k 累加起来的结果($j=0$ 时就是 $\sum_{i=2}^{n}i^k$)。j 每加一,就把最小素因子恰好是 p_j 的合数减掉(只在 $p_j^2\le n$ 时用,再往后这些合数都超过 n):

$$g_{k(n,j)}=g_{k(n,j-1)}-p_j^k\left(g_{k\left(\left\lfloor\frac{n}{p_j}\right\rfloor,j-1\right)}-\sum_{i=1}^{j-1}p_i^k\right)$$

筛到 $p_j^2>n$ 时 $g_{k(n,j)}$ 就只剩素数:$g_{k(n)}=\sum_{p\le n}p^k$。把 $f(p)=\sum_k c_kp^k$ 的每一段 p^k 各筛一遍再按系数加起来,就得到素数处的 $\sum_{p\le n}f(p)$。真正的 f 前缀和再按最小素因子递归:

$$S(n,m)=\left(\sum_{p\le n}f(p)-\sum_{i=1}^{m}f(p_i)\right)+\sum_{j>m,p_j^2\le n}\sum_{e\ge1,p_j^{e+1}\le n}\left(f\left(p_j^e\right)S\left(\left\lfloor\frac{n}{p_j^e}\right\rfloor,j\right)+f\left(p_j^{e+1}\right)\right)$$

答案 $=S(n,0)+f(1)$;模板里的 `f(n, m)` 就是这个 $S(n,m)$,m 表示“前 m 个素数已经排除”。只有 $O(\sqrt{n})$ 个不同的 $\lfloor\frac{n}{i}\rfloor$,所以状态数是 $O(\sqrt{n})$。

```cpp
// 依赖 数论/积性函数/线性筛.cpp（或自带小素数表）与 M = 998244353。
// 求积性函数在 1..n 的前缀和，1 <= n <= 1e10；solve 可重复调用，init 只需一次。
constexpr int i6 = (M + 1) / 6;
// 模 M 减法；要求 0 <= a,b < M。
void sub(int &a, int b) {
    a -= b;
    a < 0 && (a += M);
}
const int N = 1e5 + 10;
int p[N], pc, p2[N];
int s1[N], s2[N], s[N];
// 筛 1e5 内素数及前缀表；可重复调用。
void init() {
    if(pc) return;
    static const int n = 1e5;
    static int b[N];
    For(i, 2, n) {
        if(!b[i]) p[++pc] = i;
        For(j, 1, pc) {
            int x = i * p[j];
            if(x > n) break;
            b[x] = 1;
            if(i % p[j] == 0) break;
```

```
        }
    }
    p[pc + 1] = n + 1;
    For(i, 1, pc) {
        p2[i] = 1ll * p[i] * p[i] % M;
        s1[i] = (s1[i - 1] + p[i]) % M;
        s2[i] = (s2[i - 1] + p2[i]) % M;
        s[i] = (s2[i] - s1[i] + M) % M;
    }
}
ll lim, n0;
int g1[N * 2], g2[N * 2];
// 映射 n0/i 到数组下标；要求已调用 solveg 设置 n0 与 lim。
int id(ll n) {
    return n <= lim ? n : n0 / n + lim;
}
// 预处理 g 表；要求 1 <= _n0 <= 1e10 且 init 已完成。
void solveg(ll _n0) {
    n0 = _n0;
    lim = sqrt(n0);
    bool fl = n0 / lim == lim;
    auto put = [&](ll k, int slot) {
        k %= M;
        sub(g1[slot] = k * (k + 1) / 2 % M, 1);
        sub(g2[slot] = k * (k + 1) % M * (k + k + 1) % M * i6 % M, 1);
    };
    auto doit = [&](int m, int i1, int i2) {
        sub(g1[i1], 1ll * (g1[i2] - s1[m - 1] + M) * p[m] % M);
        sub(g2[i1], 1ll * (g2[i2] - s2[m - 1] + M) * p2[m] % M);
    };
    For(i, 1, lim) put(i, i);
    For(i, 1, lim - fl) put(n0 / i, i + lim);
    For(i, 1, pc) {
        ll x = p[i], y = x * x, z = n0 / x;
        For(j, 1, min(lim - fl, n0 / y)) doit(i, j + lim, id(z / j));
        rFor(j, lim, min(lim + 1, y)) doit(i, j, j / x);
    }
}
// 查询素数贡献前缀；要求 solveg(n0) 后且 n 是 n0 的除法块取值。
int g(ll n) { return (g2[id(n)] - g1[id(n)] + M) % M; }
// 素数幂处的函数值；要求 n >= 1。
ll func(ll n) {
    n %= M;
    return n * (n - 1) % M;
}
// 递归计算剩余贡献；要求 solveg(n) 已建表且 p[m] 以下素数已剔除。
int f(ll n, int m) {
    if(p[m] > n) return 0;
    ll ans = g(n) - s[m] + M;
    For(j, m + 1, pc) {
        int q = p[j];
        if(1ll * q * q > n) break;
        for(ll w = q; w * q <= n; w *= q) ans += func(w) * f(n / w, j) % M + func(w * q);
    }
    return ans % M;
}
// 求 1 + sum_{2<=i<=n} F(i)。
ll solve(ll n) {
    init();
    solveg(n);
    return (f(n, 0) + 1) % M;
}
```

5.11 下取整和

floor_sum(n, m, a, b) ：$\sum_{i=0}^{n-1}\left\lfloor\frac{ai+b}{m}\right\rfloor$。

```
// 依赖 通用/头文件.cpp 的 u64、u128。
// 类欧的裸下取整特例，O(log m)。

// 返回 sum_{i=0}^{n-1} floor((a*i+b)/m)；参数无符号、要求 m > 0，a/b 不必小于 m。

u64 floor_sum(u64 n, u64 m, u64 a, u64 b) {
    u128 ans = 0;
    for(;;) {
        if(a >= m) {
            ans += (u128)n * (n - 1) / 2 * (a / m);
            a %= m;
        }
        if(b >= m) {
            ans += (u128)n * (b / m);
            b %= m;
        }
        u64 y = (i128) a * n + b;
        if(y < m) break;
        n = y / m;
        b = y % m;
        swap(m, a);
    }
    return (u64)ans;
}
```

5.12 高斯整数

$N(a+bi)=a^2+b^2$ 叫 $a+bi$ 的模长平方,它可乘:$N(xy)=N(x)N(y)$。下面说的“拆”,都是要让 N 落回目标 n。

素数分类

素数分为三种：

- 2 是特例：$2=(1+i)(1-i)$，拆出来的两个只差 ±1、$\pm i$；
- $4k+1$ 型的 p 能拆：$p=(x+yi)(x-yi)$，也就是 $p=x^2+y^2$；
- $4k+3$ 型的 p 拆不动，自己就是高斯素数。

这三类就是全部的高斯素数。每个非零高斯整数都能写成它们的乘积，而且写法唯一，只差整体的 ± 1、$\pm i$。

判定 $x^2+y^2=n$

n 的每个 $4k+3$ 型素因子指数都是偶数时才能写成 x^2+y^2，否则不能。直接分解即可。

计数 $x^2+y^2=n$

记 $\chi_4(d)$ 在 d 模 4 余 1、余 3、为偶数时分别是 1、-1、0，则 $\chi_4(d)$ 是完全积性函数 $\Rightarrow$ $\sum_{d\mid n}\chi_4(d)$ 是积性函数。

记 d_1 是 n 的模 4 余 1 的因子个数，d_3 是模 4 余 3 的因子个数，则 $d_1-d_3=\sum_{d\mid n}\chi_4(d)$，且也是解个数。

- x、y 取任意整数（包括负数和交换顺序）：解数是 $4(d_1-d_3)$。
- 只要 $0\le x\le y$ 的那些：解数是 $\frac{d_1-d_3+s}{2}$，其中 s 数的是 n、$\frac{n}{2}$ 里完全平方的个数（0、1 或 2）。

如 3 无解，$2=1^2+1^2$，$65=8^2+1^2=7^2+4^2$。

Cornacchia

把 $4k+1$ 型的 p 拆成 $p=x^2+y^2$：

1. 从 $g=2,3,5,\dots$ 试出模 p 的非二次剩余，算 $r=g^{\frac{p-1}{4}}\bmod p$，此时 $r^2\equiv -1 \bmod p$。
2. 对 (p,r) 做辗转相除，除到余数 $\le\sqrt{p}$ 为止。
3. 停下的那个余数就是 x；拿 $p-x^2$ 开方得 y，验一下 $x^2+y^2=p$。

它跟求 $\gcd(p,r+i)$ 是一回事，但全程只用整数，不会溢出。

高斯整数 gcd

跟整数一样辗转相除，只是“除”要除到最近的整数：$\frac{x}{y}$ 先算成 $x\cdot\frac{\bar y}{|y|^2}$，再把实部虚部各自四舍五入。 可以证明每一步余数都满足 $N(r)\le\frac{N(y)}{2}$，所以只要 $O(\log N)$ 步；结果唯一到只差 ±1、$\pm i$。中间乘积会到 10^{36} 量级，代码里要用 128 位整数。

全部解

分解质因数，然后：

- 2^k 换成 $(1+i)^k$（它的模长平方就是 2^k）；
- $4k+1$ 型的 p^e：先用 Cornacchia 拆出 p 的那对 $(x+yi)$、$(x-yi)$，再换成 $(x+yi)^e$ 或 $(x-yi)^e$，两种都试；
- $4k+3$ 型的 p^e（e 必是偶数）换成实数 $p^{\frac{e}{2}}$。

把这些乘起来得到高斯整数 $A+Bi$，它的模长平方正好是 n，所以 $(|A|,|B|)$ 就是一组解。 每个 $4k+1$ 型的因子有两种选法，把所有选法 DFS 组合一遍（$4k+1$ 型的因子没几个）， 每组的 $(|A|,|B|)$ 统一成 $0\le x\le y$，排序去重就是全部解。

```cpp
// 依赖 通用/头文件.cpp 的 ll、i128 与 gcd。
// 高斯整数的四舍五入除法与欧几里得 gcd，坐标绝对值 <= 1e18。

struct gi { ll a, b; };
// p/d 四舍五入；要求 d > 0；负数按远离 0 的半数处理。
i128 rd(i128 p, i128 d) { return (p >= 0 ? p + d / 2 : p - d / 2) / d; }
// 求高斯整数 gcd；返回按单位元归一后的 gcd（实部>0、虚部>=0），gcd(0,0)=0。
// 要求坐标绝对值 <= 1e18。
gi gi_gcd(gi x, gi y) {
    while(y.a || y.b) {
        i128 d = (i128) y.a * y.a + (i128) y.b * y.b;
        i128 qa = rd((i128) x.a * y.a + (i128) x.b * y.b, d);
        i128 qb = rd((i128) x.b * y.a - (i128) x.a * y.b, d);
        gi r = {(ll) (x.a - qa * y.a + qb * y.b), (ll) (x.b - qa * y.b - qb * y.a)};
        x = y, y = r;
    }
    if(x.a < 0 || (x.a == 0 && x.b < 0)) x = {-x.a, -x.b};
    if(x.b < 0) x = {-x.b, x.a}; else if(x.a == 0) x = {x.b, 0};
    return x;
}
```

6 多项式

6.1 最短线性递推

```cpp
// 依赖 通用/头文件.cpp 的 For、ll、qpow 与 M = 998244353。
// 求最短线性递推；序列各项在 [0,M)，O(n²)。只保证拟合给定前缀，退化时系数可能不唯一。

using poly = vector<int>;
// 返回最短递推 c：a[i]=Σ_{j=1..|c|} c[j-1]a[i-j]（|c|<=i<|a|）；空/全零序列返回空。
poly BM(const poly &a) {
    int n = a.size();
    if(!n) return poly();
    int len = 0, off = 0, s = 1;
    poly res(n + 1), lst(n + 1);
    res[0] = 1;
    For(i, 0, n - 1) {
        int d = 0;
        For(j, 0, len) d = (d + 1ll * res[j] * a[i - j]) % M;
        ++off;
        if(!d) continue;
        poly tmp = res;
        int k = 1ll * d * qpow(s, M - 2) % M;
        For(j, off, n) res[j] = (res[j] - 1ll * k * lst[j - off] % M + M) % M;
        if(len * 2 > i) continue;
        len = i - len + 1, lst = tmp, s = d, off = 0;
    }
    poly c(len);
    For(i, 0, len - 1) c[i] = (M - res[i + 1]) % M;
    return c;
}
```

6.2 复数 FFT 卷积

```cpp
// 依赖 通用/头文件.cpp 的 For/db 与 complex。
// 实系数卷积，O((n+m) log(n+m))；系数绝对值须明显小于 1e15。

using cp = complex<db>;
```

```cpp
// 原地 FFT：k=1 正变换，k=-1 逆变换（含除以 n）；长度须为 2 的幂，空输入也可。
void fft(vector<cp> &a, int k) {
    int n = a.size();
    if(n <= 1) return;
    static vector<int> r;
    if((int) r.size() != n) {
        int l = __lg(n);
        r.resize(n);
        For(i, 0, n - 1) r[i] = (r[i >> 1] >> 1) | ((i & 1) << (l - 1));
    }
    For(i, 0, n - 1) if(i < r[i]) swap(a[i], a[r[i]]);
    for(int m = 1; m < n; m <<= 1) {
        cp wm = polar((db) 1, acos((db) -1) / m * k);
        for(int i = 0; i < n; i += m + m) {
            cp w = 1;
            For(j, 0, m - 1) {
                cp x = a[i + j], y = a[i + j + m] * w;
                a[i + j] = x + y, a[i + j + m] = x - y, w *= wm;
            }
        }
    }
    if(k == -1) For(i, 0, n - 1) a[i] /= n;
}
// 返回长度 |a|+|b|-1 的实系数卷积；任一为空则返回空。
vector<db> fft_conv(const vector<db> &a, const vector<db> &b) {
    int n = a.size(), m = b.size();
    if(!n || !m) return {};
    int l = 1;
    while(l < n + m - 1) l <<= 1;
    vector<cp> fa(l), fb(l);
    For(i, 0, n - 1) fa[i] = a[i];
    For(i, 0, m - 1) fb[i] = b[i];
    fft(fa, 1), fft(fb, 1);
    For(i, 0, l - 1) fa[i] *= fb[i];
    fft(fa, -1);
    vector<db> res(n + m - 1, 0);
    For(i, 0, n + m - 2) res[i] = fa[i].real();
    return res;
}
```

6.3 NTT

```cpp
// 依赖 通用/头文件.cpp 的 For、ll、qpow 与 M = 998244353。
// 模 M 的数论变换与卷积；系数在 [0,M)，变换长度 1..2^23，逆变换已含归一。

using poly = vector<int>;
// 原地变换；长度为 2 的幂，1<=|a|<=2^23，系数在 [0,M)；k=±1 分别正/逆变换，无需预处理。
void NTT(poly &a, int k) {
    static poly rev;
    static poly roots{0, 1};
    static vector<u64> tmp;
    int n = a.size();
    if(n == 1) return;
    if((int)rev.size() != n) {
        rev.resize(n);
        int l = __lg(n);
        For(i, 0, n - 1) rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (l - 1));
    }
    for(int m = roots.size(); m < n; m <<= 1) {
        roots.resize(m << 1);
        int w = qpow(3, (M - 1) / (m << 1));
        roots[m] = 1;
        For(j, 1, m - 1) roots[m + j] = (ll)roots[m + j - 1] * w % M;
    }
    tmp.resize(n);
    For(i, 0, n - 1) tmp[i] = a[rev[i]];
    for(int m = 1, s = 0; m < n; m <<= 1, ++s) {
        for(int i = 0; i < n; i += m << 1)
            For(j, 0, m - 1) {
                int x = tmp[i + j + m] * roots[m + j] % M;
                tmp[i + j + m] = tmp[i + j] + M - x;
                tmp[i + j] += x;
            }
        if(s == 17 || (m << 1) == n)
            For(i, 0, n - 1) tmp[i] %= M;
    }
    For(i, 0, n - 1) a[i] = tmp[i];
    if(k == -1) {
        reverse(a.begin() + 1, a.end());
        int inv = qpow(n, M - 2);
        For(i, 0, n - 1) a[i] = (ll)a[i] * inv % M;
    }
}
// 返回普通卷积的 |a|+|b|-1 项，任一为空返回空；系数在 [0,M)，变换长度不超过 2^23。
poly mul(poly a, poly b) {
    if(a.empty() || b.empty()) return {};
    int len = a.size() + b.size() - 1;
    int n = 1;
    while(n < len) n <<= 1;
    a.resize(n);
    b.resize(n);
    NTT(a, 1);
    NTT(b, 1);
    For(i, 0, n - 1) a[i] = (ll)a[i] * b[i] % M;
    NTT(a, -1);
    a.resize(len);
    return a;
}
```

6.4 多项式求逆

```cpp
// 依赖 通用/头文件.cpp 的 For/ll/qpow 与 ntt.cpp 的 mul。
// 牛顿迭代求逆，O(n log n)；系数在 [0,M)，f 不足 n 项时高次补零。

using poly = vector<int>;
```

```
// 返回 n 项的 g，使 f*g ≡ 1 (mod x^n)；n>=1，f 非空且 f[0]!=0。
poly poly_inv(const poly &f, int n) {
    poly g{int(qpow(f[0], M - 2))};
    for(int m = 1; m < n; m <<= 1) {
        int t = min(m << 1, n);
        poly c(f.begin(), f.begin() + min((int) f.size(), t));
        poly d = mul(g, c);
        d.resize(t);
        For(i, 0, t - 1) d[i] = ((i == 0 ? 2 : 0) - d[i] + M) % M;
        g = mul(g, d);
        g.resize(t);
    }
    g.resize(n);
    return g;
}
```

6.5 多项式对数

```
// 依赖 多项式/多项式求逆.cpp 的 poly_inv 与 ntt.cpp 的 mul。
// 求导 + 积分 + 求逆得 ln，O(n log n)；系数在 [0,M)，n < M，f 高次缺项视零。

using poly = vector<int>;
// 缓存 1..n 的模逆元。
static const poly &_ln_inv_num(int n) {
    static poly iv{0, 1};
    if((int) iv.size() <= n) {
        int m = iv.size();
        iv.resize(n + 1);
        For(i, m, n) iv[i] = M - 1ll * (M / i) * iv[M % i] % M;
    }
    return iv;
}
// 返回 ln f mod x^n 的 n 项（常数项 0）；f 非空且 f[0]=1；n<=0 返回空。
poly poly_ln(const poly &f, int n) {
    if(n <= 0) return poly();
    poly d(n - 1);
    For(i, 1, min((int) f.size(), n) - 1) d[i - 1] = 1ll * f[i] * i % M;
    d = mul(d, poly_inv(f, n));
    d.resize(n - 1);
    const poly &iv = _ln_inv_num(n);
    poly g(n);
    For(i, 1, n - 1) g[i] = 1ll * d[i - 1] * iv[i] % M;
    return g;
}
```

6.6 多项式指数

```
// 依赖 多项式/多项式ln.cpp 的 poly_ln、求逆与 ntt.cpp 的 mul。
// 牛顿迭代求 exp，O(n log n)；系数在 [0,M)，n < M，f 高次缺项视零。

using poly = vector<int>;
// 返回 exp f mod x^n 的 n 项（常数项 1）；n>=1，f 非空且 f[0]=0。
poly poly_exp(const poly &f, int n) {
    poly g{1};
    for(int m = 1; m < n; m <<= 1) {
        int t = min(m << 1, n);
        poly c = poly_ln(g, t);
        For(i, 0, t - 1) c[i] = ((i < (int) f.size() ? f[i] : 0) - c[i] + M) % M;
        c[0] = (c[0] + 1) % M;
        g = mul(g, c);
        g.resize(t);
    }
    g.resize(n);
    return g;
}
```

6.7 多项式开根

```
// 依赖 多项式/多项式求逆.cpp 的 poly_inv 与 ntt.cpp 的 mul。
// 牛顿迭代开根，O(n log n)；系数在 [0,M)，f 高次缺项视零。

using poly = vector<int>;
// 返回 n 项的 g，使 g² ≡ f (mod x^n)、g[0]=1（唯一）；n>=1，f 非空且 f[0]=1。
poly poly_sqrt(const poly &f, int n) {
    const int ih = (M + 1) / 2;
    poly g{1};
    for(int m = 1; m < n; m <<= 1) {
        int t = min(m << 1, n);
        poly c(f.begin(), f.begin() + min((int) f.size(), t));
        poly d = mul(c, poly_inv(g, t));
        d.resize(t);
        g.resize(t);
        For(i, 0, t - 1) g[i] = (g[i] + 1ll * d[i]) * ih % M;
    }
    g.resize(n);
    return g;
}
```

6.8 带余除法

```
// 依赖 多项式/多项式求逆.cpp 的 poly_inv 与 ntt.cpp 的 mul。
// 反转后求逆做带余除法，O(n log n)；系数在 [0,M)，n = |f|。

using poly = vector<int>;
// 返回 {q,r}，满足 f=q*g+r、deg r<deg g；g 非空且 g.back()!=0。
// n>=m=|g| 时长度为 {n-m+1,m-1}（不 trim）；否则返回 {空,f}。
pair<poly, poly> poly_divmod(const poly &f, const poly &g) {
    int n = f.size(), m = g.size();
    if(n < m) return {poly(), f};
    int k = n - m + 1;
    poly rf(f.rbegin(), f.rend()), rg(g.rbegin(), g.rend());
    poly q = mul(rf, poly_inv(rg, k));
    q.resize(k);
    reverse(q.begin(), q.end());
```

```
    poly r = mul(q, g);
    r.resize(n);
    For(i, 0, n - 1) r[i] = (f[i] - r[i] + M) % M;
    r.resize(m - 1);
    return {q, r};
}
poly poly_mod(const poly &f, const poly &g) { return poly_divmod(f, g).second; }
```

6.9 多点求值

```
// 依赖 多项式/多项式除法.cpp 的 poly_mod、求逆与 ntt.cpp 的 mul。
// 分治取模做多点求值，O((|f|+m) log²(|f|+m)) 时间、O(m log m) 空间。

using poly = vector<int>;
// t[k]=Π_{i=l..r}(x-xs[i])。
static void _eval_build(poly *t, int k, int l, int r, const poly &xs) {
    if(l == r) return void(t[k] = poly{(M - xs[l]) % M, 1});
    int mid = (l + r) >> 1;
    _eval_build(t, k + k, l, mid, xs), _eval_build(t, k + k + 1, mid + 1, r, xs);
    t[k] = mul(t[k + k], t[k + k + 1]);
}
static void _eval_solve(const poly *t, int k, int l, int r, const poly &f, poly
&res) {
    poly g = poly_mod(f, t[k]);
    if(l == r) return void(res[l] = g.empty() ? 0 : g[0]);
    int mid = (l + r) >> 1;
    _eval_solve(t, k + k, l, mid, g, res), _eval_solve(t, k + k + 1, mid + 1, r, g,
res);
}
// 返回各 f(xs[i])，长度 |xs|；xs 可空或重复，空 f 视作零多项式。
poly poly_eval(const poly &f, const poly &xs) {
    int m = xs.size();
    if(!m) return poly();
    vector<poly> t(4 * m);
    _eval_build(t.data(), 1, 0, m - 1, xs);
    poly res(m);
    _eval_solve(t.data(), 1, 0, m - 1, f, res);
    return res;
}
```

6.10 快速插值

```
// 依赖 多项式/多点求值.cpp 的 poly_eval 与 ntt.cpp 的 mul。
// 分治合并插值，O(m log² m)；所有输入在 [0,M)，m = |xs|。

using poly = vector<int>;
// t[k]=Π_{i=l..r}(x-xs[i])。
static void _interp_build(poly *t, int k, int l, int r, const poly &xs) {
    if(l == r) return void(t[k] = poly{(M - xs[l]) % M, 1});
    int mid = (l + r) >> 1;
    _interp_build(t, k + k, l, mid, xs), _interp_build(t, k + k + 1, mid + 1, r,
xs);
    t[k] = mul(t[k + k], t[k + k + 1]);
}
// 用已建乘积树合并该段的插值贡献，w[i]=ys[i]/Π_{j!=i}(xs[i]-xs[j])。
static poly _interp_rec(const poly *t, int k, int l, int r, const poly &w) {
    if(l == r) return poly{w[l]};
    int mid = (l + r) >> 1;
    poly a = mul(_interp_rec(t, k + k, l, mid, w), t[k + k + 1]);
    poly b = mul(_interp_rec(t, k + k + 1, mid + 1, r, w), t[k + k]);
    a.resize(max(a.size(), b.size()), 0);
    For(i, 0, (int) b.size() - 1) a[i] = (a[i] + b[i]) % M;
    return a;
}
// 返回过各 (xs[i],ys[i]) 的唯一 deg<m 多项式（恰好 m 项）；xs、ys 等长且 xs 互异，空输入返回
空。
poly poly_interp(const poly &xs, const poly &ys) {
    int m = xs.size();
    if(!m) return poly();
    vector<poly> t(4 * m);
    _interp_build(t.data(), 1, 0, m - 1, xs);
    poly dM(m);
    For(i, 1, m) dM[i - 1] = 1ll * t[1][i] * i % M;
    poly ev = poly_eval(dM, xs), w(m);
    For(i, 0, m - 1) w[i] = 1ll * ys[i] * qpow(ev[i], M - 2) % M;
    poly res = _interp_rec(t.data(), 1, 0, m - 1, w);
    res.resize(m);
    return res;
}
```

6.11 线性递推第 n 项

```
// 依赖 多项式/ntt.cpp 的 mul、通用/头文件.cpp 的 qpow 与 M = 998244353。
// 求 k 阶线性递推的第 n 项，O(k log k log n)；系数在 [0,M)。

using poly = vector<int>;
// 返回 [x^n]P/Q；n>=0、Q 非空且 Q[0]!=0；P 可空或长于 Q。
int bostan_mori(poly p, poly q, ll n) {
    if(p.empty()) return 0;
    if(q.size() == 1) return n < (ll)p.size() ? 1ll * p[n] * qpow(q[0], M - 2) % M :
0;
    while(n) {
        if(n < (ll) p.size()) p.resize(n + 1);
        if(n < (ll) q.size()) q.resize(n + 1);
        poly Qm = q;
        For(i, 1, (int) Qm.size() - 1) if(i & 1) Qm[i] = (M - Qm[i]) % M;
        poly U = mul(p, Qm), V = mul(q, Qm);
        p.clear();
        for(int i = n & 1; i < (int) U.size(); i += 2) p.push_back(U[i]);
        q.clear();
        for(int i = 0; i < (int) V.size(); i += 2) q.push_back(V[i]);
        n >>= 1;
    }
    return 1ll * p[0] * qpow(q[0], M - 2) % M;
```

```
}
```

6.12 多项式复合

```
// 依赖 多项式/ntt.cpp 的 mul。
// 双变量分治卷积做复合，O(n log² n)；系数低次在前且在 [0,M)，每层变换长度 <= 2^23。

using poly = vector<int>;
namespace composition {
using polys = vector<poly>;
// 双变量分治：g[0](x)=1；n 递减、m 倍增，返回 m 行、每行 n 项的合并结果。
polys solve(const poly &f, const polys &g, int n, int m) {
    polys ans(m);
    if(n == 1) {
        For(i, 0, m - 1) ans[i] = {i < (int)f.size() ? f[i] : 0};
        return ans;
    }
    int d = 2 * n - 1;
    int s = m * d + n;
    poly a(s);
    poly b(s);
    For(i, 0, m)
        For(j, 0, n - 1) {
        a[i * d + j] = g[i][j];
        b[i * d + j] = (j & 1) && g[i][j] ? M - g[i][j] : g[i][j];
    }
    a = mul(move(a), b);
    int u = (n + 1) / 2;
    int v = 2 * m;
    polys h(v + 1, poly(u));
    For(i, 0, v)
        For(j, 0, u - 1) h[i][j] = a[i * d + 2 * j];
    auto p = solve(f, h, u, v);
    poly c((v - 1) * d + n);
    For(i, 0, v - 1)
        For(j, 0, u - 1) c[i * d + 2 * j] = p[v - 1 - i][j];
    c = mul(move(c), move(b));
    For(i, 0, m - 1) {
        ans[i].resize(n);
        For(j, 0, n - 1) ans[i][j] = c[(v - 1 - i) * d + j];
    }
    return ans;
}
}  // namespace composition
// 返回 F(G(x)) mod x^n 的 n 项；n>=1、G[0]=0（实现忽略 G[0]）。
// F、G 不足 n 项补零，高次忽略，不改输入；空 G 视零多项式。
poly comp(const poly &f, const poly &g, int n) {
    composition::polys h(2, poly(n));
    h[0][0] = 1;
    For(i, 1, min((int)g.size(), n) - 1) h[1][i] = g[i] ? M - g[i] : 0;
    poly a(f.begin(), f.begin() + min((int)f.size(), n));
    return composition::solve(a, h, n, 1)[0];
```



```
}
```

6.13 多项式复合逆

```
// 依赖 多项式/ntt.cpp 的 NTT 与 mul。
// 拉格朗日反演求复合逆，O(n log² n)；系数在 [0,M)，每轮变换长度 <= 2^23。

using poly = vector<int>;
namespace reversion {
// 返回 [x^(n-1)]1/(1-yf(x)) 的前 n 项 y 系数；n>=1、|f|=n，仅供复合逆使用。
poly pw_pj(const poly &f, int n) {
    int m = 1;
    int d = n - 1;
    vector<poly> a(2, poly(n));
    vector<poly> b(2, poly(n));
    a[0][0] = b[0][0] = 1;
    For(i, 0, n - 1) b[1][i] = f[i] ? M - f[i] : 0;
    while(d) {
        int stride = 2 * d + 1;
        int size = m * stride + d + 1;
        int len = 1;
        while(len < 2 * size - 1) len <<= 1;
        poly pa(len);
        poly pb(len);
        poly pc(len);
        For(i, 0, m)
            For(j, 0, d) {
            pa[i * stride + j] = a[i][j];
            pb[i * stride + j] = b[i][j];
            pc[i * stride + j] = (j & 1) && b[i][j] ? M - b[i][j] : b[i][j];
        }
        NTT(pa, 1);
        NTT(pb, 1);
        NTT(pc, 1);
        For(i, 0, len - 1) {
            pa[i] = (ll)pa[i] * pc[i] % M;
            pb[i] = (ll)pb[i] * pc[i] % M;
        }
        NTT(pa, -1);
        NTT(pb, -1);
        int r = d & 1;
        d /= 2;
        m *= 2;
        a.resize(m + 1);
        b.resize(m + 1);
        For(i, 0, m) {
            a[i].resize(d + 1);
            b[i].resize(d + 1);
            For(j, 0, d) {
                a[i][j] = pa[i * stride + 2 * j + r];
                b[i][j] = pb[i * stride + 2 * j];
            }
```

```
        }
    }
    poly p(n);
    For(i, 0, n - 1) p[i] = a[i][0];
    return p;
}
}  // namespace reversion
// 返回 n 项的复合逆 G，使 F(G(x)) ≡ x mod x^n；n>=1、F[0]=0、F[1]!=0。
// 不足项补零，高次忽略，不改输入；即使 n=1 也须提供非零一次项。
poly comp_inv(poly f, int n) {
    if(n == 1) return {0};
    int p = qpow(f[1], M - 2);
    f.resize(n);
    int c = 1;
    For(i, 0, n - 1) {
        f[i] = (ll)f[i] * c % M;
        c = (ll)c * p % M;
    }
    f = reversion::pw_pj(f, n);
    reverse(f.begin(), f.end());
    poly inv(n);
    inv[1] = 1;
    For(i, 2, n - 1) inv[i] = M - (ll)(M / i) * inv[M % i] % M;
    For(i, 0, n - 1) f[i] = (ll)f[i] * inv[n - 1 - i] % M * (n - 1) % M;
    int r = M - inv[n - 1];
    poly g{1};
    while(r) {
        if(r & 1) {
            g = mul(move(g), f);
            g.resize(n);
        }
        f = mul(f, f);
        f.resize(n);
        r >>= 1;
    }
    g.resize(n);
    rFor(i, n - 1, 1) g[i] = (ll)g[i - 1] * p % M;
    g[0] = 0;
    return g;
}
```

6.14 任意模数多项式乘法

```
// 依赖 通用/头文件.cpp 的 For/ll，与 ntt.cpp 相互独立。
// 三模 NTT + CRT 做任意模卷积；2 <= P <= 2^31-1，变换长度 <= 2^20。

using poly = vector<int>;
namespace Mul {
// 原地 NTT：G 为模素数 M 的原根，长度 n>=2 为 2 的幂且 2n | M-1；k=±1 分别正/逆变换。
template <int M, int G> static void NTT(poly &a, const int k) {
    static poly r, w;
    int n = a.size();
    if(n != r.size()) {
        int l = __lg(n);
        r.resize(n);
        For(i, 0, n - 1) r[i] = (r[i >> 1] >> 1) | ((i & 1) << (l - 1));
    }
    For(i, 0, n - 1) if(i < r[i]) swap(a[r[i]], a[i]);
    ll wi = qpow(G, ((M - 1) >> 1) / n * k + M - 1, M);
    w.resize(n), w[0] = 1;
    For(i, 1, n - 1) w[i] = w[i - 1] * wi % M;
    for(int m = 1, l = __lg(n); m < n; m <<= 1, --l)
        for(int i = 0; i < n; i += m + m)
            For(j, 0, m - 1) {
                ll x = a[i + j], y = 1ll * w[j << l] * a[i + j + m] % M;
                a[i + j] = x + y >= M ? x + y - M : x + y;
                a[i + j + m] = x - y < 0 ? x - y + M : x - y;
            }
    if(k == -1) {
        ll inv = qpow(n, M - 2, M);
        For(i, 0, n - 1) a[i] = a[i] * inv % M;
    }
}
// 单模卷积；a、b 须已补零到合法变换长度 l，结果保留 l 项。
template <int M, int G> static poly conv(poly a, poly b, int l) {
    NTT<M, G>(a, 1), NTT<M, G>(b, 1);
    For(i, 0, l - 1) a[i] = 1ll * a[i] * b[i] % M;
    NTT<M, G>(a, -1);
    return a;
}
// 返回模 P 的卷积，长度 |a|+|b|-1；任一为空则返回空。
poly mul(poly a, poly b, int P) {
    int n = a.size(), m = b.size();
    if(!n || !m) return poly();
    if(n == 1 && m == 1) return poly{int(1ll * a[0] * b[0] % P)};
    int l = 1 << (__lg(n + m - 2) + 1);
    a.resize(l), b.resize(l);
    static constexpr int M[3]{998244353, 1004535809, 469762049}, G = 3;
    poly c[3]{conv<M[0], G>(a, b, l), conv<M[1], G>(a, b, l), conv<M[2], G>(a, b, l)}, d;
    d.reserve(n + m - 1);
    static constexpr ll m01 = 1ll * M[0] * M[1];
    static constexpr int i0 = 669690699, i1 = 354521948;
    For(id, 0, n + m - 2) {
        auto A = c[0][id], B = c[1][id], C = c[2][id];
        ll x = 1ll * (B - A + M[1]) % M[1] * i0 % M[1] * M[0] + A;
        d.push_back((1ll * (C - x % M[2] + M[2]) % M[2] * i1 % M[2] * (m01 % P) % P + x) % P);
    }
    return d;
}
}  // namespace Mul
```

7 计算几何

7.1 选用与依赖

整数版先抄 `整数/整数点.cpp`（IGeo），浮点版先抄 `基础/二维向量.cpp`（Geo）；二者互不依赖，可混用。各算法不自动包含依赖。

- 整数线段、多边形、凸包内点判定：再抄 `整数/线段.cpp`；整数凸包与最近点对只需整数点。
- 最近点对、凸多边形逐点最远点、闵可夫斯基和：只需二维向量。
- 凸包、上凸壳、凸包内点判定、半平面交：依赖直线与线段。
- 多边形：面积与包含判定依赖直线与线段；重心和耳切剖分只需二维向量。
- 圆：先抄 `圆/圆基础.cpp`（依赖直线与线段），再选切线、圆求交、圆并或最小圆覆盖，共用 `c2`；这些模块互不依赖。
- Delaunay、Voronoi：依赖 `凸包/凸包.cpp`，二者互不依赖。
- 三维：向量 → 直线 → 平面 → 凸包；旋转只依赖向量。共用二维向量中的容差函数。

7.2 整数

7.2.1 整数点与精确运算

```cpp
// 依赖 通用/头文件.cpp 的 ll、i128。
// 整数点与精确运算（叉积/点积/距离平方），无 eps；|坐标| < 2^62 时全部运算安全。
namespace IGeo {

struct p2i {
    ll x, y;
    bool operator==(const p2i &b) const { return x == b.x && y == b.y; }
    bool operator<(const p2i &b) const { return x != b.x ? x < b.x : y < b.y; }
};
// 三点转向 (b-a)×(c-a)，逆时针为正。
i128 cross(p2i a, p2i b, p2i c) {
    return (i128(b.x) - a.x) * (i128(c.y) - a.y) -
           (i128(b.y) - a.y) * (i128(c.x) - a.x);
}
// 三点转向的精确符号，返回 -1 / 0 / 1。
int crossop(p2i a, p2i b, p2i c) {
    i128 v = cross(a, b, c);
    return (v > 0) - (v < 0);
}
// 点积 (b-a)·(c-a)。
i128 dot(p2i a, p2i b, p2i c) {
    return (i128(b.x) - a.x) * (i128(c.x) - a.x) +
           (i128(b.y) - a.y) * (i128(c.y) - a.y);
}
// 两点距离平方；保持整数，最后需要距离时再开根号。
i128 dis2(p2i a, p2i b) {
    i128 x = i128(a.x) - b.x, y = i128(a.y) - b.y;
    return x * x + y * y;
}
```

```cpp

} // namespace IGeo
```

7.2.2 整数线段

```cpp
// 依赖 整数/整数点.cpp 及其数值范围。
// 整数线段的包含、相交与严格交叉判定，全部精确、O(1)；线段可退化为点。
namespace IGeo {

struct segi { p2i x, y; };
// 点是否在线段上，包含端点与零长线段。
bool ons(segi s, p2i p) {
    return min(s.x.x, s.y.x) <= p.x && p.x <= max(s.x.x, s.y.x) &&
           min(s.x.y, s.y.y) <= p.y && p.y <= max(s.x.y, s.y.y) &&
           !crossop(s.x, s.y, p);
}
// 点是否严格在线段内部；端点和零长线段返回 false。
bool ons_s(segi s, p2i p) { return ons(s, p) && !(p == s.x) && !(p == s.y); }
// 闭线段是否相交，包含端点、共线重叠与零长线段。
bool chkss(segi a, segi b) {
    return max(min(a.x.x, a.y.x), min(b.x.x, b.y.x)) <=
               min(max(a.x.x, a.y.x), max(b.x.x, b.y.x)) &&
           max(min(a.x.y, a.y.y), min(b.x.y, b.y.y)) <=
               min(max(a.x.y, a.y.y), max(b.x.y, b.y.y)) &&
           crossop(a.x, a.y, b.x) * crossop(a.x, a.y, b.y) <= 0 &&
           crossop(b.x, b.y, a.x) * crossop(b.x, b.y, a.y) <= 0;
}
// 两线段是否严格交叉；端点相接、共线和零长线段返回 false。
bool chkss_s(segi a, segi b) {
    return crossop(a.x, a.y, b.x) * crossop(a.x, a.y, b.y) < 0 &&
           crossop(b.x, b.y, a.x) * crossop(b.x, b.y, a.y) < 0;
}

} // namespace IGeo
```

7.2.3 整数面积与点包含

```cpp
// 依赖 整数/线段.cpp。
// 整数的带号二倍面积与奇偶规则点包含，均 O(n)；顶点按边界顺序、不重复首点。
namespace IGeo {

// 带号二倍面积，逆时针为正；允许自交（代数面积），要求最终值在 i128 内。
i128 area2(const vector<p2i> &a) {
    int n = a.size();
    u128 s = 0;
    For(i, 0, n - 1) {
        p2i u = a[i], v = a[(i + 1) % n];
        s += u128(i128(u.x) * v.y) - u128(i128(u.y) * v.x);
    }
    return s >> 127 ? -1 - i128(~s) : i128(s);
}
// 点与多边形关系：0 外、1 边界、2 内；空集返回 0，自交用奇偶规则。
int contain(const vector<p2i> &a, p2i p) {
```

```
    int n = a.size();
    int ret = 0;
    For(i, 0, n - 1) {
        p2i u = a[i], v = a[(i + 1) % n];
        if(ons({u, v}, p)) return 1;
        if((u.y > p.y) != (v.y > p.y) &&
           (crossop(u, v, p) > 0) == (v.y > u.y)) ret ^= 1;
    }
    return ret * 2;
}

}  // namespace IGeo
```

7.2.4 整数凸包与直径

```
// 依赖 整数/整数点.cpp 及其数值范围。
// 整数凸包与旋转卡壳直径；原地排序去重，返回逆时针凸包、不重复首点。
namespace IGeo {

// a 原地排序去重，返回逆时针凸包，不重复首点；nos=1 保留边上共线点，默认只留端点。
// O(n log n)；空/单点照常返回；全共线默认留两端点，nos=1 按字典序各输出一次。
vector<p2i> convex_hull(vector<p2i> &a, bool nos = false) {
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    int n = a.size();
    if(n <= 1) return a;
    if(nos) {
        bool collinear = true;
        For(i, 1, n - 2) if(crossop(a[0], a[n - 1], a[i])) collinear = false;
        if(collinear) return a;
    }
    vector<p2i> b;
    b.reserve(a.size() + 1);
    for(auto p : a) {
        while(b.size() > 1 && crossop(b[b.size() - 2], b.back(), p) <= -nos)
            b.pop_back();
        b.push_back(p);
    }
    size_t t = b.size();
    rFor(i, n - 2, 0) {
        while(b.size() > t && crossop(b[b.size() - 2], b.back(), a[i]) <= -nos)
            b.pop_back();
        if(i) b.push_back(a[i]);
    }
    return b;
}
// 逆时针严格凸包的直径平方，O(n)；可直接用 nos=0 的输出，不修改输入。
// 空/单点返回 0，线段返回两点距离平方。
i128 convex_diameter(const vector<p2i> &a) {
    int n = a.size();
    if(n <= 1) return 0;
    if(n == 2) return dis2(a[0], a[1]);
    int j = 1;
    i128 ret = 0;
    For(i, 0, n - 1) {
        int k = (i + 1) % n;
        while(cross(a[i], a[k], a[(j + 1) % n]) > cross(a[i], a[k], a[j]))
            j = (j + 1) % n;
        ret = max(ret, max(dis2(a[i], a[j]), dis2(a[k], a[j])));
    }
    return ret;
}

}  // namespace IGeo
```

7.2.5 整数凸包内点判定

```
// 依赖 整数/线段.cpp。
// 整数凸包的点包含判定，O(log n)；要求逆时针严格凸包、不重复首点。
namespace IGeo {

// O(log n) 判定：0 外、1 边界、2 内；空集、单点、线段也支持，不修改输入。
int in_convex(const vector<p2i> &a, p2i p) {
    int n = a.size();
    if(n <= 2) return n && ons({a[0], a[n - 1]}, p);
    int c = crossop(a[0], a[1], p), d = crossop(a[0], a[n - 1], p);
    if(c < 0 || d > 0) return 0;
    if(c == 0) return ons({a[0], a[1]}, p);
    if(d == 0) return ons({a[0], a[n - 1]}, p);
    int l = 1, r = n - 1;
    while(l + 1 < r) {
        int m = l + (r - l) / 2;
        if(crossop(a[0], a[m], p) >= 0) l = m;
        else r = m;
    }
    c = crossop(a[l], a[r], p);
    return c < 0 ? 0 : c == 0 ? 1 : 2;
}

}  // namespace IGeo
```

7.2.6 整数最近点对

```
// 依赖 整数/整数点.cpp。
// 分治求整数最近点对，O(n log n)；原地重排 a[0..n)，距离平方与中间量在 i128 内。
namespace IGeo {

// 返回最近距离平方；重合点返回 0，n<2 返回 -1；不转浮点、不求平方根。
i128 closest_pair2(vector<p2i> &a) {
    int n = a.size();
    if(n < 2) return -1;
    sort(a.begin(), a.end());
    if(adjacent_find(a.begin(), a.end()) != a.end()) return 0;
    i128 ans = dis2(a[0], a[1]);
    vector<p2i> b(n);
    auto rec = [&](auto &&self, int l, int r) -> void {
```

```
        if(r - l <= 1) return;
        int mid = l + (r - l) / 2;
        ll mx = a[mid].x;
        self(self, l, mid), self(self, mid, r);
        int i = l, j = mid, k = l;
        while(i < mid && j < r) b[k++] = a[i].y <= a[j].y ? a[i++] : a[j++];
        while(i < mid) b[k++] = a[i++];
        while(j < r) b[k++] = a[j++];
        For(t, l, r - 1) a[t] = b[t];
        int m = 0;
        For(t, l, r - 1) {
            i128 dx = i128(a[t].x) - mx;
            if(dx * dx < ans) b[m++] = a[t];
        }
        For(t, 0, m - 1) for(int u = t - 1; u >= 0; --u) {
            i128 dy = i128(b[t].y) - b[u].y;
            if(dy * dy >= ans) break;
            ans = min(ans, dis2(b[t], b[u]));
        }
    };
    rec(rec, 0, n);
    return ans;
}

}  // namespace IGeo
```

7.2.7 整数极角排序

```
// 依赖 整数/整数点.cpp。
// 全局精确整数极角排序，无 eps、开方或三角函数；要求 |坐标| < 2^62。
namespace IGeo {

// 比较方向，返回 -1/0/1；零向量最先，其后从正 x 轴逆时针，同向返回 0。
// 四个分量均须满足 |分量| < 2^63；各积 < 2^126，叉积绝对值 < 2^127。
int direction_cmp(i128 ax, i128 ay, i128 bx, i128 by) {
    bool az = !ax && !ay, bz = !bx && !by;
    if(az || bz) return int(bz) - int(az);
    bool ah = ay < 0 || (ay == 0 && ax < 0);
    bool bh = by < 0 || (by == 0 && bx < 0);
    if(ah != bh) return ah ? 1 : -1;
    i128 d = ax * by - ay * bx;
    return (d < 0) - (d > 0);
}
// sort(a.begin(), a.end(), polar_less{中心})；默认原点；中心点最先，同向由近到远。
// 要求各坐标减中心后的绝对值 < 2^63（均在 |坐标|<2^62 内自动满足）。
// 比较为严格弱序，仅完全相同的点等价，不修改输入。
struct polar_less {
    p2i center{0, 0};
    bool operator()(p2i a, p2i b) const {
        i128 ax = i128(a.x) - center.x, ay = i128(a.y) - center.y;
        i128 bx = i128(b.x) - center.x, by = i128(b.y) - center.y;
        int c = direction_cmp(ax, ay, bx, by);
        if(c) return c < 0;

        if(ax != bx) return ax < 0 ? ax > bx : ax < bx;
        return ay < 0 ? ay > by : ay < by;
    }
};

}  // namespace IGeo
```

7.2.8 整数闵可夫斯基和

```
// 依赖 整数/整数点.cpp、整数/极角排序.cpp。
// O(n+m) 求 A+B 的逆时针严格凸包，从最低最左点开始；输入须逆时针严格凸、不重复首点。
namespace IGeo {

vector<p2i> minkowski_sum(vector<p2i> a, vector<p2i> b) {
    if(a.empty() || b.empty()) return {};
    auto start = [](vector<p2i> &p) {
        int k = 0;
        For(i, 1, (int) p.size() - 1)
            if(p[i].y < p[k].y || (p[i].y == p[k].y && p[i].x < p[k].x)) k = i;
        rotate(p.begin(), p.begin() + k, p.end());
    };
    start(a);
    start(b);
    auto add = [](p2i u, p2i v) -> p2i {
        return {ll(i128(u.x) + v.x), ll(i128(u.y) + v.y)};
    };
    if(a.size() == 1) {
        for(auto &p : b) p = add(p, a[0]);
        return b;
    }
    if(b.size() == 1) {
        for(auto &p : a) p = add(p, b[0]);
        return a;
    }
    int n = a.size(), m = b.size(), i = 0, j = 0;
    vector<p2i> ret;
    ret.reserve(a.size() + b.size());
    while(i < n || j < m) {
        ret.push_back(add(a[i % n], b[j % m]));
        if(i == n) { ++j; continue; }
        if(j == m) { ++i; continue; }
        p2i u = a[(i + 1) % n], v = b[(j + 1) % m];
        int c = direction_cmp(i128(u.x) - a[i].x, i128(u.y) - a[i].y,
                              i128(v.x) - b[j].x, i128(v.y) - b[j].y);
        if(c <= 0) ++i;
        if(c >= 0) ++j;
    }
    return ret;
}

}  // namespace IGeo
```

7.3 基础

7.3.1 二维点与向量

```cpp
// 依赖 通用/头文件.cpp 的 db 与 eps、angle_eps 两个容差常量。
// p2 兼作点与向量，是后续几何模板的共用基座；坐标须有限，角度用弧度。
namespace Geo {

const db eps = 1e-10;
const db angle_eps = 1e-10;
const db pi = acos(db(-1));
int sign(db x) { return x < -eps ? -1 : x > eps; }
int cmp(db x, db y) { return sign(x - y); }

struct p2 {
    db x, y;
    db det(const p2 &b) const { return x * b.y - y * b.x; }
    // 极角，范围 [-pi,pi]；要求非零向量。
    db alpha() const { return atan2(y, x); }
    // 按 eps 判等；不是可传递的等价关系，不能直接用于排序去重。
    bool operator==(const p2 &b) const {
        return !cmp(x, b.x) && !cmp(y, b.y);
    }
    bool operator<(const p2 &b) const {
        return x != b.x ? x < b.x : y < b.y;
    }
};
p2 operator+(p2 a, p2 b) { return {a.x + b.x, a.y + b.y}; }
p2 operator-(p2 a, p2 b) { return {a.x - b.x, a.y - b.y}; }
p2 operator*(p2 a, db k) { return {a.x * k, a.y * k}; }
p2 operator*(db k, p2 a) { return a * k; }
db dot(p2 a, p2 b) { return a.x * b.x + a.y * b.y; }
p2 operator/(p2 a, db k) { return {a.x / k, a.y / k}; }
p2 operator+=(p2 &a, p2 b) { return a = a + b; }
p2 operator-=(p2 &a, p2 b) { return a = a - b; }
p2 operator/=(p2 &a, db k) { return a = a / k; }

db dis2(p2 a) { return dot(a, a); }
// 两点距离写 dis(a-b)。
db dis(p2 a) {
    const db lo = ldexp(db(1), numeric_limits<db>::min_exponent / 2);
    const db hi = ldexp(db(1), numeric_limits<db>::max_exponent / 2 - 1);
    db m = max(abs(a.x), abs(a.y));
    return m >= lo && m <= hi ? sqrt(dis2(a)) : hypot(a.x, a.y);
}
// 单位化，要求非零向量。
p2 unit(p2 a) {
    db d = dis(a);
    if(isnormal(d)) return a / d;
    a /= max(abs(a.x), abs(a.y));
    return a / sqrt(dis2(a));
}
// 逆时针旋转 90 度。
p2 r90(p2 a) { return {-a.y, a.x}; }
// 绕原点逆时针旋转 t 弧度。
p2 rot(p2 a, db t) {
    db c = cos(t), s = sin(t);
    return {a.x * c - a.y * s, a.x * s + a.y * c};
}

}  // namespace Geo
using namespace Geo;
```

7.3.2 直线与线段

```cpp
// 依赖 基础/二维向量.cpp。
// 直线与线段的方向判定、相交、投影与距离；线段可退化为点，距离类接口返回平方。
namespace Geo {

struct seg {
    p2 x, y;
    p2 dir() const { return y - x; }
};
// 有向直线与点的叉积；q 在左侧为正。
db cross(const seg &l, const p2 &q) { return l.dir().det(q - l.x); }
// 非零方向按最大分量缩放；不改变方向，避免原始平方的溢出/下溢。
p2 scaled_dir(p2 d) { return d / max(abs(d.x), abs(d.y)); }
// 点对有向直线的侧别，垂距<=eps 算在线上；零方向返回0，供点线段使用。
int crossop(const seg &l, const p2 &q) {
    p2 d = l.dir();
    if(d.x == 0 && d.y == 0) return 0;
    d = scaled_dir(d);
    db h = d.det(q - l.x);

    if(abs(h) > 2 * eps) return (h > 0) - (h < 0);
    if(abs(h) <= eps) return 0;
    db t = eps * sqrt(dis2(d));
    return (h > t) - (h < -t);
}
// 点是否在直线上；要求 l 非零长。
bool inc(seg l, const p2 &q) { return !crossop(l, q); }
// 唯一交点：两方向非零且 |sin夹角|>angle_eps；与方向长度无关。
bool chkll(const seg &a, const seg &b) {
    p2 u = a.dir(), v = b.dir();
    if((u.x == 0 && u.y == 0) || (v.x == 0 && v.y == 0)) return false;
    u = scaled_dir(u), v = scaled_dir(v);
    db h = abs(u.det(v));
    if(h > 2 * angle_eps) return true;
    if(h <= angle_eps) return false;
    return h > angle_eps * sqrt(dis2(u) * dis2(v));
}
// 两直线近似重合；要求均非零长。
bool eqll(const seg &a, const seg &b) {
    return !chkll(a, b) && inc(a, b.x) && inc(b, a.x);
}
// 两非零直线的交点；要求真正不平行，局部偏移及结果可表示。
```

```cpp
// 推荐先chkll；夹角<=angle_eps时不保证精度。
p2 isll(const seg &a, const seg &b) {
    p2 u = scaled_dir(a.dir()), v = scaled_dir(b.dir());
    return a.x + u * ((b.x - a.x).det(v) / u.det(v));
}
// 三点转向：最小高<=eps 算共线；全排列同一阈值，奇排列只翻转符号。
// 坐标差与缩放后的叉积须可可靠判号，阈值附近仍受浮点误差影响。
int turnop(p2 a, p2 b, p2 c) {
    int s = 1;
    if(b < a) swap(a, b), s = -s;
    if(c < b) swap(b, c), s = -s;
    if(b < a) swap(a, b), s = -s;
    p2 u = b - a, v = c - a;
    db m = max({abs(u.x), abs(u.y), abs(v.x), abs(v.y)});
    if(m <= eps / 2) return 0;

    if(m <= ldexp(db(1), numeric_limits<db>::max_exponent / 4)) {
        db t = eps * m;
        if(isnormal(t)) {
            db h = u.det(v);
            if(abs(h) > 4 * t) return s * ((h > 0) - (h < 0));
        }
    }
    u /= m, v /= m;
    db h = u.det(v), t = eps / m;

    if(abs(h) > 4 * t) return s * ((h > 0) - (h < 0));
    if(abs(h) <= t) return 0;
    t *= sqrt(max({dis2(u), dis2(v), dis2(v - u)}));
    return s * ((h > t) - (h < -t));
}

// 标量 m 是否在 a、b 的闭区间内，含 eps 容差。
bool isMid(const db &a, const db &m, const db &b) {
    return cmp(a, m) * cmp(b, m) <= 0;
}
bool isMid(const p2 &a, const p2 &m, const p2 &b) {
    return isMid(a.x, m.x, b.x) && isMid(a.y, m.y, b.y);
}
// 点在线段的eps垂距带且在eps扩展包围盒内；含端点，点线段按坐标eps。
bool ons(const seg &s, const p2 &q) { return isMid(s.x, q, s.y) && !crossop(s, q); }
// 严格内部：垂距<=eps，沿段方向距两端均>eps；长度<=2eps及点线段无内部。
bool ons_s(const seg &s, const p2 &q) {
    p2 d = s.dir();
    if(d.x == 0 && d.y == 0) return false;
    d = scaled_dir(d);
    db t = eps * sqrt(dis2(d));
    p2 w = q - s.x;
    return abs(d.det(w)) <= t && dot(w, d) > t && dot(q - s.y, d) < -t;
}
// 闭线段是否相交，含端点与共线重叠；支持点线段。
bool chkss(const seg &a, const seg &b) {
    auto overlap = [](db l1, db r1, db l2, db r2) {
        if(l1 > r1) swap(l1, r1);
        if(l2 > r2) swap(l2, r2);
        return cmp(r1, l2) >= 0 && cmp(r2, l1) >= 0;
    };
    return overlap(a.x.x, a.y.x, b.x.x, b.y.x) &&
           overlap(a.x.y, a.y.y, b.x.y, b.y.y) &&
           crossop(a, b.x) * crossop(a, b.y) <= 0 &&
           crossop(b, a.x) * crossop(b, a.y) <= 0;
}
// 线段是否严格交叉；端点相接、共线与点线段不算。
bool chkss_s(const seg &a, const seg &b) {
    return crossop(a, b.x) * crossop(a, b.y) < 0 &&
           crossop(b, a.x) * crossop(b, a.y) < 0;
}

// 点在无限直线上的投影；要求 l 非零长，中间量及结果可表示。
inline p2 proj(const seg &l, const p2 &q) {
    p2 d = l.dir();
    d /= max(abs(d.x), abs(d.y));
    p2 w = q - l.x;
    db n = dis2(d), hi = numeric_limits<db>::max() / 4;

    db t = abs(w.x) <= hi && abs(w.y) <= hi ? dot(d, w) / n :
           dot(d * (1 / n), w);
    return l.x + d * t;
}
// 点关于无限直线的对称点；要求 l 非零长。
p2 reflect(const seg &l, const p2 &q) {
    p2 h = proj(l, q);
    return h + (h - q);
}
// 点到线段的最近距离平方；支持点线段，沿用投影包围盒的 eps 扩展。
inline db nearest(const seg &s, const p2 &q) {
    p2 d = s.dir(), w = q - s.x;
    db m = max(abs(d.x), abs(d.y));
    if(m == 0) return dis2(w);
    d /= m;
    db n = dis2(d), a = dot(d, w);
    if(a < -eps * n) return dis2(w);
    p2 v = q - s.y;
    db b = dot(d, v);
    if(b > eps * n) return dis2(v);
    if(a > -b) w = v;
    db h = d.det(w);
    return (h / n) * h;
}
// 两线段最近距离平方；相交返回 0，支持点线段。
db disss(const seg &a, const seg &b) {
    if(chkss(a, b)) return 0;
    return min({nearest(a, b.x), nearest(a, b.y),
                nearest(b, a.x), nearest(b, a.y)});
}

```

```
148 }  // namespace Geo
```

7.3.3 最近点对

```
// 依赖 基础/二维向量.cpp。
// 分治 + 按 y 归并求最近点对距离（非平方），O(n log n) 时间、O(n) 空间，不改输入。

namespace Geo {

db dis_closest(vector<p2> a) {
    int n = a.size();
    if(n < 2) return 1e100L;
    sort(a.begin(), a.end());
    vector<p2> b(n);
    db ans = 1e100L;
    auto rec = [&](auto &&self, int l, int r) -> void {
        if(r - l <= 1) return;
        int mid = (l + r) >> 1;
        db mx = a[mid].x;
        self(self, l, mid);
        self(self, mid, r);
        int i = l, j = mid, k = l;
        while(i < mid && j < r) b[k++] = a[i].y <= a[j].y ? a[i++] : a[j++];
        while(i < mid) b[k++] = a[i++];
        while(j < r) b[k++] = a[j++];
        for(int t = l; t < r; ++t) a[t] = b[t];
        int m = 0;
        for(int t = l; t < r; ++t)
            if(abs(a[t].x - mx) < ans) b[m++] = a[t];
        for(int t = 0; t < m; ++t)
            for(int u = t - 1; u >= 0 && b[t].y - b[u].y < ans; --u)
                ans = min(ans, dis(b[t] - b[u]));
    };
    rec(rec, 0, n);
    return ans;
}

}  // namespace Geo
```

7.4 凸包

7.4.1 凸包、直径与裁剪

```
// 依赖 基础/直线与线段.cpp。
// 凸包、旋转卡壳直径与半平面裁剪；顶点不重复首点，size() 须在 int 内。
namespace Geo {

// a 原地排序、精确去重，返回逆时针凸包（不重复首点）；nos=1 留边上共线点，默认只留端点。
// O(n log n)，转向用最小高eps；空/单点照常返回，全共线时默认留两端点，nos=1 按字典序各输出一次。
vector<p2> convex_hull(vector<p2> &a, bool nos = false) {
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end(), [](p2 x, p2 y) { return x.x == y.x && x.y == y.y; }), a.end());
    int n = a.size();
    if(n <= 1) return a;
    if(nos) {
        bool collinear = true;
        For(i, 1, n - 2) if(turnop(a[0], a[n - 1], a[i])) collinear = false;
        if(collinear) return a;
    }
    vector<p2> b;
    b.reserve(a.size() + 1);
    for(auto p : a) {
        while(b.size() > 1 && turnop(b[b.size() - 2], b.back(), p) <= -nos)
            b.pop_back();
        b.push_back(p);
    }
    size_t t = b.size();
    rFor(i, n - 2, 0) {
        while(b.size() > t && turnop(b[b.size() - 2], b.back(), a[i]) <= -nos)
            b.pop_back();
        if(i) b.push_back(a[i]);
    }
    return b;
}

// 逆时针凸多边形的直径平方，O(n)，不修改输入；n<=1 返回 0，线段也支持。
// 全共线须按沿线顺序给点，相邻重复点须先去掉。
db convex_diameter(const vector<p2> &a) {
    int n = a.size();
    if(n <= 1) return 0;
    int is = 0, js = 0;
    For(k, 1, n - 1) {
        if(a[k] < a[is]) is = k;
        if(a[js] < a[k]) js = k;
    }
    int i = is, j = js;
    db ret = dis2(a[i] - a[j]);
    do {
        if((a[(i + 1) % n] - a[i]).det(a[(j + 1) % n] - a[j]) >= 0) j = (j + 1) % n;
        else i = (i + 1) % n;
        ret = max(ret, dis2(a[i] - a[j]));
    } while(i != is || j != js);
    return ret;
}

// 用非零有向直线 q 裁凸多边形，保留左侧及边界；O(n)，不修改输入、保持旋向。
// 允许空/点/线段，结果可能有重复相邻点。
vector<p2> convex_cut(const vector<p2> &a, const seg &q) {
    int n = a.size();
    vector<p2> ret;
    For(i, 0, n - 1) {
        p2 u = a[i], v = a[(i + 1) % n];
        int d1 = crossop(q, u), d2 = crossop(q, v);
        if(d1 >= 0) ret.push_back(u);
        if(d1 * d2 < 0) ret.push_back(isll({u, v}, q));
```

```
    }
    return ret;
}

}  // namespace Geo
```

7.4.2 凸多边形逐点最远点

```
// 依赖 基础/二维向量.cpp。
// O(n log n) 求凸多边形每点的最远点下标；并列取从 i+1 起沿输入顺序遇到的首点。
namespace Geo {

vector<int> convex_farthest(const vector<p2> &a) {
    int n = a.size();
    vector<int> ans(n);
    if(n <= 1) return ans;

    auto solve = [&](auto &&self, int l, int r, int ql, int qr) -> void {
        if(l > r) return;
        int m = (l + r) / 2, k = max(ql, m + 1);
        db best = dis2(a[m] - a[k % n]);
        For(j, k + 1, min(qr, m + n - 1)) {
            db d = dis2(a[m] - a[j % n]);
            if(d > best) { best = d; k = j; }
        }
        ans[m] = k;
        self(self, l, m - 1, ql, k);
        self(self, m + 1, r, k, qr);
    };
    solve(solve, 0, n - 1, 1, 2 * n - 2);
    for(int &k : ans) k %= n;
    return ans;
}

}  // namespace Geo
```

7.4.3 闵可夫斯基和

```
// 依赖 基础/二维向量.cpp。
// O(n+m) 求 A+B 的逆时针凸包（从最低最左点起、不重复首点）；输入须逆时针严格凸。
namespace Geo {

// 求凸集 A+B={a+b}；要求如上，坐标和与叉积须在 db 的有限精度范围内。
vector<p2> minkowski_sum(vector<p2> a, vector<p2> b) {
    if(a.empty() || b.empty()) return {};
    auto start = [](vector<p2> &p) {
        int k = 0;
        For(i, 1, (int) p.size() - 1)
            if(p[i].y < p[k].y || (p[i].y == p[k].y && p[i].x < p[k].x)) k = i;
        rotate(p.begin(), p.begin() + k, p.end());
    };
    start(a);
    start(b);
    if(a.size() == 1) {
        for(p2 &p : b) p += a[0];
        return b;
    }
    if(b.size() == 1) {
        for(p2 &p : a) p += b[0];
        return a;
    }

    auto half = [](p2 p) { return p.y < 0 || (p.y == 0 && p.x < 0); };
    int n = a.size(), m = b.size(), i = 0, j = 0;
    vector<p2> ret;
    while(i < n || j < m) {
        ret.push_back(a[i % n] + b[j % m]);
        if(i == n) { ++j; continue; }
        if(j == m) { ++i; continue; }
        p2 u = a[(i + 1) % n] - a[i], v = b[(j + 1) % m] - b[j];
        if(half(u) != half(v)) {
            if(half(u) < half(v)) ++i;
            else ++j;
        } else {
            db d = u.det(v);
            if(d >= 0) ++i;
            if(d <= 0) ++j;
        }
    }
    return ret;
}

}  // namespace Geo
```

7.4.4 上凸壳

```
// 依赖 基础/直线与线段.cpp。
// O(n log n) 求从左到右的上凸链；a 原地按 x 递增、y 递减排序，每个 x 只留最大 y。
namespace Geo {

vector<p2> upper_hull(vector<p2> &a, bool nos = false) {
    sort(a.begin(), a.end(), [](p2 u, p2 v) { return u.x != v.x ? u.x < v.x : u.y > v.y; });
    vector<p2> b;
    b.reserve(a.size());
    for(auto p : a) {
        if(!b.empty() && b.back().x == p.x) continue;
        while(b.size() > 1 && turnop(b[b.size() - 2], b.back(), p) >= (int) nos)
            b.pop_back();
        b.push_back(p);
    }
    return b;
}

}  // namespace Geo
```

7.4.5 凸包内点判定

```cpp
// 依赖 基础/直线与线段.cpp。
// 以 a[0] 为扇心二分，O(log n) 返回0 外/1 边界/2 内；要求逆时针严格凸、不重复首点。

namespace Geo {

int in_convex(const vector<p2> &a, const p2 &p) {
    int n = a.size();
    if(n < 3) return 0;
    if(crossop({a[0], a[1]}, p) < 0 || crossop({a[0], a[n - 1]}, p) > 0) return 0;
    if(ons({a[0], a[1]}, p) || ons({a[n - 1], a[0]}, p)) return 1;
    int l = 1, r = n - 1;
    while(l + 1 < r) {
        int m = (l + r) >> 1;
        if(crossop({a[0], a[m]}, p) >= 0)
            l = m;
        else
            r = m;
    }
    int c = crossop({a[l], a[l + 1]}, p);
    if(c == 0) return 1;
    if(c < 0) return 0;
    if(crossop({a[0], a[l]}, p) == 0) return ons({a[0], a[l]}, p) ? 2 : 0;
    return 2;
}

}  // namespace Geo
```

7.4.6 半平面交

```cpp
// 依赖 基础/直线与线段.cpp。
// 有向直线左侧求交 O(n log n)；要求交集有界（通常加包围盒），退化（点/线段）返回空。

namespace Geo {

// 按精确极角、单位法向偏移排序，同角强约束在前；方向须非零，偏移须有限。
bool hpi_less(const seg &a, const seg &b) {
    db x = a.dir().alpha(), y = b.dir().alpha();
    if(x != y) return x < y;
    return unit(a.dir()).det(a.x) > unit(b.dir()).det(b.x);
}
// 近同向合并时a基点在b左侧则取a；缩放后比较，避免极短方向乘积下溢。
bool hpi_stronger(const seg &a, const seg &b) {
    return scaled_dir(b.dir()).det(a.x - b.x) > 0;
}
vector<p2> hpi(vector<seg> vs) {
    if(vs.empty()) return {};
    struct key { db angle, offset; int id; };
    vector<key> order;
    order.reserve(vs.size());
    for(int i = 0; i < (int) vs.size(); ++i) {
        p2 d = vs[i].dir();
        order.push_back({d.alpha(), unit(d).det(vs[i].x), i});
    }

    sort(order.begin(), order.end(), [](const key &a, const key &b) {
        return a.angle != b.angle ? a.angle < b.angle : a.offset > b.offset;
    });
    vector<seg> ls;
    db last = 0;
    for(auto k : order) {
        seg l = vs[k.id];
        if(!ls.empty() && abs(last - k.angle) <= angle_eps) {
            if(hpi_stronger(l, ls.back())) ls.back() = l, last = k.angle;
            continue;
        }
        ls.push_back(l);
        last = k.angle;
    }
    if(ls.size() > 1 && abs(ls.front().dir().alpha() + 2 * pi - last) <= angle_eps)
{
        if(hpi_stronger(ls.back(), ls.front()))
            ls.erase(ls.begin());
        else
            ls.pop_back();
    }
    int n = ls.size();
    if(n < 3) return {};
    vector<int> q(n + 1);
    int h = 0, t = 2;
    q[0] = 0, q[1] = 1;
    for(int i = 2; i < n; ++i) {
        while(t - h >= 2 && crossop(ls[i], isll(ls[q[t - 2]], ls[q[t - 1]])) < 0) --
t;
        while(t - h >= 2 && crossop(ls[i], isll(ls[q[h]], ls[q[h + 1]])) < 0) ++h;
        q[t++] = i;
    }
    while(t - h >= 2 && crossop(ls[q[h]], isll(ls[q[t - 2]], ls[q[t - 1]])) < 0) --
t;
    while(t - h >= 2 && crossop(ls[q[t - 1]], isll(ls[q[h]], ls[q[h + 1]])) < 0) +
+h;
    if(t - h < 3) return {};
    vector<p2> ret;
    for(int i = h; i < t; ++i) ret.push_back(isll(ls[q[i]], ls[q[i + 1 < t ? i + 1 :
h]]));
    vector<p2> qs;
    for(int i = 0; i < (int) ret.size(); ++i) if(qs.empty() || !(qs.back() ==
ret[i])) qs.push_back(ret[i]);
    while(qs.size() > 1 && qs.front() == qs.back()) qs.pop_back();
    if(qs.size() < 3) return {};
    db s = 0;
    for(int i = 1; i + 1 < (int) qs.size(); ++i) s += (qs[i] - qs[0]).det(qs[i + 1]
- qs[0]);
    db p = 0;
```


32

```
    for(int i = 0; i < (int) qs.size(); ++i) p += dis(qs[(i + 1) % qs.size()] - qs[i]);
    if(sign(s / p) <= 0) return {};
    return qs;
}

}  // namespace Geo
```

7.4.7 凸包方向极值

```
// 依赖 基础/二维向量.cpp。
// O(log n) 时间、O(1) 额外空间求 dot(a[i],d) 最大的下标；要求严格逆时针凸包，d 非零。
namespace Geo {

// 返回 dot(a[i],d) 最大的下标；空集 -1；并列取极值边的逆时针起点（两点取 0）。
int convex_extreme(const vector<p2> &a, p2 d) {
    int n = a.size();
    if(!n) return -1;
    int e;
    frexp(max(abs(d.x), abs(d.y)), &e);
    d = {ldexp(d.x, -e), ldexp(d.y, -e)};

    auto cmp = [&](int i, int j) {
        db v = dot(d, a[j % n] - a[i % n]);
        return (v > 0) - (v < 0);
    };
    if(n <= 2) return n == 2 && cmp(0, 1) > 0 ? 1 : 0;
    auto peak = [&](int i) {
        return cmp(i + 1, i) >= 0 && cmp(i, i ? i - 1 : n - 1) < 0;
    };
    int l = 0, r = n;
    if(peak(0)) return 0;
    while(l + 1 < r) {
        int m = l + (r - l) / 2;
        if(peak(m)) return m;
        int ls = cmp(l + 1, l), ms = cmp(m + 1, m);
        if(ls < ms || (ls == ms && ls == cmp(l, m))) r = m;
        else l = m;
    }
    return l;
}

}  // namespace Geo
```

7.4.8 凸包直线求交

```
// 依赖 凸包/凸包方向极值.cpp。
// O(log n) 求凸包与无限直线的交集端点（无交 0 / 相切 1 / 穿过或重合边 2），不用 eps 与开方。
namespace Geo {

// 返回凸包与直线交集的端点，沿 d 排序且不重复：无交 0，相切 1，穿过/重合边 2。
// 空包返回空；单点在线上才返回；两点按闭线段处理；不修改输入。
vector<p2> convex_line(const vector<p2> &a, p2 o, p2 d) {
    int n = a.size();
    if(!n) return {};
    int e;
    frexp(max(abs(d.x), abs(d.y)), &e);
    d = {ldexp(d.x, -e), ldexp(d.y, -e)};
    auto side = [&](int i) { return d.det(a[i] - o); };
    int hi = convex_extreme(a, r90(d));
    int lo = convex_extreme(a, r90(d) * -1);
    db low = side(lo), high = side(hi);
    if(low > 0 || high < 0) return {};
    vector<p2> ans;
    if(low == 0 || high == 0) {
        int k = low == 0 ? lo : hi;
        ans.push_back(a[k]);
        if(n > 1) {
            int j = (k + 1) % n;
            if(side(j) != 0) j = k ? k - 1 : n - 1;
            if(j != k && side(j) == 0) ans.push_back(a[j]);
        }
    } else {

        auto cut = [&](int s, int t) {
            int l = 0, r = t >= s ? t - s : n - (s - t);
            auto index = [&](int k) { return k < n - s ? s + k : k - (n - s); };
            bool positive = side(s) > 0;
            while(l + 1 < r) {
                int m = l + (r - l) / 2;
                db v = side(index(m));
                if(v == 0) return a[index(m)];
                if((v > 0) == positive) l = m;
                else r = m;
            }
            int u = index(l), v = index(r);
            db x = side(u), y = side(v);
            return a[u] + (a[v] - a[u]) * (x / (x - y));
        };
        ans.push_back(cut(lo, hi));
        if(n > 2) ans.push_back(cut(hi, lo));
    }
    if(ans.size() == 2 && dot(ans[1] - ans[0], d) < 0) swap(ans[0], ans[1]);
    return ans;
}

}  // namespace Geo
```

7.5 多边形

7.5.1 面积与点包含

```
// 依赖 基础/直线与线段.cpp。
// 多边形的带号面积与奇偶规则点包含，均 O(n)；顶点 a[0..n) 按边界顺序、不重复首点。
namespace Geo {

```

```
// 带号面积，逆时针为正；n<=2 返回 0，自交时为代数面积；点差与局部叉积须可表示。
db area(const vector<p2> &a) {
    int n = a.size();
    db ret = 0;
    For(i, 1, n - 2) ret += (a[i] - a[0]).det(a[i + 1] - a[0]);
    return ret / 2;
}
// 点与多边形关系：0 外、1 边界、2 内；支持凹多边形与重复顶点，空集返回 0。
int contain(const vector<p2> &a, const p2 &p) {
    int n = a.size();
    int ret = 0;
    For(i, 0, n - 1) {
        p2 u = a[i], v = a[(i + 1) % n];
        if(ons({u, v}, p)) return 1;
        if(cmp(u.y, v.y) <= 0) swap(u, v);
        if(cmp(p.y, u.y) > 0 || cmp(p.y, v.y) <= 0) continue;
        ret ^= crossop({u, v}, p) > 0;
    }
    return ret * 2;
}

}  // namespace Geo
```

7.5.2 缠绕数包含

```
// 依赖 基础/直线与线段.cpp。
// 缠绕数判定点与多边形的关系，O(n)；要求 n >= 3，顶点可顺/逆时针。

namespace Geo {

int wn_contain(const vector<p2> &a, const p2 &p) {
    int n = a.size();
    if(n < 3) return 0;
    int wn = 0;
    for(int i = 0; i < n; ++i) {
        p2 u = a[i], v = a[(i + 1) % n];
        if(u == v) continue;
        if(ons({u, v}, p)) return 1;
        int su = sign(u.y - p.y), sv = sign(v.y - p.y);
        if(su <= 0 && sv > 0 && crossop({u, v}, p) > 0) ++wn;
        else if(su > 0 && sv <= 0 && crossop({u, v}, p) < 0) --wn;
    }
    return wn ? 2 : 0;
}

}  // namespace Geo
```

7.5.3 多边形重心

```
// 依赖 基础/二维向量.cpp。
// 周长与带号面积加权的重心，均 O(n)；顶点按边界顺序、可顺/逆时针，重心可能在外部。

namespace Geo {

// 多边形闭合周长(顶点顺序任意)。
db perimeter(const vector<p2> &a) {
    int n = a.size();
    db s = 0;
    for(int i = 0; i < n; ++i) s += dis(a[(i + 1) % n] - a[i]);
    return s;
}
// 面积重心；|二倍面积|/周长<=eps 时返回顶点平均，空集返回 (0,0)。
p2 centroid(const vector<p2> &a) {
    int n = a.size();
    if(n == 0) return {0, 0};
    p2 ret = {0, 0};
    db s = 0, half_l1 = 0;
    for(int i = 0; i < n; ++i) {
        p2 u = a[i] - a[0], v = a[(i + 1) % n] - a[0];
        db c = u.det(v);
        s += c;
        ret = ret + (u + v) * c;
        p2 d = a[(i + 1) % n] - a[i];
        half_l1 += abs(d.x) / 2 + abs(d.y) / 2;
    }

    if(half_l1 > 0 && abs(s) / half_l1 > 4 * eps)
        return a[0] + ret / (3 * s);
    db p = perimeter(a);
    if(p == 0 || !sign(s / p)) {
        p2 av = {0, 0};
        for(int i = 0; i < n; ++i) av = av + (a[i] - a[0]);
        return a[0] + av / (db) n;
    }
    return a[0] + ret / (3 * s);
}

}  // namespace Geo
```

7.5.4 简单多边形三角剖分

```
// 依赖 基础/二维向量.cpp。
// 耳切剖分逆时针简单多边形，O(n²)；返回原顶点下标三角形，真顶点 m 个时共 m-2 个。

namespace Geo {

vector<array<int, 3>> ear_clip(const vector<p2> &a) {
    int n = a.size();
    vector<array<int, 3>> ret;
    if(n < 3) return ret;
    vector<int> prv(n), nxt(n), blocks(n, -1);
    vector<char> live(n, 1);
    int cnt = n;
    For(i, 0, n - 1) prv[i] = i ? i - 1 : n - 1, nxt[i] = i + 1 == n ? 0 : i + 1;
    auto inside = [&](p2 p, p2 x, p2 y, p2 z) {
```

```
        return (y - x).det(p - x) >= 0 && (z - y).det(p - y) >= 0 && (x - z).det(p - z) >= 0;
    };
    auto drop = [&](int x) { nxt[prv[x]] = nxt[x], prv[nxt[x]] = prv[x], live[x] = 0, --cnt; };
    for(bool changed = true; changed && cnt > 3;) {
        changed = false;
        For(x, 0, n - 1) if(live[x] && cnt > 3) {
            p2 u = a[x] - a[prv[x]], v = a[nxt[x]] - a[x];
            if(abs(u.det(v)) <= 1e-13L * dis(u) * dis(v)) drop(x), changed = true;
        }
    }
    queue<int> ears;
    auto refresh = [&](int x) {
        blocks[x] = -1;
        int p = prv[x], q = nxt[x];
        if(!live[x] || (a[x] - a[p]).det(a[q] - a[x]) <= 0) return;
        blocks[x] = 0;
        For(i, 0, n - 1) if(live[i] && i != p && i != x && i != q)
            blocks[x] += inside(a[i], a[p], a[x], a[q]);
        if(!blocks[x]) ears.push(x);
    };
    For(i, 0, n - 1) if(live[i]) refresh(i);
    while(cnt > 2) {
        while(!ears.empty() && (!live[ears.front()] || blocks[ears.front()] != 0)) ears.pop();
        if(ears.empty()) return {};
        int x = ears.front(), p = prv[x], q = nxt[x];
        ears.pop();
        ret.push_back({p, x, q});
        drop(x);
        if(cnt == 2) break;

        For(i, 0, n - 1) if(live[i] && i != p && i != q && blocks[i] > 0 &&
                inside(a[x], a[prv[i]], a[i], a[nxt[i]]))
            if(--blocks[i] == 0) ears.push(i);
        refresh(p), refresh(q);
    }
    return ret;
}

}  // namespace Geo
```

7.6 圆

7.6.1 圆与内外接圆

```
// 依赖 基础/直线与线段.cpp。
// 圆与内外接圆、包含关系与三点外接圆；r >= 0，r=0 表示点圆。
namespace Geo {

struct c2 {
    p2 o;
    db r;
};
// 以 ab 为直径的圆；允许 a=b。
c2 cir(p2 a, p2 b) { return {(a + b) / 2, dis(a - b) / 2}; }
// 三角形内切圆；点差及结果须有限；共线时 r=0，全重合时圆心取 a。
c2 incir(p2 a, p2 b, p2 c) {
    auto span = [](p2 p) { return max(abs(p.x), abs(p.y)); };
    if(span(c - b) < min(span(b - a), span(c - a))) swap(a, b);
    p2 u = b - a, v = c - a;
    db U = max(abs(u.x), abs(u.y)), V = max(abs(v.x), abs(v.y)), s = max(U, V);
    if(s == 0) return {a, 0};
    db B = V ? dis(v / V) : 0, C = U ? dis(u / U) : 0;
    db p = (U / s) * C + (V / s) * B + dis(u / s - v / s);
    auto mul = [&](db x, db y) {
        if(abs(x) < abs(y)) swap(x, y);
        return (x / s) * (y / p);
    };
    p2 o{mul(u.x, V) * B + mul(v.x, U) * C,
         mul(u.y, V) * B + mul(v.y, U) * C};
    return {a + o, abs(mul(u.x, v.y) - mul(u.y, v.x))};
}
// 点是否在圆盘内，含边界，按长度 eps 判定；字面量显式用 p2{...} 或 c2{...}。
bool in_circle(const c2 &c, const p2 &p) { return cmp(dis(c.o - p), c.r) <= 0; }
// a 是否包含 b，含内切、重合及点圆；同样按长度 eps 判定。
bool in_circle(const c2 &a, const c2 &b) {
    db d = dis(a.o - b.o), x = max(d, b.r);

    return cmp(a.r, x) >= 0 && cmp(a.r - x, min(d, b.r)) >= 0;
}
// 三点外接圆直径；最小高<=eps 返回-1，点差及直径须可表示。
db circumcircle_diameter(p2 a, p2 b, p2 c) {
    if(!turnop(a, b, c)) return -1;
    p2 u = scaled_dir(b - a), v = scaled_dir(c - a);
    db sine = abs(u.det(v)) / sqrt(dis2(u) * dis2(v));
    return dis(b - c) / sine;
}
// 三点外心 {x,y}；要求不共线，点差、局部运算及结果可表示；近共线仍病态。
array<db, 2> circumcenter(p2 a, p2 b, p2 c) {
    p2 u = b - a, v = c - a;
    db U = max(abs(u.x), abs(u.y)), V = max(abs(v.x), abs(v.y));
    u = u / U, v = v / V;
    db d = 2 * u.det(v);
    p2 o = (r90(u) * (dis2(v) * V) - r90(v) * (dis2(u) * U)) / d;
    return {a.x + o.x, a.y + o.y};
}

}  // namespace Geo
```

7.6.2 点圆与两圆切线

```
// 依赖 圆/圆基础.cpp。
// 点圆与两圆的切线，O(1)；半径非负，极近相切时条数受浮点舍入影响。
namespace Geo {
```

```

struct tangent {
    p2 a, b, dir;  // 两圆上的切点与单位切线方向；相切时 a=b，dir 仍非零。
};
// inner=0 求外公切线，inner=1 求内公切线，返回 0/1/2 条；两点圆返回连线。
vector<tangent> tangents(c2 a, c2 b, bool inner = false) {
    p2 d = b.o - a.o;
    db z = dis(d), r = inner ? -b.r : b.r, dr = a.r - r;
    if(z == 0 || z < abs(dr)) return {};
    p2 u = d / z;
    db q = dr / z, h = sqrt(max(db(0), (z - abs(dr)) / z * (1 + abs(q))));
    vector<tangent> out;
    int count = (h == 0 || (a.r == 0 && b.r == 0)) ? 1 : 2;
    out.reserve(count);
    For(k, 0, count - 1) {
        p2 normal = u * q + r90(u) * (k ? -h : h);
        out.push_back({a.o + normal * a.r, b.o + normal * r, r90(normal)});
    }
    return out;
}
// 过点 p 作圆的切线；切点在 a，b=p；点在圆内或与点圆重合时返回空。
vector<tangent> tangents(c2 c, p2 p) { return tangents(c, c2{p, 0}); }

}  // namespace Geo
```

7.6.3 圆相关求交与面积交

```
// 依赖 圆/圆基础.cpp（与最小圆覆盖共用 c2）。
// 圆与直线/线段/圆/多边形的交点与面积交：交点 O(1)，多边形面积 O(n)；无孤立交点返回空。

namespace Geo {

// 圆与直线返回 0/1/2 交点，沿 l.x→l.y 递增；按径向距离差 eps 判相切。
// 仅数学零方向退化；点差及局部运算须有限，圆心距直线的舍入误差须小于 eps。
vector<p2> cir_line(const c2 &c, const seg &l) {
    p2 d = l.y - l.x;
    db s = max(abs(d.x), abs(d.y));
    if(s == 0) return {};
    p2 u = d / s;
    u = u / dis(u);
    db side = u.det(l.x - c.o), dist = abs(side);
    p2 f = c.o + r90(u) * side;
    int k = cmp(c.r, dist);
    if(k < 0) return {};
    if(k == 0) return {f};
    db h = sqrtl(c.r - dist) * sqrtl(c.r + dist);
    return {f - u * h, f + u * h};
}

// 圆与线段的交点，沿 l.x→l.y 递增；用 ons 过滤直线交点。
vector<p2> cir_seg(const c2 &c, const seg &l) {
    auto r = cir_line(c, l);
    r.erase(remove_if(r.begin(), r.end(), [&](p2 p) { return !ons(l, p); }), r.end());
    return r;
}

// 交点按相对 c1.o 的精确极角递增；点圆按径向长度 eps，两点圆在 eps 内取 c1.o。
// 非点圆圆心距<=eps 视同心，按第一圆 h²/r 的长度 eps 判近切，再过滤径向残差。
vector<p2> cir_cir(const c2 &c1, const c2 &other) {
    p2 dv = other.o - c1.o;
    db d = dis(dv);
    if(c1.r == 0) return !cmp(d, other.r) ? vector<p2>{c1.o} : vector<p2>{};
    if(other.r == 0) return !cmp(d, c1.r) ? vector<p2>{other.o} : vector<p2>{};
    if(!sign(d)) return {};
    p2 u = dv / d;
    db x = (d * d + c1.r * c1.r - other.r * other.r) / (2 * d);
    db h2 = c1.r * c1.r - x * x;

    db sc2 = h2 / c1.r;
    if(sign(sc2) < 0) return {};
    vector<p2> r;
    r.reserve(2);
    p2 base = c1.o + u * x;
    if(sign(sc2) > 0) {
        db h = sqrtl(h2);
        p2 w = r90(u) * h;
        r.push_back(base + w);
        r.push_back(base - w);
    } else
        r.push_back(base);
    auto good = [&](const p2 &p) {
        db t1 = dis(p - c1.o) - c1.r, t2 = dis(p - other.o) - other.r;
        db e = eps + db(1e-10) * max(c1.r, other.r);
        return abs(t1) <= e && abs(t2) <= e;
    };
    r.erase(remove_if(r.begin(), r.end(), [&](p2 p) { return !good(p); }), r.end());
    sort(r.begin(), r.end(), [&](const p2 &a, const p2 &b) { return (a - c1.o).alpha() < (b - c1.o).alpha(); });
    return r;
}

// 两圆交面积；按长度 eps 判内外切，内含取小圆面积，外离返回 0；平方等中间量须可表示。
db cir_cir_area(const c2 &c1, const c2 &other) {
    db r1 = c1.r, r2 = other.r;
    if(r1 < r2) swap(r1, r2);
    db d = dis(c1.o - other.o);
    if(d >= r1 + r2 - eps) return 0;
    if(d <= r1 - r2 + eps) return pi * r2 * r2;
    db a1 = 2 * acosl((r1 * r1 + d * d - r2 * r2) / (2 * r1 * d));
    db a2 = 2 * acosl((r2 * r2 + d * d - r1 * r1) / (2 * r2 * d));
    db x = (d * d + r1 * r1 - r2 * r2) / (2 * d);
    db h = sqrtl(max((db) 0, r1 * r1 - x * x));
    return max((db) 0, (r1 * r1 * a1 / 2 - d * h / 2) + (r2 * r2 * a2 / 2 - d * h / 2));
```

```
}

// 有向夹角，保留 atan2 的正负号，不能归一到 [0,2pi)。
db arg2(const p2 &u, const p2 &v) { return atan2l(u.det(v), dot(u, v)); }
// 圆盘与三角形 (c.o,a,b) 的有向交面积的两倍；内部平移到圆心。
db tri_disk2(const c2 &c, p2 a, p2 b) {
    a = a - c.o, b = b - c.o;
    db r2 = c.r * c.r;

    int va = cmp(dis(a), c.r) <= 0, vb = cmp(dis(b), c.r) <= 0;
    if(va && vb) return a.det(b);
    vector<p2> v = cir_seg(c2{{0, 0}, c.r}, {a, b});
    if(v.empty()) return arg2(a, b) * r2;
    db s = 0;
    s += va ? a.det(v[0]) : arg2(a, v[0]) * r2;
    s += vb ? v.back().det(b) : arg2(v.back(), b) * r2;
    if(v.size() > 1) s += v[0].det(v[1]);
    return s;
}

// 逆时针简单多边形（可凹）与圆的交面积，非负；r<=eps 或不足三点返回 0。
db cir_poly_area(const vector<p2> &a, const c2 &c) {
    int n = a.size();
    if(n < 3 || !sign(c.r)) return 0;
    db s = 0;
    for(int i = 0; i < n; ++i) s += tri_disk2(c, a[i], a[(i + 1) % n]);
    return s > 0 ? s / 2 : 0;
}

}  // namespace Geo
```

7.6.4 圆并面积与周长

```
// 依赖 圆/圆基础.cpp。
// O(n² log n) 求圆并的面积与周长（包括洞的边界）；半径非负，极近相切受浮点舍入影响。
namespace Geo {

// 求所有圆盘并集的 {面积,周长}，包括洞的边界长度；空集返回 {0,0}。
array<db, 2> cir_union(const vector<c2> &a) {
    int n = a.size();
    auto ge = [](db x, db y, db z) {
        return x >= max(y, z) && x - max(y, z) >= min(y, z);
    };
    vector<int> f(n);
    iota(f.begin(), f.end(), 0);
    auto root = [&](int x) { while(f[x] != x) x = f[x] = f[f[x]]; return x; };

    For(i, 0, n - 1) For(j, 0, i - 1)
        if(a[i].r > 0 && a[j].r > 0 && !ge(dis(a[i].o - a[j].o), a[i].r, a[j].r))
            f[root(i)] = root(j);
    db area = 0, perimeter = 0;
    For(i, 0, n - 1) {
        db r = a[i].r;
        if(r == 0) continue;
        vector<array<db, 2>> arcs;
        bool covered = false;
        For(j, 0, n - 1) if(j != i && a[j].r > 0) {
            db R = a[j].r;
            p2 v = a[j].o - a[i].o;
            db d = dis(v);
            if(ge(R, d, r) && (R > r || d > 0 || j < i)) { covered = true; break; }
            if(ge(d, r, R) || ge(r, d, R)) continue;
            db scale = max({d, r, R});
            auto gap = [&](db x, db y, db z) {
                return max(db(0), min(y, z) / scale - (x - max(y, z)) / scale);
            };

            db x = sqrt(gap(d, r, R)) * sqrt(gap(r, d, R));
            db y = sqrt(d / scale + r / scale + R / scale) * sqrt(gap(R, d, r));
            db half = 2 * atan2(x, y);
            if(half == 0) continue;
            db mid = v.alpha();
            if(mid < 0) mid += 2 * pi;
            db l = mid - half, h = mid + half;
            if(l < 0) {
                arcs.push_back({l + 2 * pi, 2 * pi}); arcs.push_back({0, h});
            } else if(h > 2 * pi) {
                arcs.push_back({l, 2 * pi}); arcs.push_back({0, h - 2 * pi});
            } else arcs.push_back({l, h});
        }
        if(covered) continue;
        if(arcs.empty()) { area += pi * r * r; perimeter += 2 * pi * r; continue; }
        p2 o = a[i].o - a[root(i)].o;
        auto add = [&](db l, db h) {
            db d = h - l, correction = 0;
            if(o.x != 0 || o.y != 0) {
                db mid = (l + h) / 2;
                correction = sin(d / 2) * dot(o, p2{cos(mid), sin(mid)});
            }
            area += r * (r * d / 2 + correction);
            perimeter += r * d;
        };
        sort(arcs.begin(), arcs.end());
        db last = 0;
        for(auto v : arcs) {
            if(v[0] > last) add(last, v[0]);
            last = max(last, v[1]);
        }
        if(last < 2 * pi) add(last, 2 * pi);
    }
    return {max(db(0), area), perimeter};
}

}  // namespace Geo
```

7.6.5 最小圆覆盖

```cpp
// 依赖 圆/圆基础.cpp。
// 随机增量法求最小覆盖圆，期望 O(n)、最坏 O(n³)；三点共线时取最远两点为直径。
namespace Geo {

// 返回覆盖所有点的最小圆；空输入返回 {{0,0},-1} 作无解标记。
c2 mcc(vector<p2> a) {
    if(a.empty()) return {{0, 0}, -1};
    static mt19937_64 mcc_rng(chrono::steady_clock::now().time_since_epoch().count());
    shuffle(a.begin(), a.end(), mcc_rng);
    c2 c = {a[0], 0};
    for(int i = 1; i < (int) a.size(); ++i) {
        if(in_circle(c, a[i])) continue;
        c = cir(a[0], a[i]);
        for(int j = 0; j < i; ++j) {
            if(in_circle(c, a[j])) continue;
            c = cir(a[i], a[j]);
            for(int k = 0; k < j; ++k) {
                if(in_circle(c, a[k])) continue;
                if(!turnop(a[i], a[j], a[k])) {
                    db dij = dis2(a[i] - a[j]), dik = dis2(a[i] - a[k]), djk = dis2(a[j] - a[k]);
                    if(dij >= dik && dij >= djk)
                        c = cir(a[i], a[j]);
                    else if(dik >= djk)
                        c = cir(a[i], a[k]);
                    else
                        c = cir(a[j], a[k]);
                } else {
                    auto o = circumcenter(a[i], a[j], a[k]);
                    p2 oc = {o[0], o[1]};
                    c = {oc, dis(oc - a[k])};
                }
            }
        }
    }
    return c;
}

}  // namespace Geo
```

7.7 平面剖分

7.7.1 Delaunay 三角剖分

```cpp
// 依赖 凸包/凸包.cpp 的凸包与凸包内点判定。
// 随机增量 + Lawson 翻边求 Delaunay，返回拼平的逆时针三角形坐标；最坏 O(n²)。
namespace Geo {
namespace Delaunay {

vector<p2> pts;  // 去重后的点,按 (x, y) 精确升序
struct Tri { int v[3], f[3], ver; };  // v 逆时针;f[i] 是顶点 i 对边的邻接三角形(-1 表示凸包边)
vector<Tri> tri;
vector<char> live;
vector<int> fre;
int verc;

// 分配三角形槽，优先复用空闲下标。
int nw() {
    if(!fre.empty()) {
        int t = fre.back();
        fre.pop_back(), live[t] = 1;
        return t;
    }
    tri.push_back(Tri()), live.push_back(1);
    return (int) tri.size() - 1;
}
void rm(int t) { live[t] = 0, fre.push_back(t); }
bool lt(p2 u, p2 v) { return u.x != v.x ? u.x < v.x : u.y < v.y; }
// pts 下标对应的三点转向，与凸包共用容差。
int ori(int a, int b, int c) { return turnop(pts[a], pts[b], pts[c]); }
// 按相对容差判断 p 是否在外接圆内；要求 a,b,c 按逆时针排列。
int ill(int p, int a, int b, int c) {
    p2 A = pts[a] - pts[p], B = pts[b] - pts[p], C = pts[c] - pts[p];
    db t1 = dis2(A) * B.det(C), t2 = dis2(B) * C.det(A), t3 = dis2(C) * A.det(B);
    return t1 + t2 + t3 > 1e-12L * (abs(t1) + abs(t2) + abs(t3));
}
// 返回三角形 t 中有向边 (a,b) 的编号 [0,2]，不存在返回 -1。
int finde(int t, int a, int b) {
    for(int i = 0; i < 3; ++i) if(tri[t].v[(i + 1) % 3] == a && tri[t].v[(i + 2) % 3] == b) return i;
    return -1;
}
// 将 t 的边 i 与共享反向边的 g 双向相连；g=-1 标记凸包边。
void lnk(int t, int i, int g) {
    tri[t].f[i] = g;
    if(g >= 0) tri[g].f[finde(g, tri[t].v[(i + 2) % 3], tri[t].v[(i + 1) % 3])] = t;
}
// 创建逆时针三角形并返回编号；要求 a,b,c 非共线，必要时交换 b,c。
int mk(int a, int b, int c) {
    if(ori(a, b, c) < 0) swap(b, c);
    int t = nw();
    tri[t].v[0] = a, tri[t].v[1] = b, tri[t].v[2] = c;
    tri[t].f[0] = tri[t].f[1] = tri[t].f[2] = -1, tri[t].ver = ++verc;
    return t;
}
typedef array<int, 3> Tst;  // 待检查的边:(三角形, 该边对面顶点在三角形里的下标, 三角形版本号)
// 消耗待检边栈并翻转非法边，更新邻接；凸包边不翻，过期版本跳过。
void flipall(vector<Tst> &st) {
    while(!st.empty()) {
        Tst e = st.back();
        st.pop_back();
```

```
        int t = e[0], i = e[1];
        if(!live[t] || tri[t].ver != e[2]) continue;
        int a = tri[t].v[(i + 1) % 3], b = tri[t].v[(i + 2) % 3], g = tri[t].f[i];
        if(g < 0) continue;
        int k = finde(g, b, a), q = tri[g].v[k];
        if(!ill(q, tri[t].v[0], tri[t].v[1], tri[t].v[2])) continue;
        int p = tri[t].v[i];
        int X = tri[t].f[(i + 1) % 3], Y = tri[t].f[(i + 2) % 3];
        int W = tri[g].f[(k + 1) % 3], Z = tri[g].f[(k + 2) % 3];
        tri[t].v[0] = p, tri[t].v[1] = a, tri[t].v[2] = q;
        tri[g].v[0] = p, tri[g].v[1] = q, tri[g].v[2] = b;
        lnk(t, 0, W);
        lnk(t, 2, Y);
        lnk(t, 1, g);
        lnk(g, 0, Z);
        lnk(g, 1, X);
        tri[t].ver = ++verc, tri[g].ver = ++verc;
        st.push_back({t, 0, tri[t].ver}), st.push_back({t, 2, tri[t].ver});
        st.push_back({g, 0, tri[g].ver}), st.push_back({g, 1, tri[g].ver});
    }
}

vector<p2> delaunay(vector<p2> a) {
    pts = move(a);
    tri.clear();
    live.clear();
    fre.clear();
    verc = 0;
    sort(pts.begin(), pts.end(), lt);
    pts.erase(unique(pts.begin(), pts.end(), [](p2 u, p2 v) { return u.x == v.x &&
u.y == v.y; }), pts.end());
    int n = pts.size();
    if(n < 3) return {};
    auto hb = convex_hull(pts);
    int k = hb.size();
    if(k < 3) return {};
    vector<int> hv(k), fs(k - 2);
    for(int j = 0; j < k; ++j) hv[j] = (int) (lower_bound(pts.begin(), pts.end(),
hb[j], lt) - pts.begin());
    vector<Tst> st;
    For(i, 1, k - 2) fs[i - 1] = mk(hv[0], hv[i], hv[i + 1]);
    For(i, 1, k - 3) lnk(fs[i - 1], 1, fs[i]);
    for(int t = 0; t < (int) tri.size(); ++t)
        for(int i = 0; i < 3; ++i) st.push_back({t, i, tri[t].ver});
    flipall(st);
    vector<int> in;
    vector<char> onh(n, 0);
    for(int j = 0; j < k; ++j) onh[hv[j]] = 1;
    for(int i = 0; i < n; ++i) if(!onh[i]) in.push_back(i);
    mt19937 rng(20240915);
    shuffle(in.begin(), in.end(), rng);
    int last = fs[0];
    for(int ii = 0; ii < (int) in.size(); ++ii) {
        int p = in[ii], cur = last, go, side[3];
        for(;;) {
            go = -1;
            for(int j = 0; j < 3; ++j) {
                side[j] = ori(tri[cur].v[(j + 1) % 3], tri[cur].v[(j + 2) % 3], p);
                if(side[j] < 0) { go = j; break; }
            }
            if(go < 0 || tri[cur].f[go] < 0) break;
            cur = tri[cur].f[go];
        }

        for(int j = go + 1; go >= 0 && j < 3; ++j)
            side[j] = ori(tri[cur].v[(j + 1) % 3], tri[cur].v[(j + 2) % 3], p);
        int e = -1;
        for(int j = 0; j < 3; ++j) if(!side[j]) { e = j; break; }
        st.clear();
        if(e < 0) {
            int a = tri[cur].v[0], b = tri[cur].v[1], c = tri[cur].v[2];
            int f0 = tri[cur].f[0], f1 = tri[cur].f[1], f2 = tri[cur].f[2];
            rm(cur);
            int t1 = mk(a, b, p), t2 = mk(b, c, p), t3 = mk(c, a, p);
            lnk(t1, 2, f2);
            lnk(t2, 2, f0);
            lnk(t3, 2, f1);
            lnk(t1, 0, t2);
            lnk(t2, 0, t3);
            lnk(t3, 0, t1);
            st.push_back({t1, 2, tri[t1].ver}), st.push_back({t2, 2, tri[t2].ver}),
st.push_back({t3, 2, tri[t3].ver});
            last = t1;
        } else {
            int a = tri[cur].v[(e + 1) % 3], b = tri[cur].v[(e + 2) % 3], c =
tri[cur].v[e];
            int g = tri[cur].f[e];
            int fc1 = tri[cur].f[(e + 1) % 3], fc2 = tri[cur].f[(e + 2) % 3];
            if(g < 0) {
                rm(cur);
                int t1 = mk(c, a, p), t2 = mk(c, p, b);
                lnk(t1, 1, t2);
                lnk(t1, 2, fc2);
                lnk(t2, 1, fc1);
                st.push_back({t1, 2, tri[t1].ver}), st.push_back({t2, 1,
tri[t2].ver});
                last = t1;
            } else {
                int kk = finde(g, b, a), d = tri[g].v[kk];
                int fn1 = tri[g].f[(kk + 1) % 3], fn2 = tri[g].f[(kk + 2) % 3];
                rm(cur);
                rm(g);
                int t1 = mk(c, a, p), t2 = mk(c, p, b), u1 = mk(d, b, p), u2 = mk(d,
p, a);
                lnk(t1, 0, u2);
                lnk(t1, 1, t2);
```

```
                lnk(t2, 0, u1);
                lnk(u1, 1, u2);
                lnk(t1, 2, fc2);
                lnk(t2, 1, fc1);
                lnk(u1, 2, fn2);
                lnk(u2, 1, fn1);
                st.push_back({t1, 2, tri[t1].ver}), st.push_back({t2, 1,
    tri[t2].ver});
                st.push_back({u1, 2, tri[u1].ver}), st.push_back({u2, 1,
    tri[u2].ver});
                last = t1;
            }
        }
        flipall(st);
    }
    vector<p2> res;
    for(int t = 0; t < (int) tri.size(); ++t) if(live[t]) {
        int a = tri[t].v[0], b = tri[t].v[1], c = tri[t].v[2];
        res.push_back(pts[a]), res.push_back(pts[b]), res.push_back(pts[c]);
    }
    return res;
}

}  // namespace Delaunay
using namespace Delaunay;
}  // namespace Geo
```

7.7.2 Voronoi 图

```
// 依赖 凸包/凸包.cpp 的 convex_cut，不依赖 Delaunay。
// 逐站点用中垂线裁包围盒，返回每个站点的逆时针区域；最坏 O(n³) 时间、O(n²) 空间。
namespace Geo {
namespace Voronoi {

array<p2, 2> box;  // 裁剪用的包围盒:box[0] 左下、box[1] 右上

// ab 中垂线，a 在左侧；要求 a!=b。
seg bisector(p2 a, p2 b) {
    p2 m = (a + b) / 2;
    return {m, m + r90(b - a)};
}

vector<vector<p2>> voronoi(const vector<p2> &a) {
    int n = a.size();
    vector<vector<p2>> res;
    if(!n) return res;
    db x1 = a[0].x, x2 = a[0].x, y1 = a[0].y, y2 = a[0].y;
    for(int i = 1; i < n; ++i) x1 = min(x1, a[i].x), x2 = max(x2, a[i].x), y1 =
  min(y1, a[i].y), y2 = max(y2, a[i].y);
    db r = max(x2 - x1, y2 - y1) + 1;
    box = {p2{x1 - r, y1 - r}, p2{x2 + r, y2 + r}};
    vector<p2> ini{{box[0].x, box[0].y}, {box[1].x, box[0].y},
                   {box[1].x, box[1].y}, {box[0].x, box[1].y}};
    for(int i = 0; i < n; ++i) {
        vector<p2> cur = ini;
        for(p2 &p : cur) p -= a[i];
        for(int j = 0; j < n; ++j) {
            if(j == i || a[i] == a[j] || cur.size() < 3) continue;
            cur = convex_cut(cur, bisector({0, 0}, a[j] - a[i]));
        }
        for(p2 &p : cur) p += a[i];
        res.push_back(move(cur));
    }
    return res;
}

}  // namespace Voronoi
using namespace Voronoi;
}  // namespace Geo
```

7.8 三维

7.8.1 三维向量

```
// 依赖 基础/二维向量.cpp 的 eps/sign/cmp，与 p2 同在 Geo 命名空间。
// 三维向量的加减/点积/叉积/模长/混合积；unit 与夹角类要求向量非零，判等用 eps。
namespace Geo {

struct p3 {
    db x, y, z;
    bool operator==(const p3 &b) const {
        return !cmp(x, b.x) && !cmp(y, b.y) && !cmp(z, b.z);
    }
    bool operator!=(const p3 &b) const { return !(*this == b); }
    bool operator<(const p3 &b) const {
        return x != b.x ? x < b.x : y != b.y ? y < b.y : z < b.z;
    }
};
p3 operator+(p3 a, p3 b) { return {a.x + b.x, a.y + b.y, a.z + b.z}; }
p3 operator-(p3 a, p3 b) { return {a.x - b.x, a.y - b.y, a.z - b.z}; }
p3 operator-(p3 a) { return {-a.x, -a.y, -a.z}; }
p3 operator*(p3 a, db k) { return {a.x * k, a.y * k, a.z * k}; }
p3 operator*(db k, p3 a) { return {a.x * k, a.y * k, a.z * k}; }
p3 operator/(p3 a, db k) { return {a.x / k, a.y / k, a.z / k}; }
p3 operator+=(p3 &a, p3 b) { return a = a + b; }
p3 operator-=(p3 &a, p3 b) { return a = a - b; }
p3 operator*=(p3 &a, db k) { return a = a * k; }
p3 operator/=(p3 &a, db k) { return a = a / k; }
db dot(p3 a, p3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
p3 cross(p3 a, p3 b) {
    return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
}
// 返回混合积 a·(b×c)，即平行六面体带号体积。
db det(p3 a, p3 b, p3 c) { return dot(a, cross(b, c)); }
db dis2(p3 a) { return dot(a, a); }
db dis(p3 a) { return hypot(a.x, a.y, a.z); }
```

```
p3 unit(p3 a) {
    a /= max({abs(a.x), abs(a.y), abs(a.z)});
    return a / dis(a);
}
db cosang(p3 a, p3 b) { return max((db) -1, min((db) 1, dot(unit(a), unit(b)))); }
db sinang(p3 a, p3 b) { return min((db) 1, dis(cross(unit(a), unit(b)))); }
// 夹角，弧度范围 [0,pi]。
db angle(p3 a, p3 b) {
    a /= max({abs(a.x), abs(a.y), abs(a.z)});
    b /= max({abs(b.x), abs(b.y), abs(b.z)});
    return atan2(dis(cross(a, b)), dot(a, b));
}
// 返回三角形 abc 面积的 2 倍。
db area2(p3 a, p3 b, p3 c) { return dis(cross(b - a, c - a)); }
// 返回四面体 abcd 带号体积的 6 倍。
db volume6(p3 a, p3 b, p3 c, p3 d) { return det(b - a, c - a, d - a); }
// 返回与 a 垂直的向量；a 非零时结果也非零。
p3 perp(p3 a) {
    return abs(a.x) > abs(a.z) ? p3{a.y, -a.x, 0} : p3{0, -a.z, a.y};
}
bool colinear(p3 a, p3 b, p3 c) { return cmp(dis(cross(b - a, c - a)), 0) == 0; }
bool coplanar(p3 a, p3 b, p3 c, p3 d) { return sign(abs(det(b - a, c - a, d - a)))
  == 0; }

}  // namespace Geo
using namespace Geo;
```

7.8.2 三维直线与线段

```
// 依赖 三维/三维向量.cpp。line3 存 o=a、d=b-a（d 非零、不必单位化）。
// 点线距、线线距与线段相交；用绝对长度 eps 与夹角正弦 eps，近平行时最近点病态。
namespace Geo {

struct line3 {
    p3 o, d;
    line3() {}
    line3(p3 a, p3 b) : o(a), d(b - a) {}
    p3 at(db t) const { return o + d * t; }
};
struct seg3 {
    p3 x, y;
    p3 dir() const { return y - x; }
};

p3 proj(const line3 &l, p3 p) {
    p3 e = unit(l.d);
    return l.o + e * dot(e, p - l.o);
}
p3 reflect(const line3 &l, p3 p) { return proj(l, p) * 2 - p; }
db dis(const line3 &l, p3 p) { return dis(cross(unit(l.d), p - l.o)); }
bool ons(const line3 &l, p3 p) { return !sign(dis(l, p)); }
bool ispara(const line3 &a, const line3 &b) { return !sign(sinang(a.d, b.d)); }
// 零长线段约定与任意线段平行。
bool ispara(const seg3 &a, const seg3 &b) {
    return dis(a.dir()) == 0 || dis(b.dir()) == 0 || !sign(sinang(a.dir(),
  b.dir()));
}
bool isperp(const line3 &a, const line3 &b) { return !sign(cosang(a.d, b.d)); }

// pa 在 a 上、pb 在 b 上；按 eps 判平行时取 pa=a.o，最近点对可不唯一。
db dis(const line3 &a, const line3 &b, p3 &pa, p3 &pb) {
    p3 e = unit(a.d), f = unit(b.d), n = cross(e, f), w = b.o - a.o;
    db sn = dis(n);
    if(!sign(sn)) {
        pa = a.o, pb = proj(b, pa);
    } else {
        n /= sn;
        pa = a.o + e * (dot(cross(w, f), n) / sn);
        pb = b.o + f * (dot(cross(w, e), n) / sn);
    }
    return dis(pa - pb);
}
db dis(const line3 &a, const line3 &b) { p3 pa, pb; return dis(a, b, pa, pb); }
p3 closest_on(const line3 &a, const line3 &b) { p3 pa, pb; return dis(a, b, pa, pb),
  pa; }

// 点在线段上的最近点；短但非零的线段不退化。
p3 proj(const seg3 &s, p3 p) {
    p3 d = s.dir();
    db len = dis(d);
    if(len == 0) return s.x;
    p3 e = unit(d);
    db t = dot(e, p - s.x);
    if(t <= 0) return s.x;
    if(t >= len) return s.y;
    return s.x + e * t;
}
db dis(const seg3 &s, p3 p) { return dis(proj(s, p) - p); }
bool ons(const seg3 &s, p3 p) { return !sign(dis(s, p)); }
// 严格内部：沿线段方向到两端均超过 eps；零长段无内部。
bool ons_s(const seg3 &s, p3 p) {
    if(dis(s.dir()) == 0) return false;
    p3 e = unit(s.dir());
    return ons(s, p) && sign(dot(e, p - s.x)) > 0 && sign(dot(e, s.y - p)) > 0;
}

// 返回最短距离及段上的最近点 pa、pb；包括点段、平行段。
db dis(const seg3 &a, const seg3 &b, p3 &pa, p3 &pb) {
    db la = dis(a.dir()), lb = dis(b.dir());
    p3 e = la == 0 ? p3{0, 0, 0} : unit(a.dir());
    p3 f = lb == 0 ? p3{0, 0, 0} : unit(b.dir());

    auto project = [](const seg3 &s, p3 dir, db len, p3 p) {
        if(len == 0) return s.x;
        db t = dot(dir, p - s.x);
        if(t <= 0) return s.x;
```

```
        if(t >= len) return s.y;
        return s.x + dir * t;
    };
    db best = dis(a.x - b.x);
    pa = a.x, pb = b.x;
    auto upd = [&](p3 x, p3 y) {
        db d = dis(x - y);
        if(d < best) best = d, pa = x, pb = y;
    };
    upd(a.x, project(b, f, lb, a.x)), upd(a.y, project(b, f, lb, a.y));
    upd(project(a, e, la, b.x), b.x), upd(project(a, e, la, b.y), b.y);
    if(la == 0 || lb == 0) return best;
    p3 n = cross(e, f), w = b.x - a.x;
    db sn = dis(n);
    if(sn > 0) {
        n /= sn;
        db s = dot(cross(w, f), n) / sn, t = dot(cross(w, e), n) / sn;
        if(s >= 0 && s <= la && t >= 0 && t <= lb) upd(a.x + e * s, b.x + f * t);
    }
    return best;
}
db dis(const seg3 &a, const seg3 &b) { p3 pa, pb; return dis(a, b, pa, pb); }

// 含端点相接、共线重叠、点段；最短距离 <= eps 即相交。
bool chkss(const seg3 &a, const seg3 &b) { return !sign(dis(a, b)); }
// 严格相交：非平行、最近点均在严格内部；共线/点段/端点不算。
bool chkss_s(const seg3 &a, const seg3 &b) {
    if(ispara(a, b)) return false;
    p3 pa, pb;
    return !sign(dis(a, b, pa, pb)) && ons_s(a, pa) && ons_s(b, pb);
}
// 唯一公共点（eps 意义，见头部）；失败不修改 p，成功取 a 上的最近点。
bool isss(const seg3 &a, const seg3 &b, p3 &p) {
    p3 pa, pb;
    if(sign(dis(a, b, pa, pb))) return false;
    db la = dis(a.dir()), lb = dis(b.dir());
    if(la > 0 && lb > 0 && ispara(a, b)) {
        p3 e = unit(a.dir());
        db u = dot(e, b.x - a.x), v = dot(e, b.y - a.x);
        if(sign(min(la, max(u, v)) - max((db) 0, min(u, v))) > 0) return false;
    }
    return p = pa, true;
}

}  // namespace Geo
using namespace Geo;
```

7.8.3 三维平面

```
// 依赖 三维/三维直线.cpp（及其三维向量依赖）。
// 点法式平面（n 为单位法向）：带号距离、两平面交线、线面交；eps 为绝对距离容差。
namespace Geo {

struct plane {
    p3 n;
    db d;
    plane() {}
    // 过三点，法向 (b-a)×(c-a)；a=b 或 c 到直线 ab 距离 <= eps 时 n=0。
    plane(p3 a, p3 b, p3 c) {
        p3 u = b - a;
        n = {0, 0, 0}, d = 0;
        if(Geo::dis(u) == 0) return;
        p3 t = cross(unit(u), c - a);
        if(!sign(Geo::dis(t))) return;
        n = unit(t), d = dot(n, a);
    }
    // 过点 o、法向 nor 的平面(nor 非零,内部单位化)。
    static plane from(p3 o, p3 nor) {
        plane r;
        r.n = unit(nor), r.d = dot(r.n, o);
        return r;
    }
    db side(p3 p) const { return dot(n, p) - d; }
    db dis(p3 p) const { return abs(side(p)); }
    p3 proj(p3 p) const { return p - n * side(p); }
    p3 reflect(p3 p) const { return p - n * (2 * side(p)); }
    bool ons(p3 p) const { return !sign(side(p)); }
};
bool ispara(const plane &a, const plane &b) { return !sign(sinang(a.n, b.n)); }
bool isperp(const plane &a, const plane &b) { return !sign(cosang(a.n, b.n)); }
// 返回两平面的锐夹角，弧度范围 [0, pi/2]。
db angle(const plane &a, const plane &b) { return acos(max((db) -1, min((db) 1,
abs(dot(a.n, b.n))))); }
// 返回交线；按 eps 判平行（含重合）时返回 d=0 的退化直线。
line3 ispl(const plane &a, const plane &b) {
    p3 w = cross(a.n, b.n);
    if(!sign(sinang(a.n, b.n))) return line3(a.proj(b.n * b.d), a.proj(b.n * b.d));
    p3 o = (cross(b.n, w) * a.d + cross(w, a.n) * b.d) / dis2(w);
    return line3(o, o + w);
}
// 返回直线与平面的唯一交点。
p3 islp(const plane &p, const line3 &l) { return l.o + l.d * ((p.d - dot(p.n,
l.o)) / dot(p.n, l.d)); }
line3 perpthrough(const plane &p, p3 o) { return line3(o, o + p.n); }
// 直线是否整个在平面内
bool ons(const plane &p, const line3 &l) { return !sign(p.side(l.o)) && !
sign(dot(p.n, l.d)); }
db dis(const plane &p, p3 x) { return p.dis(x); }
p3 proj(const plane &p, p3 x) { return p.proj(x); }

}  // namespace Geo
using namespace Geo;
```

7.8.4 三维旋转

```
// 依赖 三维/三维向量.cpp 的 unit/cross/perp，角度用弧度。
```

```cpp
// 罗德里格斯旋转、旋转矩阵与坐标基底；轴与法向须非零，内部安全单位化。
namespace Geo {

struct mat3 {
    db a[3][3];
    p3 operator*(p3 v) const {
        return {a[0][0] * v.x + a[0][1] * v.y + a[0][2] * v.z,
                a[1][0] * v.x + a[1][1] * v.y + a[1][2] * v.z,
                a[2][0] * v.x + a[2][1] * v.y + a[2][2] * v.z};
    }
};
mat3 operator*(const mat3 &x, const mat3 &y) {
    mat3 r{};
    for(int i = 0; i < 3; ++i)
        for(int j = 0; j < 3; ++j) {
            db s = 0;
            for(int k = 0; k < 3; ++k) s += x.a[i][k] * y.a[k][j];
            r.a[i][j] = s;
        }
    return r;
}
mat3 transpose(const mat3 &m) {
    mat3 r{};
    for(int i = 0; i < 3; ++i)
        for(int j = 0; j < 3; ++j) r.a[i][j] = m.a[j][i];
    return r;
}
// 绕过原点、方向为 axis 的轴作右手旋转。
p3 rot(p3 v, p3 axis, db ang) {
    p3 k = unit(axis), c = cross(k, v), d = k * dot(k, v);
    return v * cos(ang) + c * sin(ang) + d * (1 - cos(ang));
}
// 绕过点 o、方向为 axis 的轴旋转。
p3 rot(p3 v, p3 axis, db ang, p3 o) { return o + rot(v - o, axis, ang); }
mat3 rotmat(p3 axis, db ang) {
    p3 k = unit(axis);
    db c = cos(ang), s = sin(ang), t = 1 - c;
    mat3 m{};
    m.a[0][0] = t * k.x * k.x + c;
    m.a[0][1] = t * k.x * k.y - s * k.z;
    m.a[0][2] = t * k.x * k.z + s * k.y;
    m.a[1][0] = t * k.x * k.y + s * k.z;
    m.a[1][1] = t * k.y * k.y + c;
    m.a[1][2] = t * k.y * k.z - s * k.x;
    m.a[2][0] = t * k.x * k.z - s * k.y;
    m.a[2][1] = t * k.y * k.z + s * k.x;
    m.a[2][2] = t * k.z * k.z + c;
    return m;
}
// 返回右手单位正交基，w 与 n 同向。
void basis(p3 n, p3 &u, p3 &v, p3 &w) {
    w = unit(n), u = unit(perp(w)), v = cross(w, u);
}
// 世界坐标转局部坐标；以 o 为原点，u/v/w 须为单位正交基。
p3 tolocal(p3 x, p3 o, p3 u, p3 v, p3 w) {
    p3 d = x - o;
    return {dot(u, d), dot(v, d), dot(w, d)};
}
// 局部坐标转世界坐标，与 tolocal 使用同一原点和基。
p3 toworld(p3 l, p3 o, p3 u, p3 v, p3 w) { return o + u * l.x + v * l.y + w * l.z; }

}  // namespace Geo
using namespace Geo;
```

7.8.5 三维凸包

```cpp
// 依赖 三维/三维平面.cpp（及其直线、向量依赖）。
// 增量法三维凸包，期望 O(n²)；返回朝外三角面片（face3 存输入下标），退化时返回空。
namespace Geo {

struct face3 {
    int a, b, c;
};

vector<face3> convex3d(const vector<p3> &input) {
    int n = input.size();
    vector<face3> ret;
    if(n < 4) return ret;
    vector<p3> a(n);
    for(int i = 0; i < n; ++i) a[i] = input[i] - input[0];

    int A = 0, B = -1, C = -1, D = -1;
    for(int i = 0; i < n; ++i) if(sign(dis(a[i] - a[A]))) { B = i; break; }
    if(B < 0) return ret;
    p3 w0 = a[B] - a[A];
    for(int i = 0; i < n; ++i) if(sign(dis(cross(a[i] - a[A], w0)) / dis(w0))) { C = i; break; }
    if(C < 0) return ret;
    plane p0(a[A], a[B], a[C]);
    for(int i = 0; i < n; ++i) if(sign(abs(p0.side(a[i])))) { D = i; break; }
    if(D < 0) return ret;

    vector<int> id;
    id.push_back(A), id.push_back(B), id.push_back(C), id.push_back(D);
    for(int i = 0; i < n; ++i) if(i != A && i != B && i != C && i != D) id.push_back(i);
    vector<p3> b(n);
    for(int i = 0; i < n; ++i) b[i] = a[id[i]];
    struct F {
        int a, b, c;
        plane p;
    };
    vector<F> f;

    f.push_back(F{0, 1, 2, p0});
    p0.n = -p0.n, p0.d = -p0.d;
```

```
    f.push_back(F{0, 2, 1, p0});
    unordered_map<ll, int> vis;
    For(i, 3, n - 1) {
        vector<F> keep, kill;
        for(int t = 0; t < f.size(); ++t) {
            if(f[t].p.side(b[i]) > eps) kill.push_back(f[t]);
            else keep.push_back(f[t]);
        }
        if(kill.empty()) continue;
        int e[3][2];
        for(int t = 0; t < kill.size(); ++t) {
            e[0][0] = kill[t].a, e[0][1] = kill[t].b;
            e[1][0] = kill[t].b, e[1][1] = kill[t].c;
            e[2][0] = kill[t].c, e[2][1] = kill[t].a;
            for(int k = 0; k < 3; ++k) vis[(ll) e[k][0] * n + e[k][1]] = i;
        }

        for(int t = 0; t < kill.size(); ++t) {
            e[0][0] = kill[t].a, e[0][1] = kill[t].b;
            e[1][0] = kill[t].b, e[1][1] = kill[t].c;
            e[2][0] = kill[t].c, e[2][1] = kill[t].a;
            for(int k = 0; k < 3; ++k)
                if(vis[(ll) e[k][0] * n + e[k][1]] == i && vis[(ll) e[k][1] * n + e[k][0]] != i)
                    keep.push_back(F{e[k][0], e[k][1], i, plane(b[e[k][0]], b[e[k][1]], b[i])});
        }
        f.swap(keep);
    }
    for(int t = 0; t < f.size(); ++t) ret.push_back(face3{id[f[t].a], id[f[t].b], id[f[t].c]});
    return ret;
}

// 返回面片总面积，与朝向无关。
db hull_area(const vector<p3> &a, const vector<face3> &f) {
    db s = 0;
    for(int t = 0; t < f.size(); ++t) s += area2(a[f[t].a], a[f[t].b], a[f[t].c]);
    return s / 2;
}
// 凸包体积；f 须闭合且朝向一致，局部三重积须有限；空面集返回 0。
db hull_volume(const vector<p3> &a, const vector<face3> &f) {
    if(f.empty()) return 0;
    p3 o = a[f[0].a];
    db s = 0;
    for(int t = 0; t < f.size(); ++t) s += det(a[f[t].a] - o, a[f[t].b] - o, a[f[t].c] - o);
    return abs(s) / 6;
}

}  // namespace Geo
using namespace Geo;
```

8 通用

8.1 头文件

```
// 基础头：后面所有模板都假定这层已在上方。
// 提供 For/rFor/all/cmin/cmax/mtc 宏、类型别名、模数 M 与 qpow 两参/三参。

#include <bits/stdc++.h>
#define For(i, l, r) for(int i = l, i##_e = r; i <= i##_e; ++i)
#define rFor(i, r, l) for(int i = r, i##_e = l; i >= i##_e; --i)
#define all(a) a.begin(), a.end()
#define cmin(a, b) a = min<remove_reference<decltype(a)>::type>(a, b)
#define cmax(a, b) a = max<remove_reference<decltype(a)>::type>(a, b)
#define mtc() \
    int T;    \
    cin >> T; \
    while(T--) work();
using namespace std;
using ll = long long;
using ull = unsigned long long;
using i64 = long long;
using u64 = unsigned long long;
using i128 = __int128;
using u128 = __uint128_t;
using db = long double;
using pii = array<int, 2>;
using pll = array<ll, 2>;
// 普通多项式与逆元统一用这个模数；系数保持 [0, M)。
constexpr int M = 998244353;
// 返回 a^b mod M；要求 a,b >= 0，b=0 返回 1。
// 模数是编译期常量，乘模不走路除法，热路径比三参版本快。
ll qpow(ll a, ll b) {
    ll s = 1;
    a %= M;
    for(; b; b >>= 1, a = a * a % M)
        if(b & 1) s = s * a % M;
    return s;
}
// 返回 a^b mod p；要求 a,b >= 0、1 <= p <= 3037000500（ll 模乘安全界）。
// b=0 返回 1（含 p=1）；模数不是 M 时用（如原根/BSGS 的输入 p）。
ll qpow(ll a, ll b, ll p) {
    ll s = 1;
    a %= p;
    for(; b; b >>= 1, a = a * a % p)
        if(b & 1) s = s * a % p;
    return s;
}
```

8.2 大质数表

每个数量级取 10^k 以上最小的 10 个素数横着排,表里只写偏移 x(读作 10^k+x)。

10^k	1	2	3	4	5	6	7	8	9	10
10^2	+1	+3	+7	+9	+13	+27	+31	+37	+39	+49
10^3	+9	+13	+19	+21	+31	+33	+39	+49	+51	+61
10^4	+7	+9	+37	+39	+61	+67	+69	+79	+91	+93
10^5	+3	+19	+43	+49	+57	+69	+103	+109	+129	+151
10^6	+3	+33	+37	+39	+81	+99	+117	+121	+133	+151
10^7	+19	+79	+103	+121	+139	+141	+169	+189	+223	+229
10^8	+7	+37	+39	+49	+73	+81	+123	+127	+193	+213
10^9	+7	+9	+21	+33	+87	+93	+97	+103	+123	+181
10^{10}	+19	+33	+61	+69	+97	+103	+121	+141	+147	+207
10^{11}	+3	+19	+57	+63	+69	+73	+91	+103	+129	+171
10^{12}	+39	+61	+63	+91	+121	+163	+169	+177	+189	+193
10^{13}	+37	+51	+99	+129	+183	+259	+267	+273	+279	+283
10^{14}	+31	+67	+97	+99	+133	+139	+169	+183	+261	+357
10^{15}	+37	+91	+159	+187	+223	+241	+249	+259	+273	+279
10^{16}	+61	+69	+79	+99	+453	+481	+597	+613	+639	+669
10^{17}	+3	+13	+19	+21	+49	+81	+99	+141	+181	+337
10^{18}	+3	+9	+31	+79	+177	+183	+201	+283	+381	+387

8.3 常数速查表

n	1	2	3	4	5	6	7	8	9	10
log10 n	0	0.30103	0.477121	0.60206	0.69897	0.778151	0.845098	0.90309	0.954243	1
n!	1	2	6	24	120	720	5040	40320	362880	3628800
C(n, n/2)	1	2	3	6	10	20	35	70	126	252
lcm(1..n)	1	2	6	12	60	60	420	840	2520	2520
P_n	1	2	3	5	7	11	15	22	30	42
n	**11**	**12**	**13**	**14**	**15**	**16**	**17**	**18**	**19**	**20**
log10 n	1.041393	1.079181	1.113943	1.146128	1.176091	1.20412	1.230449	1.255273	1.278754	1.30103
n!	39916800	479001600	6.23e9	8.72e10	1.31e12	2.09e13	3.56e14	6.40e15	1.22e17	2.43e18
C(n, n/2)	462	924	1716	3432	6435	12870	24310	48620	92378	184756
lcm(1..n)	27720	27720	360360	360360	360360	720720	12252240	12252240	232792560	232792560
P_n	56	77	101	135	176	231	297	385	490	627
n	**21**	**22**	**23**	**24**	**25**	**26**	**27**	**28**	**29**	**30**
log10 n	1.322219	1.342423	1.361728	1.380211	1.39794	1.414973	1.431364	1.447158	1.462398	1.477121
n!	5.11e19	1.12e21	2.59e22	6.20e23	1.55e25	4.03e26	1.09e28	3.05e29	8.84e30	2.65e32
C(n, n/2)	352716	705432	1352078	2704156	5200300	10400600	20058300	40116600	77558760	155117520
lcm(1..n)	232792560	232792560	5.35e9	5.35e9	2.68e10	2.68e10	8.03e10	8.03e10	2.33e12	2.33e12
P_n	792	1002	1255	1575	1958	2436	3010	3718	4565	5604

P_n 超过 10^9 的最小 $n=115$;$P_{40}=37338$、$P_{50}=204226$、$P_{60}=966467$、$P_{100}=190569292$。

卡特兰数 $C_n=\frac{1}{n+1}\binom{2n}{n}=\binom{2n}{n}-\binom{2n}{n+1}$,递推 $C_n=C_{n-1}\frac{4n-2}{n+1}$;$C_{35}<2^{63}$($n\le 35$ 都在 64 位整数内),表里 $n\ge 19$ 起改写成 3 位有效数字的科学记数法。

n	0	1	2	3	4	5	6	7	8	9
C_n	1	1	2	5	14	42	132	429	1430	4862
n	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**	**18**	**19**
C_n	16796	58786	208012	742900	2674440	9694845	35357670	129644790	477638700	1.77e9
n	**20**	**21**	**22**	**23**	**24**	**25**	**26**	**27**	**28**	**29**
C_n	6.56e9	2.45e10	9.15e10	3.43e11	1.29e12	4.86e12	1.84e13	6.95e13	2.64e14	1.00e15
n	**30**	**31**	**32**	**33**	**34**	**35**	**36**	**37**	**38**	**39**
C_n	3.81e15	1.45e16	5.55e16	2.12e17	8.13e17	3.12e18	1.20e19	4.60e19	1.77e20	6.80e20

10^k 以内的增长上界:ω 是不同质因子个数,d 是因子个数,π(n) 是不超过 n 的素数个数。

$n\le$	max ω(n)	max d(n)	π(n)	$n\le$	max ω(n)	max d(n)	π(n)
10^1	2	4	4	10^{10}	10	2304	4.55e8
10^2	3	12	25	10^{11}	10	4032	4.12e9
10^3	4	32	168	10^{12}	11	6720	3.7e10
10^4	5	64	1229	10^{13}	12	10752	—
10^5	6	128	9592	10^{14}	12	17280	—
10^6	7	240	78498	10^{15}	13	26880	—
10^7	8	448	664579	10^{16}	13	41472	—
10^8	8	768	5761455	10^{17}	14	64512	—
10^9	9	1344	5.08e7	10^{18}	15	103680	—

素数定理:$\pi(x)\approx\frac{x}{\ln x}$;— 表示未列出值。

高合成数:从小到大,因子个数刷新纪录的 n;左右两组依次往下读。

n	分解	d(n)	n	分解	d(n)
2	2	2	6983776800	$2^5\cdot3^3\cdot5^2\cdot7\cdot11\cdot13\cdot17\cdot19$	2304
4	2^2	3	13967553600	$2^6\cdot3^3\cdot5^2\cdot7\cdot11\cdot13\cdot17\cdot19$	2688
6	$2\cdot3$	4	27935107200	$2^7\cdot3^3\cdot5^2\cdot7\cdot11\cdot13\cdot17\cdot19$	3072
12	$2^2\cdot3$	6	64250746560	$2^6\cdot3^3\cdot5\cdot7\cdot11\cdot13\cdot17\cdot19\cdot23$	3584
24	$2^3\cdot3$	8	97772875200	$2^6\cdot3^3\cdot5^2\cdot7^2\cdot11\cdot13\cdot17\cdot19$	4032
60	$2^2\cdot3\cdot5$	12	128501493120	$2^7\cdot3^3\cdot5\cdot7\cdot11\cdot13\cdot17\cdot19\cdot23$	4096
120	$2^3\cdot3\cdot5$	16	240940299600	$2^4\cdot3^4\cdot5^2\cdot7\cdot11\cdot13\cdot17\cdot19\cdot23$	4800
240	$2^4\cdot3\cdot5$	20	481880599200	$2^5\cdot3^4\cdot5^2\cdot7\cdot11\cdot13\cdot17\cdot19\cdot23$	5760
360	$2^3\cdot3^2\cdot5$	24	963761198400	$2^6\cdot3^4\cdot5^2\cdot7\cdot11\cdot13\cdot17\cdot19\cdot23$	6720
840	$2^3\cdot3\cdot5\cdot7$	32	1927522396800	$2^7\cdot3^4\cdot5^2\cdot7\cdot11\cdot13\cdot17\cdot19\cdot23$	7680
1680	$2^4\cdot3\cdot5\cdot7$	40	3373164194400	$2^5\cdot3^4\cdot5^2\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23$	8640
2520	$2^3\cdot3^2\cdot5\cdot7$	48	6746328388800	$2^6\cdot3^4\cdot5^2\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23$	10080
7560	$2^3\cdot3^3\cdot5\cdot7$	64	9316358251200	$2^6\cdot3^3\cdot5^2\cdot7\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29$	10752
15120	$2^4\cdot3^3\cdot5\cdot7$	80	13492656777600	$2^7\cdot3^4\cdot5^2\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23$	11520
27720	$2^3\cdot3^2\cdot5\cdot7\cdot11$	96	32607253879200	$2^5\cdot3^3\cdot5^2\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29$	13824
55440	$2^4\cdot3^2\cdot5\cdot7\cdot11$	120	65214507758400	$2^6\cdot3^3\cdot5^2\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29$	16128
83160	$2^3\cdot3^3\cdot5\cdot7\cdot11$	128	97821761637600	$2^5\cdot3^4\cdot5^2\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29$	17280
110880	$2^5\cdot3^2\cdot5\cdot7\cdot11$	144	130429015516800	$2^7\cdot3^3\cdot5^2\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29$	18432
221760	$2^6\cdot3^2\cdot5\cdot7\cdot11$	168	260858031033600	$2^8\cdot3^3\cdot5^2\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29$	20736
498960	$2^4\cdot3^4\cdot5\cdot7\cdot11$	200	391287046550400	$2^7\cdot3^4\cdot5^2\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29$	23040
720720	$2^4\cdot3^2\cdot5\cdot7\cdot11\cdot13$	240	866421317361600	$2^6\cdot3^4\cdot5^2\cdot7\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29\cdot31$	26880
1441440	$2^5\cdot3^2\cdot5\cdot7\cdot11\cdot13$	288	1010824870255200	$2^5\cdot3^3\cdot5^2\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29\cdot31$	27648
3603600	$2^4\cdot3^2\cdot5^2\cdot7\cdot11\cdot13$	360	2021649740510400	$2^6\cdot3^3\cdot5^2\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29\cdot31$	32256
7207200	$2^5\cdot3^2\cdot5^2\cdot7\cdot11\cdot13$	432	4043299481020800	$2^7\cdot3^3\cdot5^2\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29\cdot31$	36864
8648640	$2^6\cdot3^3\cdot5\cdot7\cdot11\cdot13$	448	8086598962041600	$2^8\cdot3^3\cdot5^2\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29\cdot31$	41472
14414400	$2^6\cdot3^2\cdot5^2\cdot7\cdot11\cdot13$	504	12129898443062400	$2^7\cdot3^4\cdot5^2\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29\cdot31$	46080
32432400	$2^4\cdot3^4\cdot5^2\cdot7\cdot11\cdot13$	600	30324746107656000	$2^6\cdot3^4\cdot5^3\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29\cdot31$	53760
61261200	$2^4\cdot3^2\cdot5^2\cdot7\cdot11\cdot13\cdot17$	720	60649492215312000	$2^7\cdot3^4\cdot5^3\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29\cdot31$	61440
73513440	$2^5\cdot3^3\cdot5\cdot7\cdot11\cdot13\cdot17$	768	74801040398884800	$2^6\cdot3^3\cdot5^2\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29\cdot31\cdot37$	64512
122522400	$2^5\cdot3^2\cdot5^2\cdot7\cdot11\cdot13\cdot17$	864	112201560598327200	$2^5\cdot3^4\cdot5^2\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29\cdot31\cdot37$	69120
245044800	$2^6\cdot3^2\cdot5^2\cdot7\cdot11\cdot13\cdot17$	1008	224403121196654400	$2^6\cdot3^4\cdot5^2\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29\cdot31\cdot37$	80640
367567200	$2^5\cdot3^3\cdot5^2\cdot7\cdot11\cdot13\cdot17$	1152	448806242393308800	$2^7\cdot3^4\cdot5^2\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29\cdot31\cdot37$	92160
735134400	$2^6\cdot3^3\cdot5^2\cdot7\cdot11\cdot13\cdot17$	1344	897612484786617600	$2^8\cdot3^4\cdot5^2\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29\cdot31\cdot37$	103680
2095133040	$2^4\cdot3^4\cdot5\cdot7\cdot11\cdot13\cdot17\cdot19$	1600	1122015605983272000	$2^6\cdot3^4\cdot5^3\cdot7^2\cdot11\cdot13\cdot17\cdot19\cdot23\cdot29\cdot31\cdot37$	107520
3491888400	$2^4\cdot3^3\cdot5^2\cdot7\cdot11\cdot13\cdot17\cdot19$	1920			